

Azure Databricks For Data Engineers

Hands-on Project using Spark, Delta Lake, Unity Catalog and Lakeflow Jobs

Fully Refreshed (2026)





Ramesh Retnasamy

Data Engineer/ Machine Learning Engineer



<https://www.linkedin.com/in/ramesh-retnasamy/>

INSTRUCTOR

Ramesh Retnasamy . 200,000+ Learners

Best Selling Data & AI Engineering Instructor

Udemy Instructor Partner

239,342

Total learners

53,278

Reviews



udemy

Bestseller



Azure Databricks & Spark For Data Engineers: Hands-on Project

Real World Project on Formula1 Racing using Databricks,...

Ramesh Retnasamy . 200,000+ Learners

Course ★ 4.6 26,352 ratings

20.5 total hours 192 lectures

databricks

CERTIFIED

★ ★

Data Engineer Associate

Databricks Certified Data Engineer Associate - Ultimate Prep

UPDATED JULY 2025 Syllabus | Full Coverage of Databricks...

Ramesh Retnasamy . 200,000+ Learners

Course ★ 4.7 3,494 ratings

20 total hours 144 lectures All Levels

databricks

CERTIFIED

★ ★

Data Engineer Associate

Databricks Certified Data Engineer Associate- Practice Exams

UPDATED JULY 2025 Syllabus | Practice Exams with Detailed...

Ramesh Retnasamy . 200,000+ Learners

Practice tests Course ★ 4.5

923 ratings 100 questions All Levels

Bestseller



Azure Data Factory For Data Engineers - Project on Covid19

Real world project for Data Engineers using Azure Data...

Ramesh Retnasamy . 200,000+ Learners

Course ★ 4.6 17,490 ratings

12.5 total hours 145 lectures All Levels

Bestseller



Azure Synapse Analytics For Data Engineers - Hands On Project

Hands on Project for Data Engineers using all the service...

Ramesh Retnasamy . 200,000+ Learners

Course ★ 4.6 4,094 ratings

13.5 total hours 132 lectures All Levels



Comprehensive SQL Course For Data Professionals

Become a Confident SQL Developer by Developing a Re...

Ramesh Retnasamy . 200,000+ Learners

Course ★ 4.6 925 ratings

13.5 total hours 130 lectures All Levels



Azure Databricks



PySpark



Spark SQL



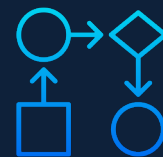
Azure Databricks



Delta Lake



Unity Catalog



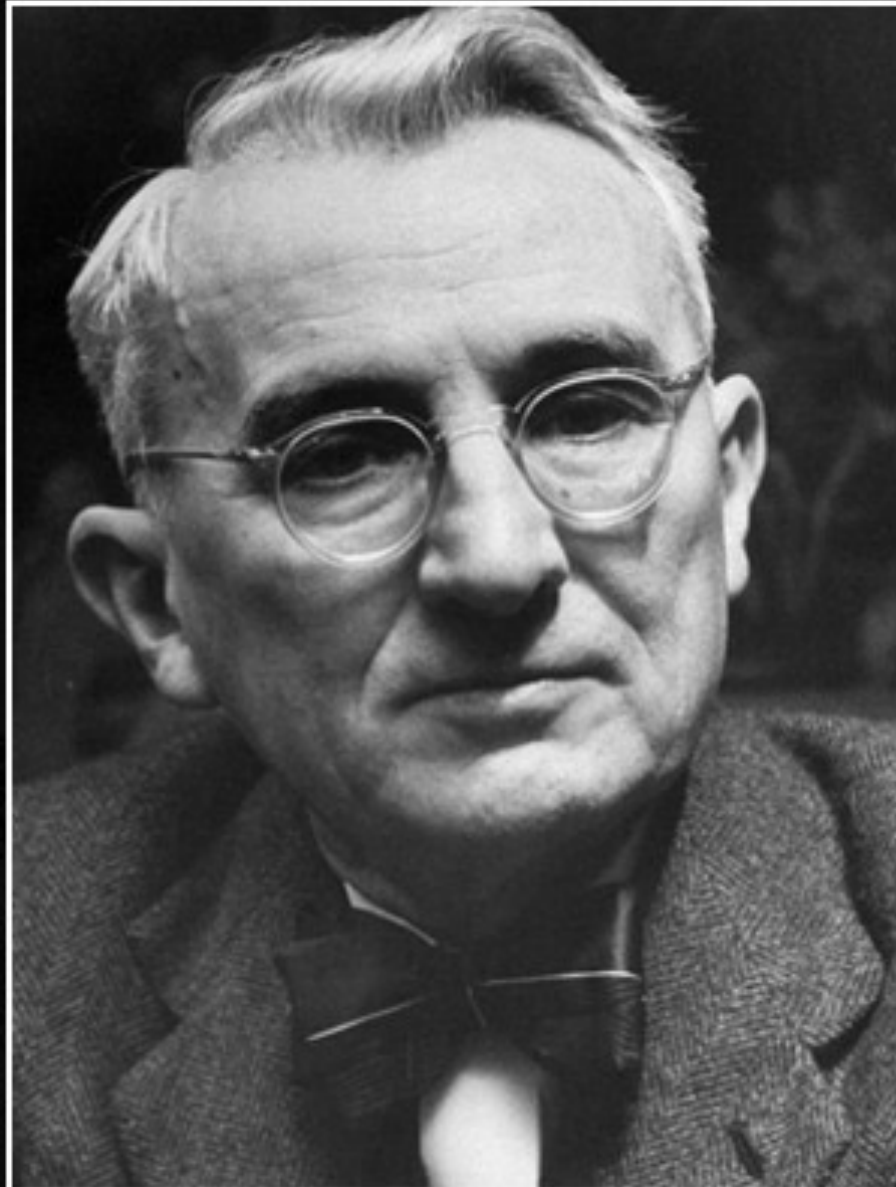
Lakeflow Jobs



Dashboards



Genie



Learning is an active process. We learn by doing.. Only knowledge that is used sticks in your mind.

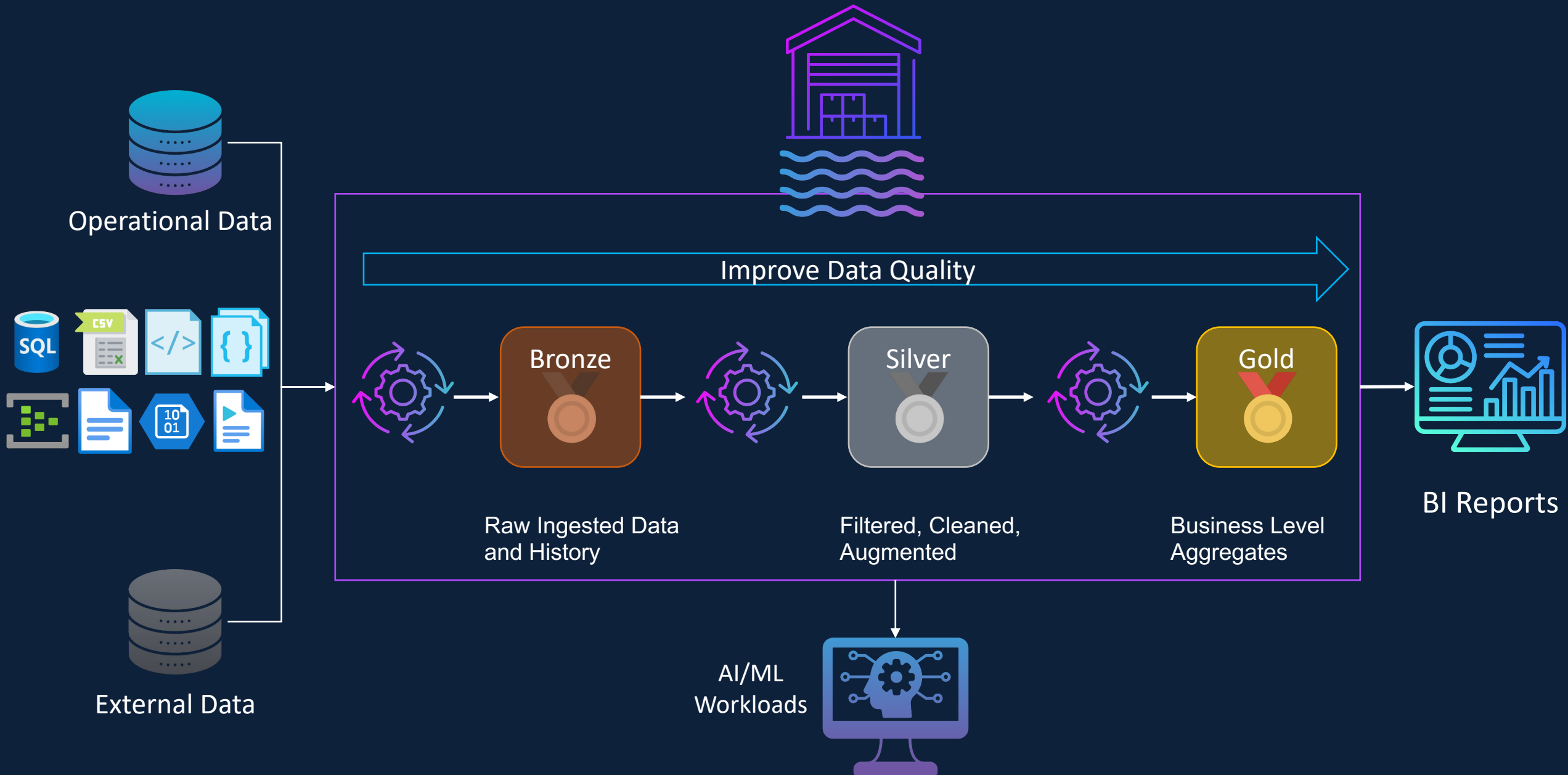
— Dale Carnegie —

AZ QUOTES



Formula1 Data Lakehouse

Solution Architecture



Microsoft Azure

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

AI/ML

Workspace

Tree view: ON

...

Type to search

Drafts

databricks-course

formula1-project

00-common

01-setup

02-bronze

01.Ingest Circuits File

02.Ingest Races File

03.Ingest Constructors File

04.Ingest Drivers File

05.Ingest Results File

06.Ingest Sprints File

03-silver

04-gold

05-analytics

formula1-project-incremental-load

introduction-to-delta-lake

introduction-to-notebooks

introduction-to-unity-catalog

z-course-images

01.Ingest Circuits File

File Edit View Run Help Python Tabs: ON Last edit was 51 days ago

Ingest circuits.csv file

1. Read the file using spark dataframe reader API

2. Add Metadata Columns

Source File

Ingestion Timestamp

3. Write to bronze delta table

Feb 12, 2026 (<1s)

2

%run ../00-common/01.environment-config

Feb 12, 2026 (<1s)

3

%run ../00-common/02.bronze-helpers

Feb 12, 2026 (<1s)

4

source_file = f"{landing_folder_path}/circuits.csv"
table_name = f"{catalog_name}.{bronze_schema}.circuits"

Feb 12, 2026 (<1s)

6

from pyspark.sql.types import StructType, StructField, StringType, DoubleType

circuits_schema = StructType([
 StructField('circuitId', StringType()),
 StructField('url', StringType()),

Step 1 - Read the CSV file using the dataframe reader API

https://adb-7405612240191638.18.azuredatabricks.net/editor/notebooks/913049566061301?o=7405612240...

Microsoft Azure

databricks

Search data, notebooks, recents, and more...

+ P

databricks-course-ws

R

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

AI/ML

Jobs & Pipelines

Create new

Ingestion pipeline

Ingest data from apps, databases and files

ETL pipeline

Build ETL pipelines using SQL and Python

Job

Orchestrate notebooks, pipelines, queries and more

Jobs & pipelines

Runs

Send feedback

Filter by name or ID substring

All

Jobs

Pipelines

Owned by me

Accessible by me

Favorites

Tags

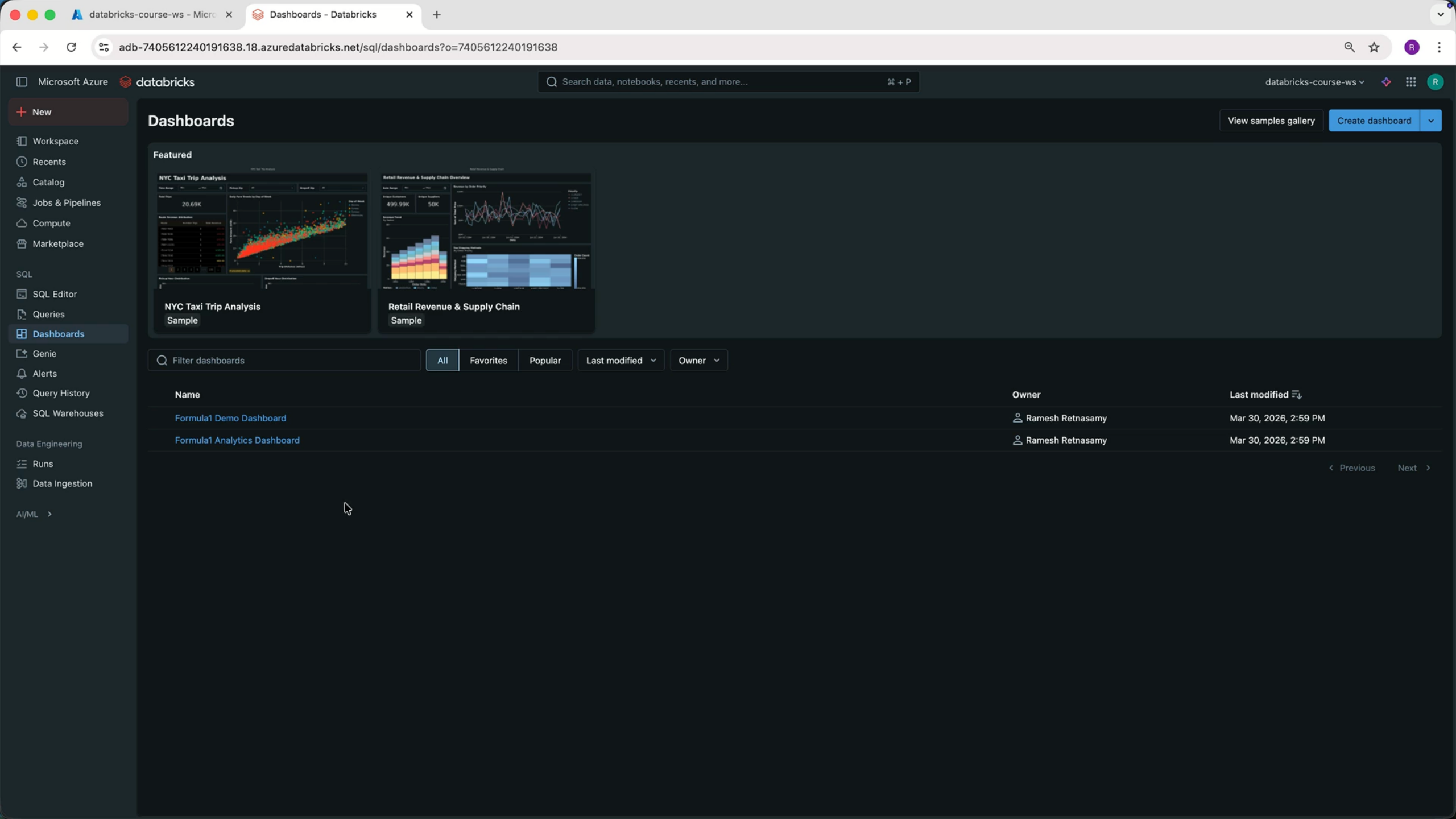
Run as

Create

Name	Type	Tags	Run as	Trigger	Recent runs	
job_formula1_incremental_batch_orchestration	Job		Ramesh Retnasamy			
job_formula1_lakehouse_full_refresh	Job		Ramesh Retnasamy	Table update		
job_formula1_lakehouse_incremental_refresh	Job		Ramesh Retnasamy	Table update		

Previous

Next



Who is this course for



University students looking for a career in Data Engineering

Developers transitioning into Data Engineering

Data Engineers moving to Azure or Databricks

Data Architects exploring Azure Data Engineering

Who is this course for



Learners who want a hands-on, project-based course



Those focused on Databricks and core Spark (not Declarative Pipelines or Streaming)



No Prior Databricks Knowledge Required

Basic SQL & Python Knowledge Required

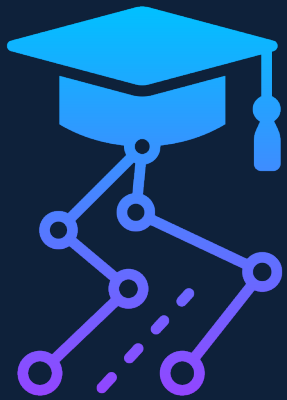
Azure Cloud Subscription Required



Ask Questions, I will answer 😊

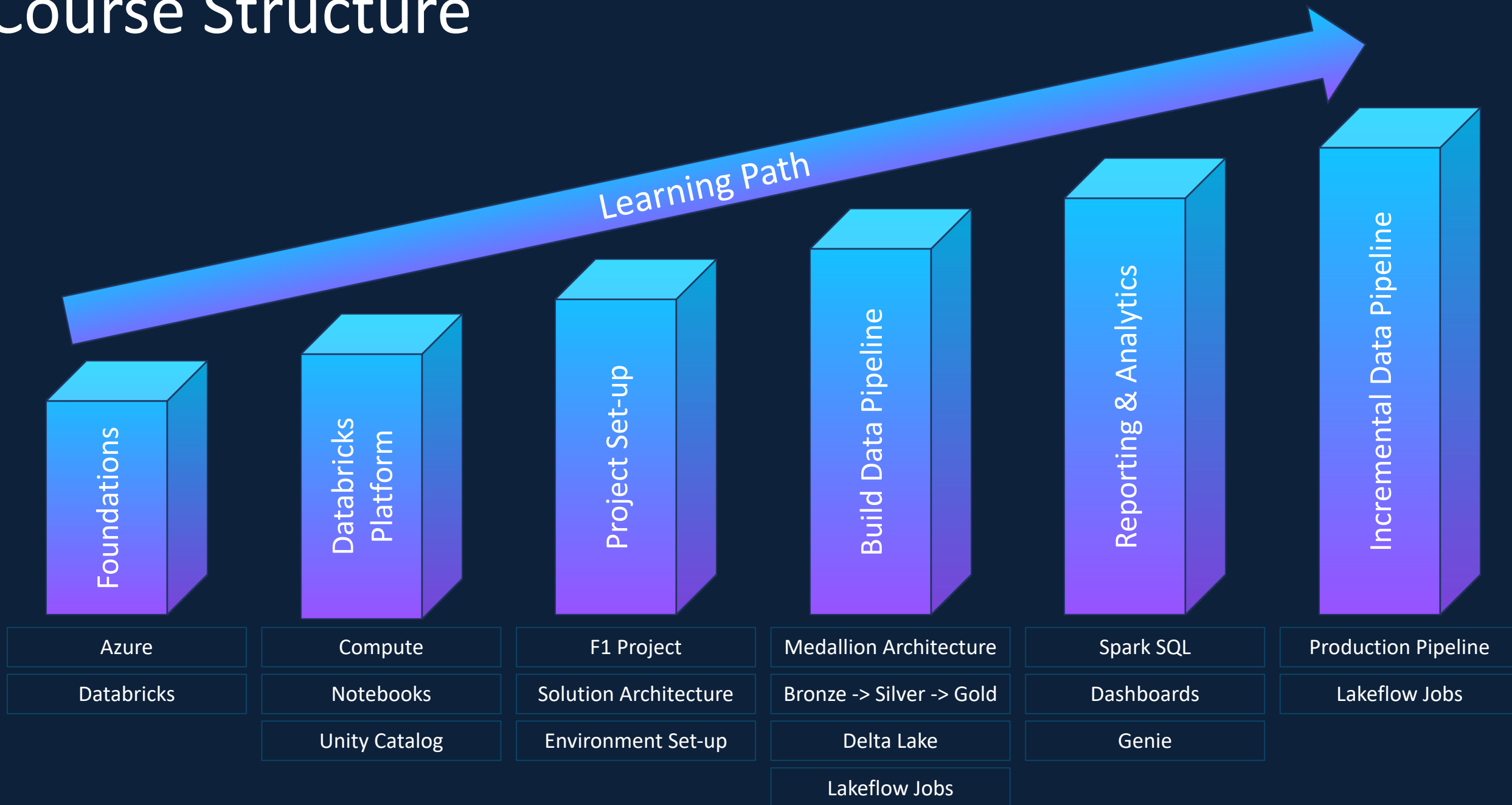
Udemy life time access

Udemy 30 day money back guarantee



Course Structure

Course Structure



Course Update Announcement



Creating Azure Free Subscription



Azure Portal Overview



Introduction to Azure Databricks

Introduction to Databricks



Apache Spark

Apache Spark is a lightning-fast unified analytics engine for big data processing and machine learning



100% Open source under Apache License

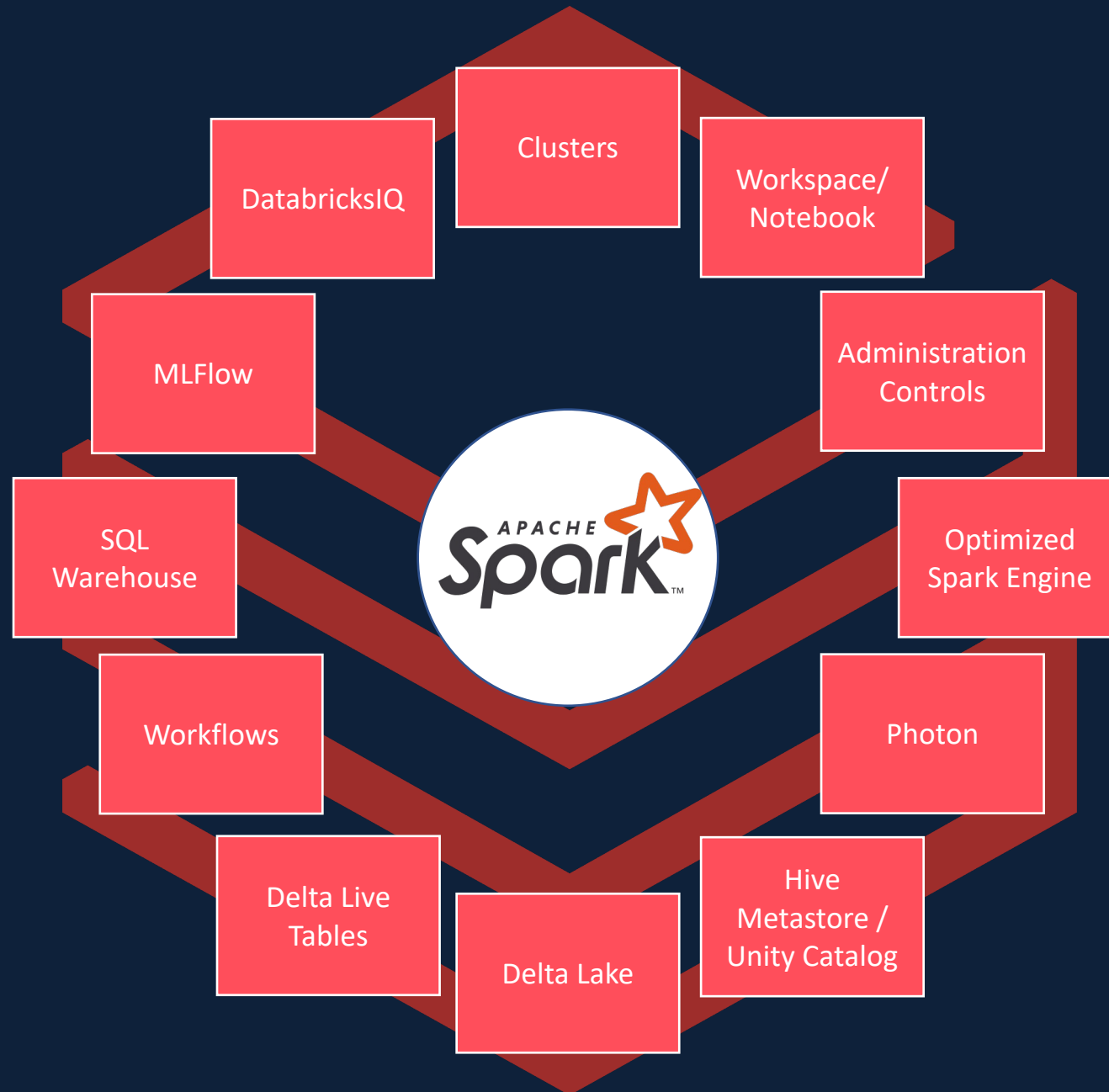
Simple and easy to use APIs

In-memory processing engine

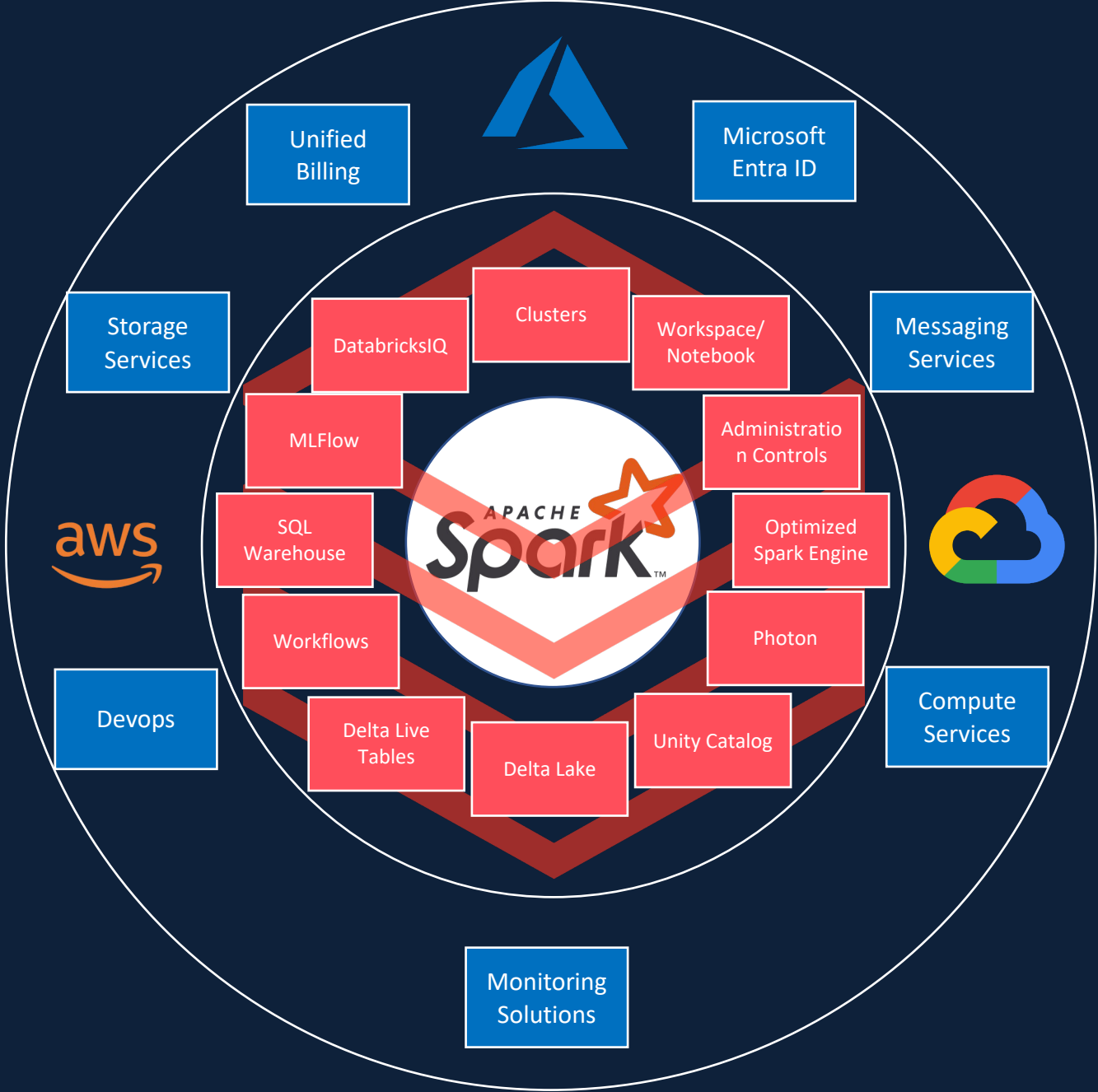
Distributed computing Platform

Unified engine which supports SQL, streaming, ML and graph processing

Databricks



Databricks





Creating a Databricks Service

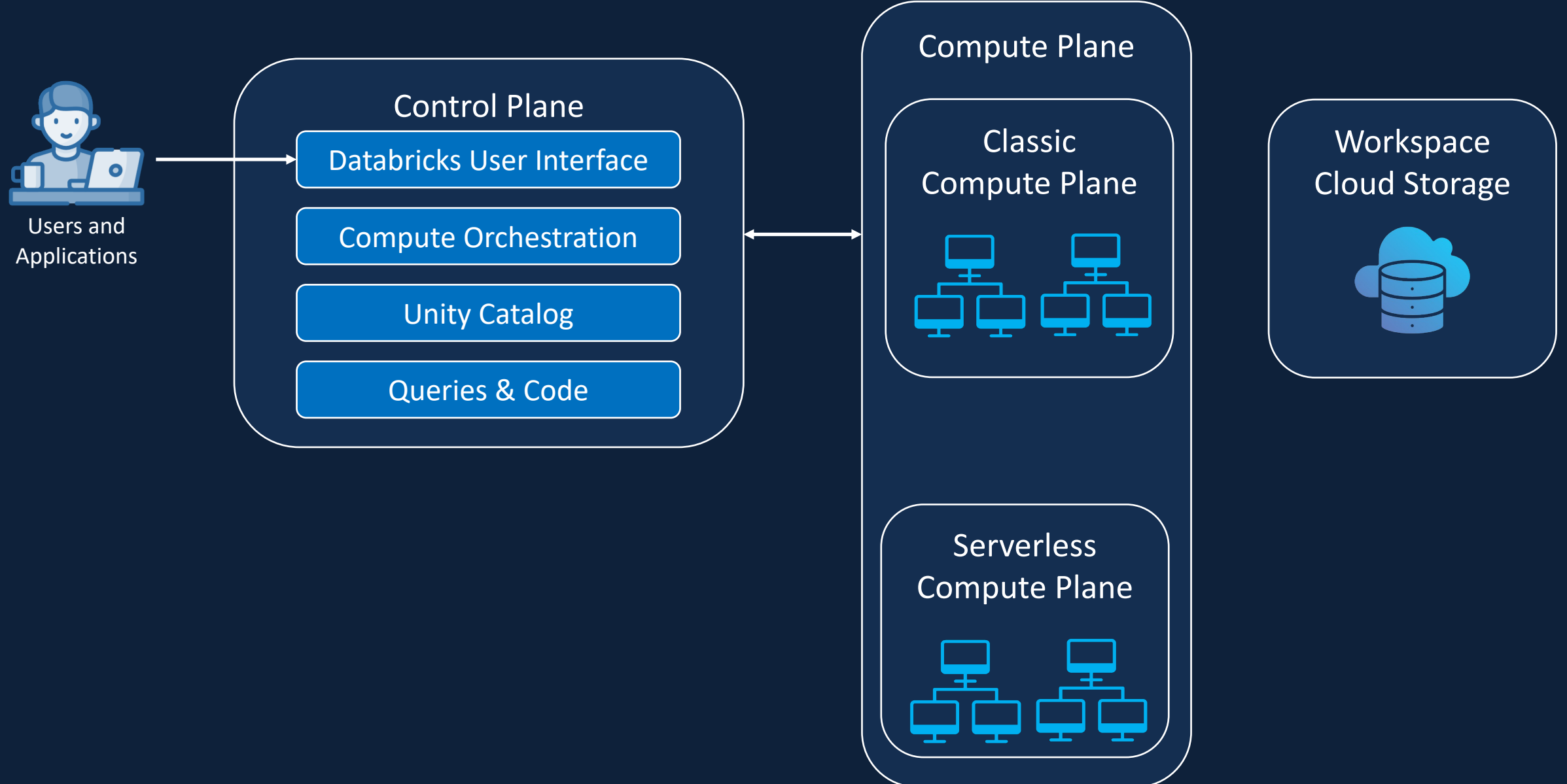


Databricks User Interface

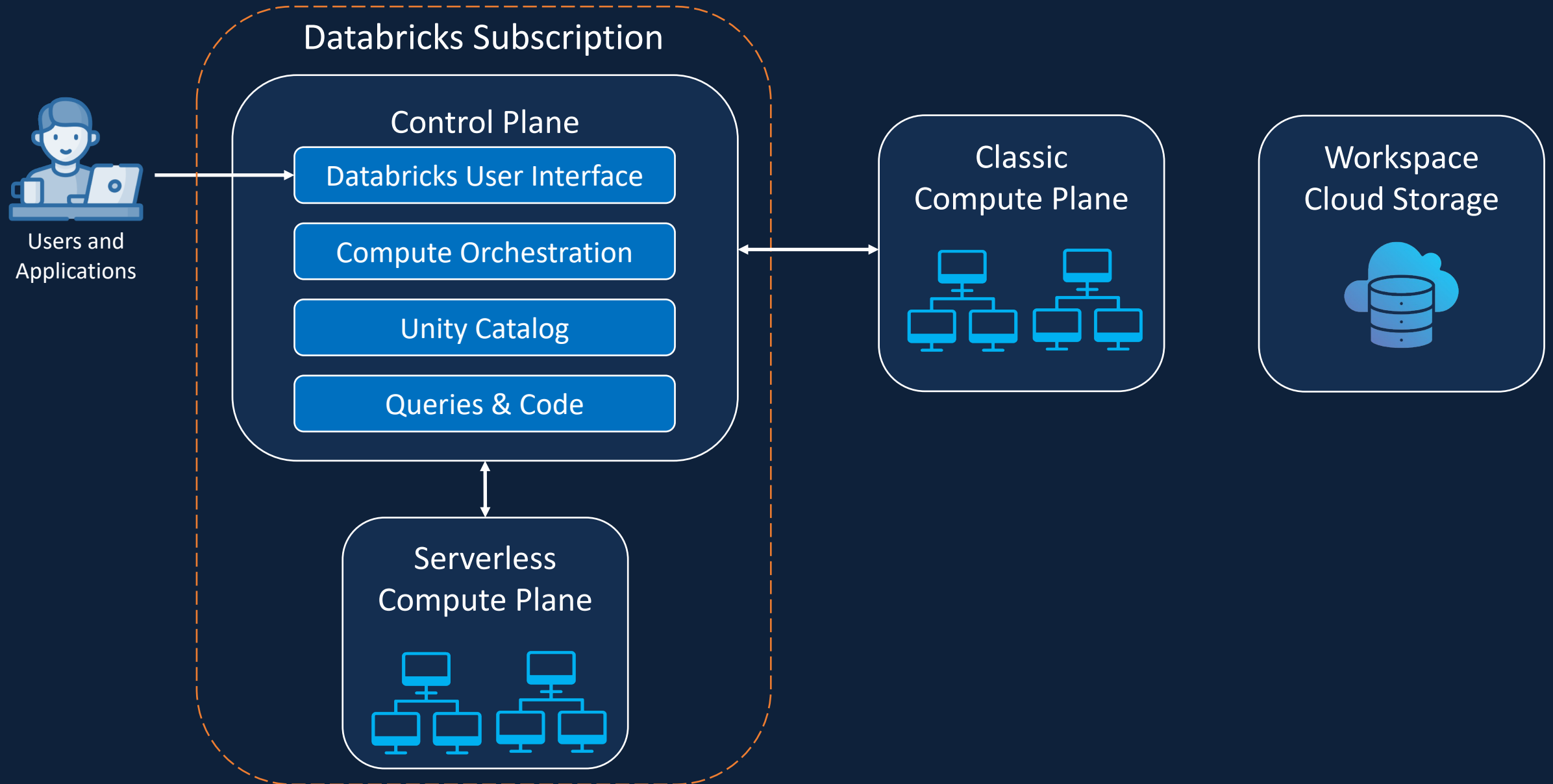


Databricks Architecture

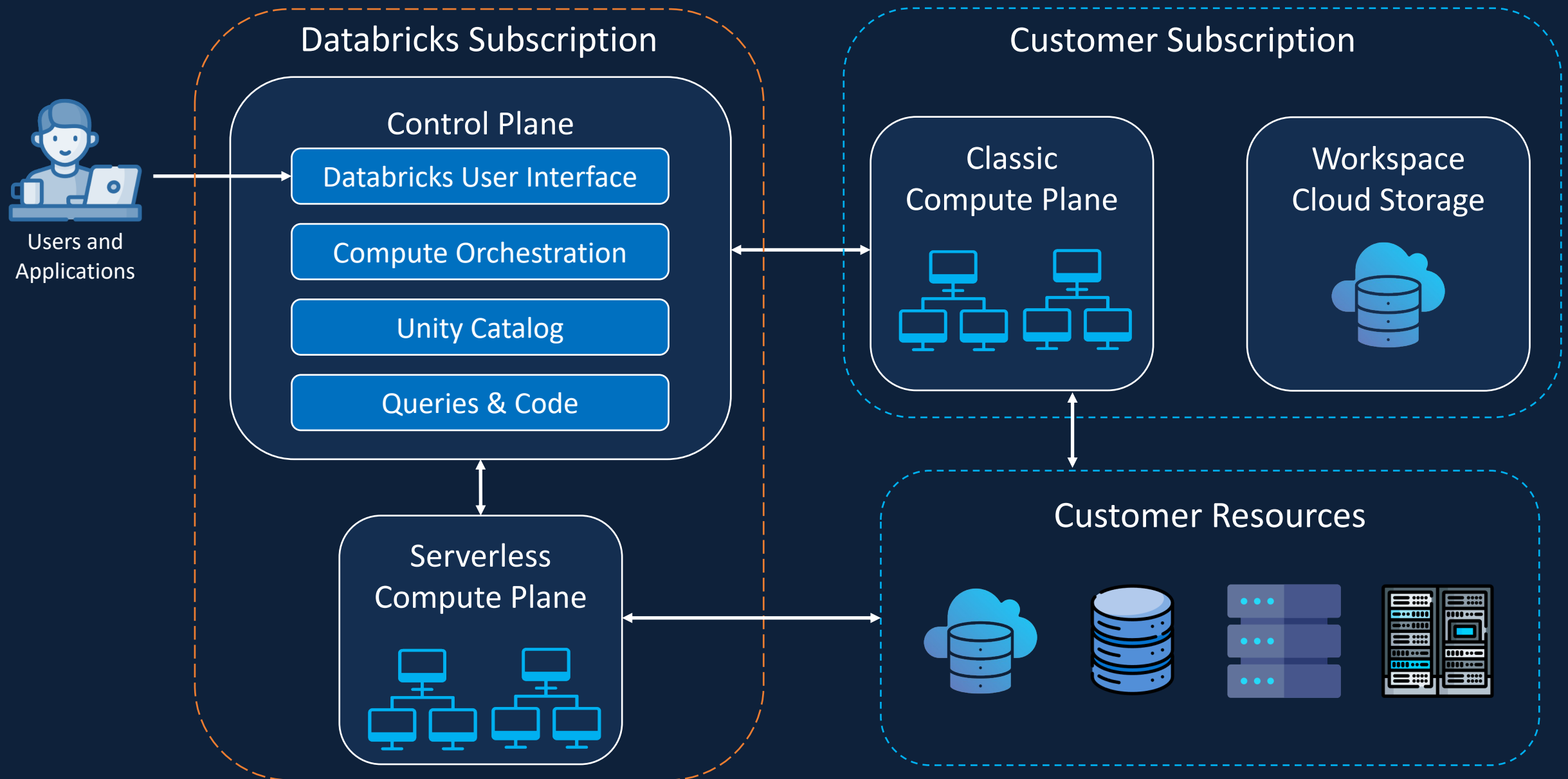
Databricks Architecture



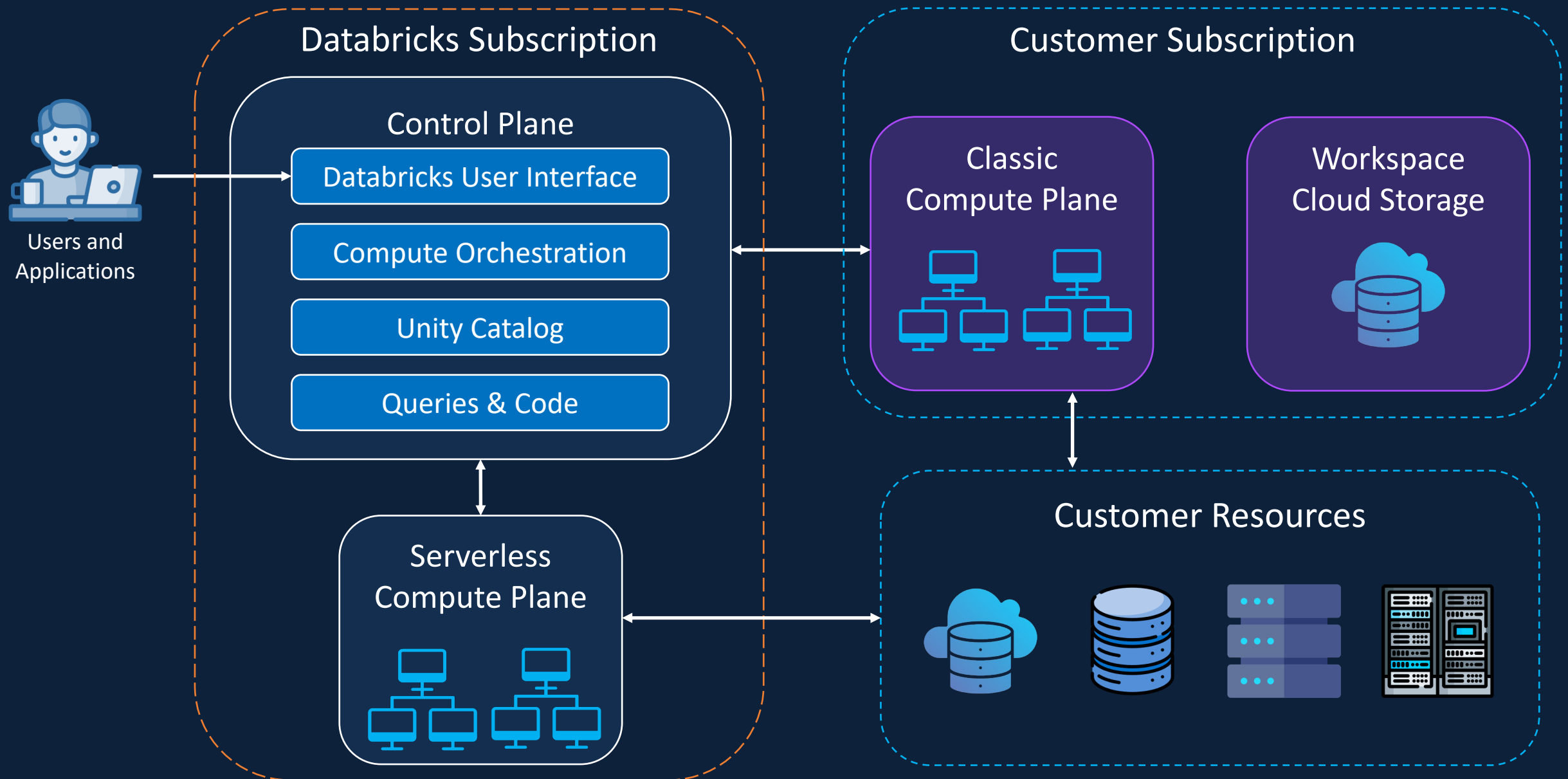
Databricks Architecture



Databricks Architecture



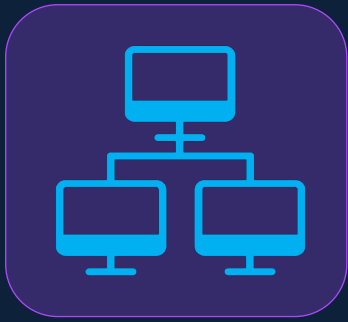
Databricks Architecture





Databricks Compute

Databricks Compute



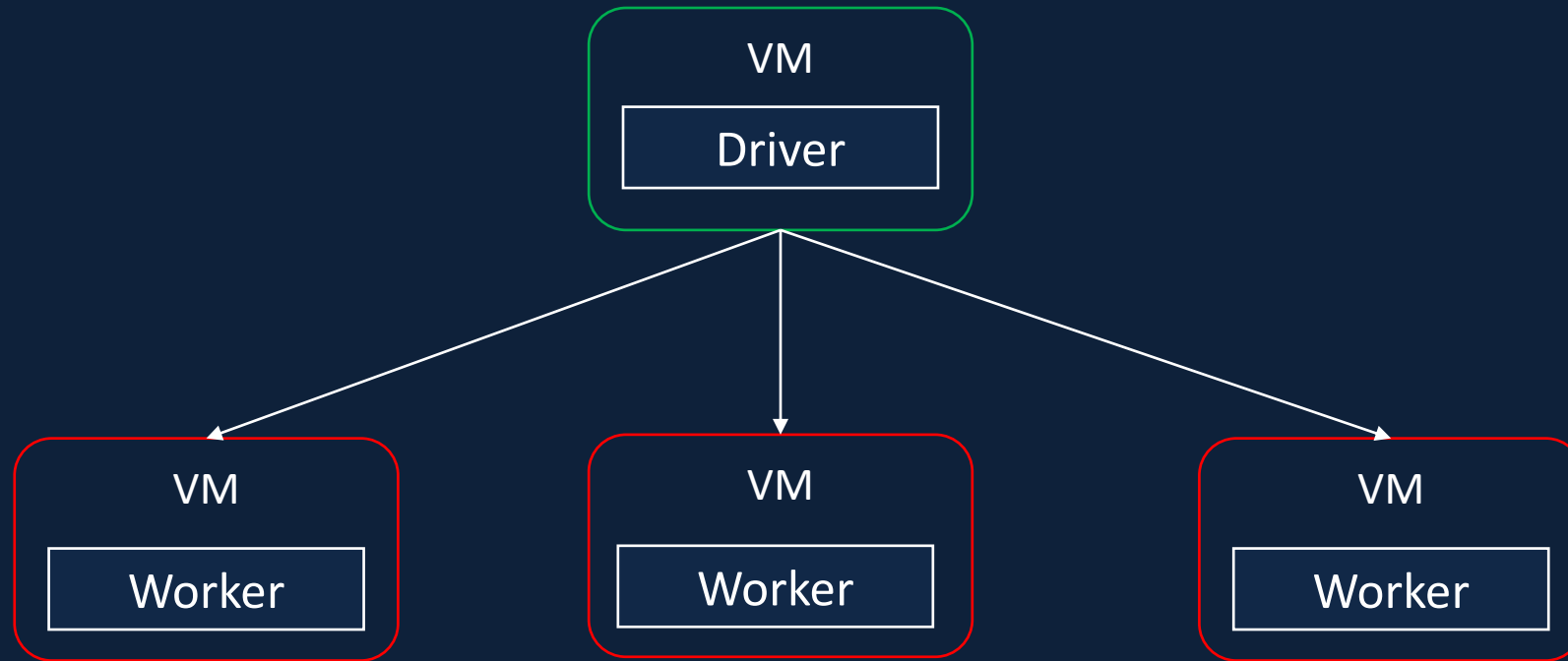
What is Databricks Compute

Configuring Databricks Compute

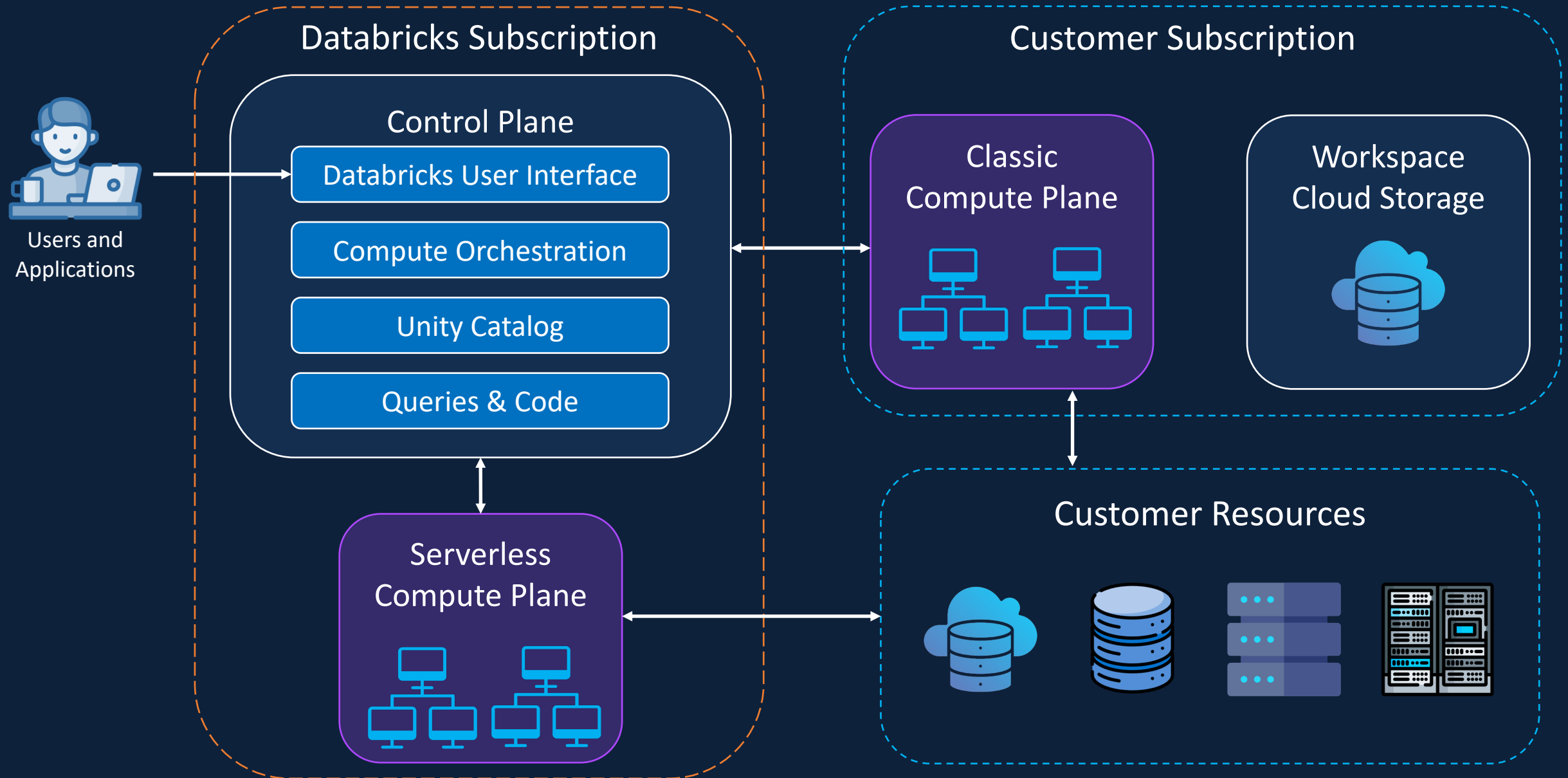
Creating Databricks Compute

Troubleshooting common issues

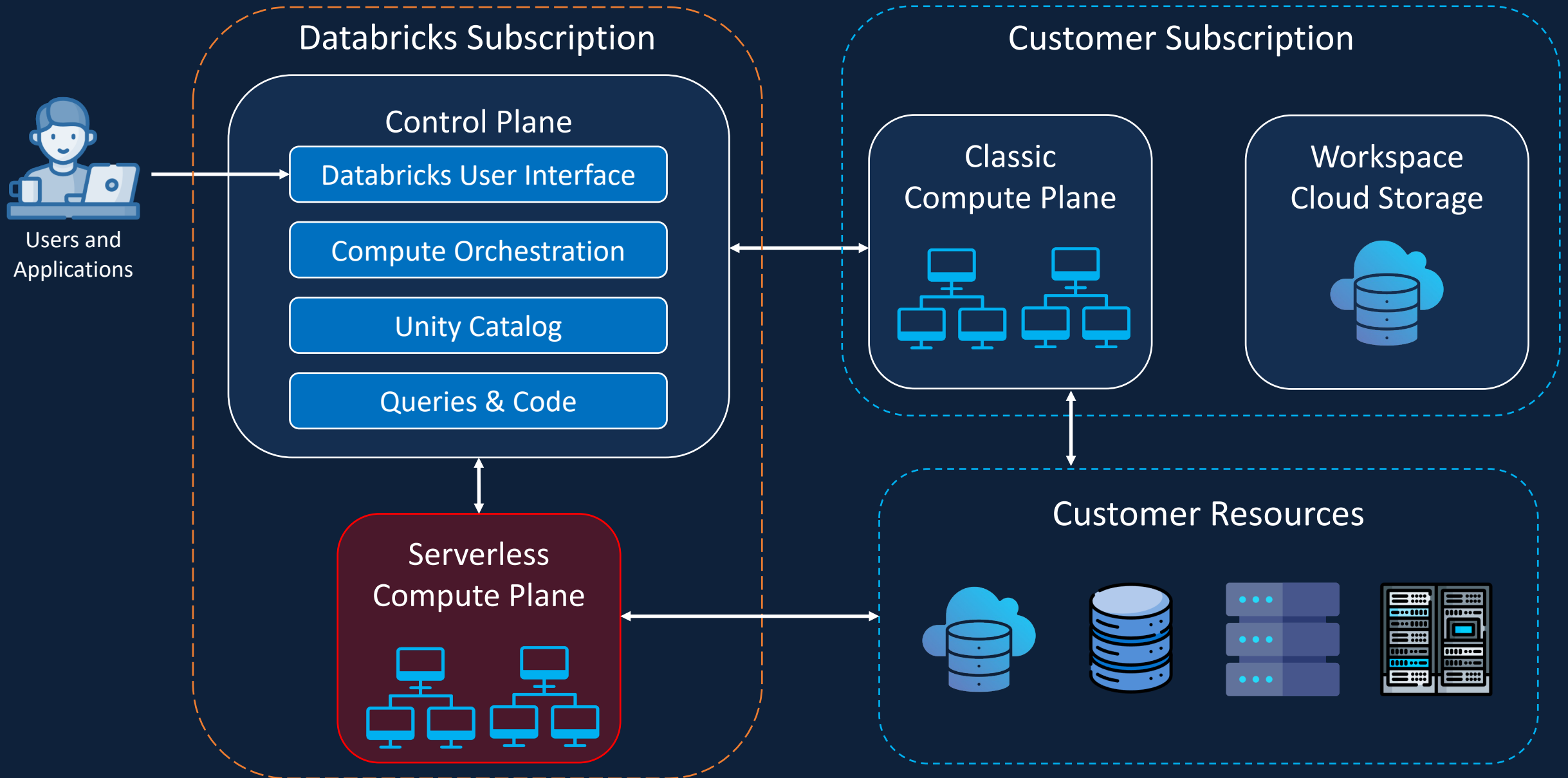
Databricks Compute



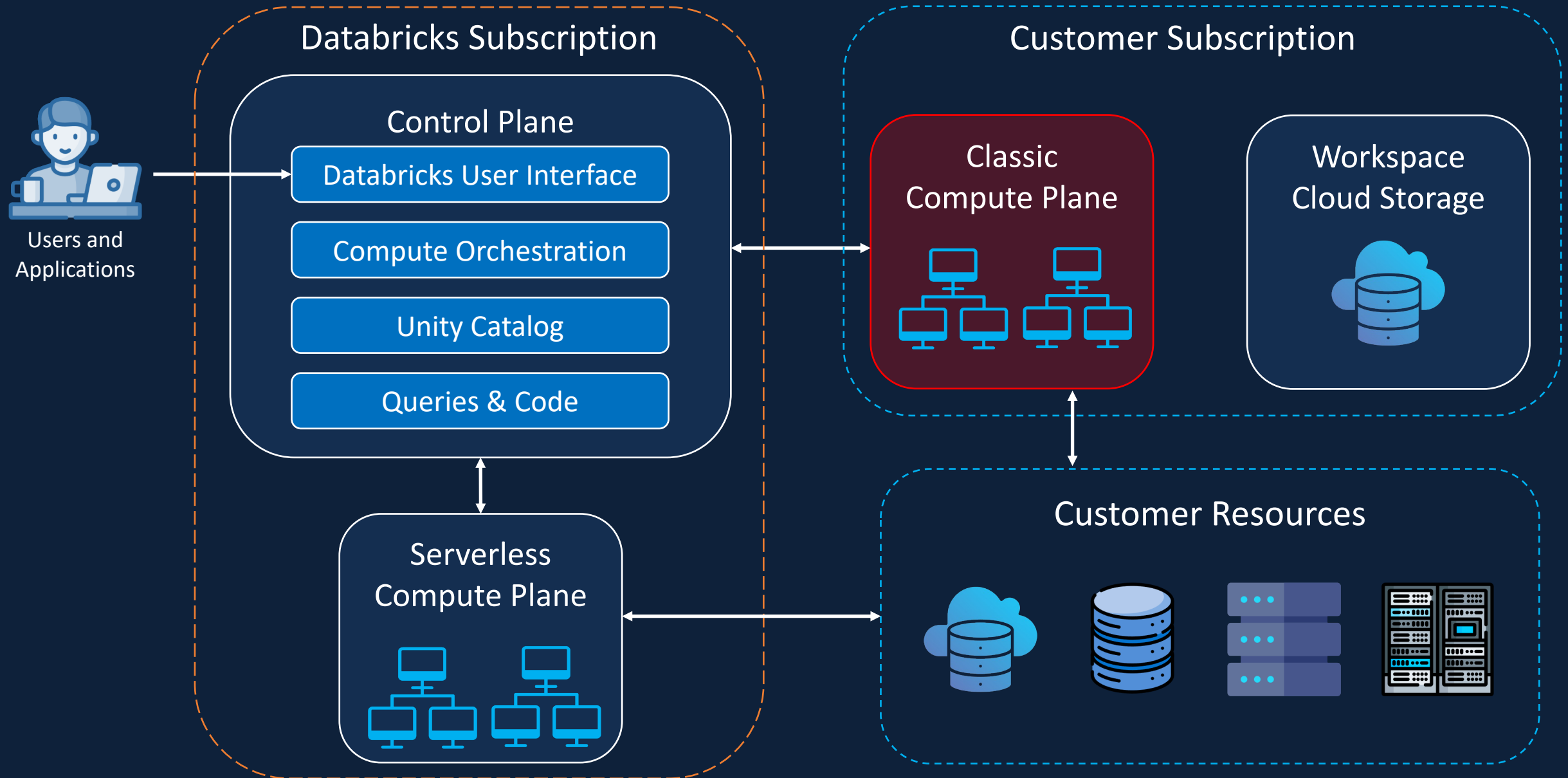
Databricks Compute



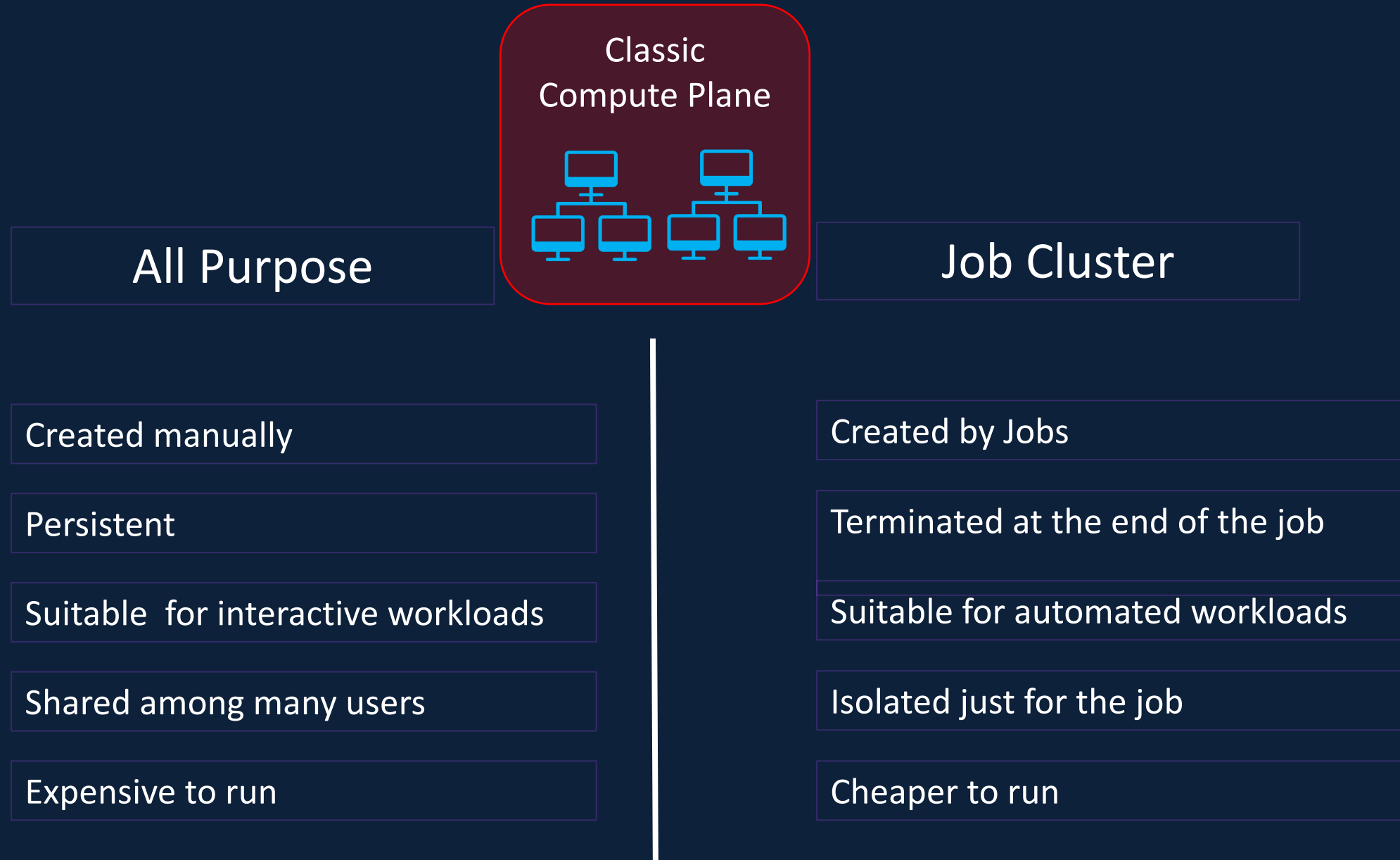
Databricks Compute



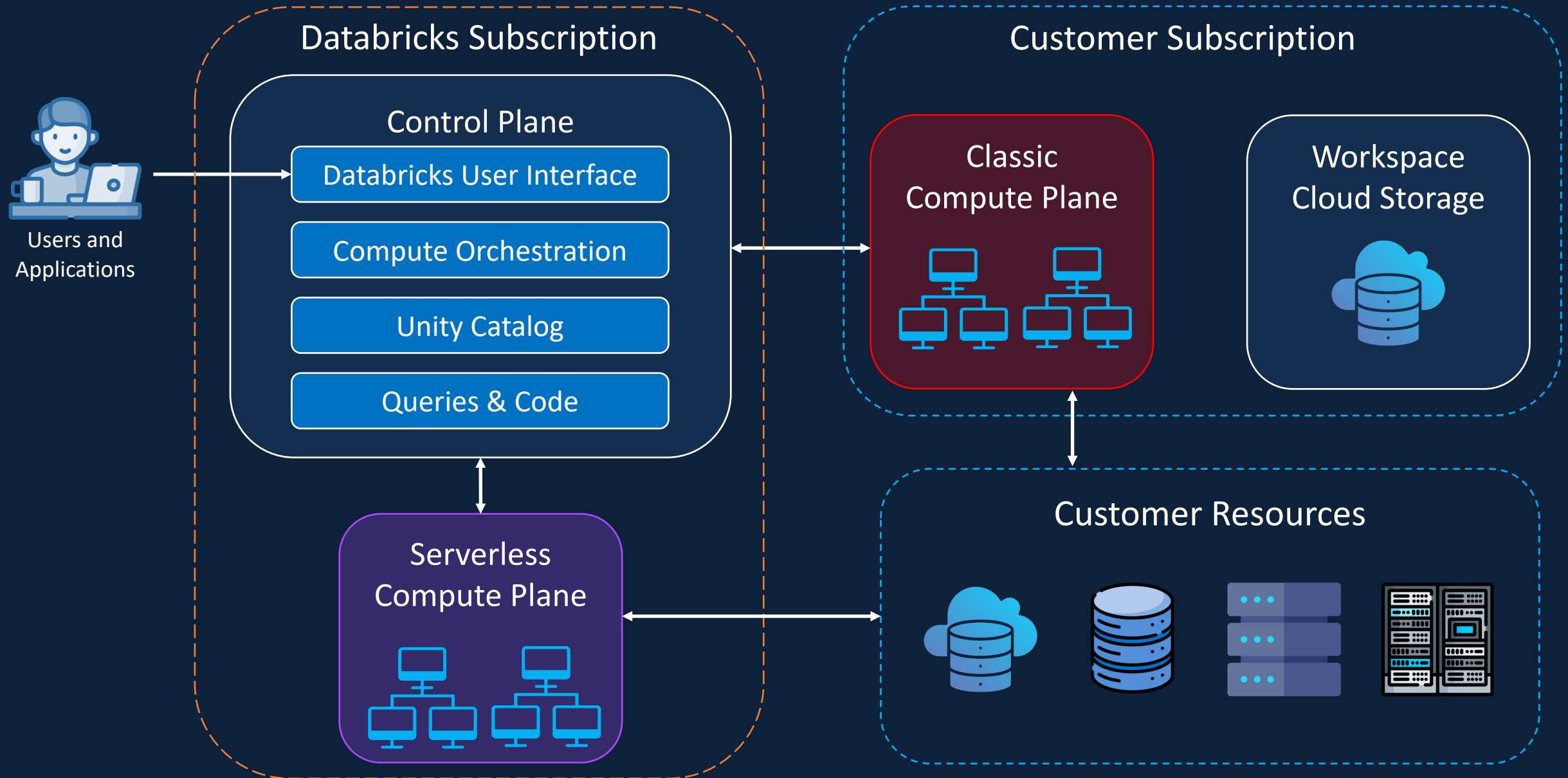
Databricks Compute



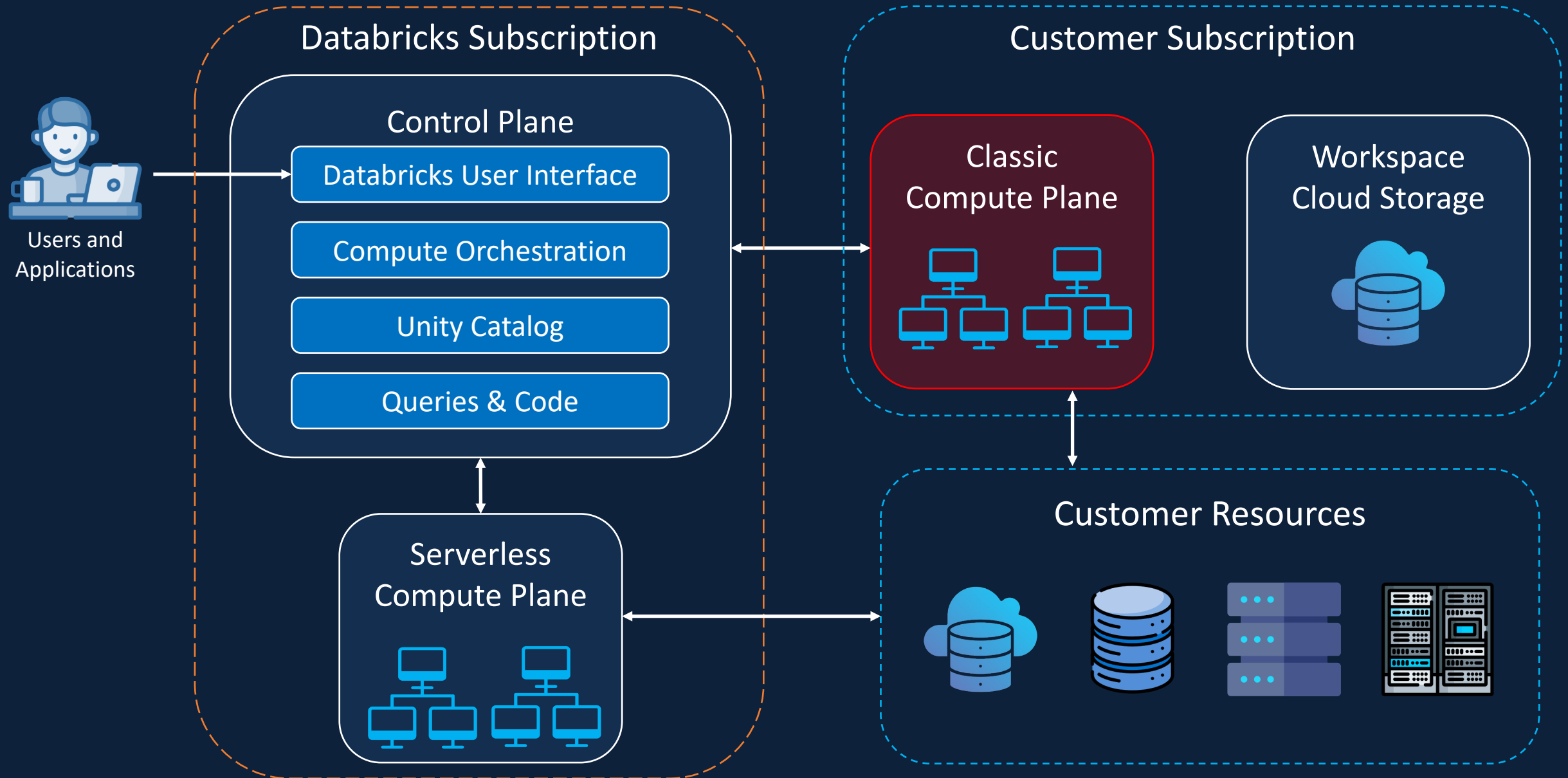
Databricks Classic Compute



Databricks Compute



Cluster Configuration



Cluster Configuration

Classic Compute Plane



Compute Configuration

Policy 

Unrestricted 

☒ Multi node ☐ Single node

Access mode  **Single user or group access** 

Dedicated (formerly: Single user)  Ramesh Retnasamy 

Performance

Databricks runtime version 

Runtime: 17.3 LTS (Scala 2.13, Spark 4.0.0) 

☒ Use Photon Acceleration 

Worker type  **Min workers** **Max workers**

Standard_E4ps_v6 32 GB Memory, 4 Cores  1 1 ☒ Spot instances 

Driver type

Same as worker 32 GB Memory, 4 Cores 

☒ Enable autoscaling 

☒ Terminate after 120 minutes of inactivity 

Tags

Add tags

Key	Value	Add
-----	-------	-----

> Automatically added tags

Create compute Cancel

Cluster Configuration

Single/ Multi Node

Multi Node

Single Node

Cluster Configuration

Single/ Multi Node

Access Mode

Dedicated (formerly Single User)

Only One User Access
Supports Python, SQL, Scala, R

Standard (formerly Shared)

Multiple User Access
Only available in Premium.
Supports Python, Scala and SQL

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Databricks Runtime

Apache
Spark

Supporting
Libraries

Photon

Databricks Runtime ML

Apache
Spark

Supporting
Libraries

Popular ML
Libraries

Photon

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Termination

- Terminates the cluster after X minutes of inactivity
- Default value for Single Node and Standard clusters is 120 minutes
- Users can specify a value between 10 and 43200 mins as the duration

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Auto Scaling

- User specifies the min and max worker nodes
- Auto scales between min and max based on the workload
- Users can opt for spot instances for the worker nodes

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

Memory Optimized

Compute Optimized

Storage Optimized

General Purpose

GPU Accelerated

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

Cluster Policy

Compute Configuration

Policy ⓘ
Unrestricted 

☒ Multi node ☐ Single node

Access mode ⓘ
Dedicated (formerly: Single user) 

Single user or group access ⓘ
Ramesh Retnasamy 

Performance
Databricks runtime version ⓘ
Runtime: 17.3 LTS (Scala 2.13, Spark 4.0.0) 

☒ Use Photon Acceleration ⓘ

Worker type ⓘ
Standard_E4ps_v6 32 GB Memory, 4 Cores 

Min workers
1

Max workers
1

☒ Spot instances ⓘ

Driver type
Same as worker 32 GB Memory, 4 Cores 

☒ Enable autoscaling ⓘ

☒ Terminate after 120 minutes of inactivity ⓘ

Tags ⓘ
Add tags

> Automatically added tags

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

Cluster Policy

Compute Configuration

Policy 
Personal Compute 

Single user or group access 
Ramesh Retnasamy 

Performance
Databricks runtime version 
Runtime: 18.0 ML (Scala 2.13, Spark 4.1.0) 

Node type 
Standard_D4ds_v4 16 GB Memory, 4 Cores 

☒ Terminate after 20 minutes of inactivity 

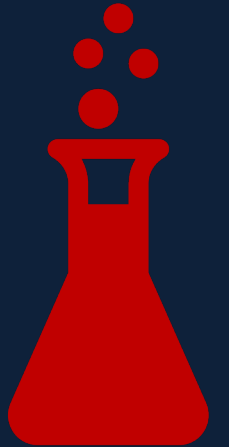
Tags 
Add tags

Key	Value	Add
> Automatically added tags		
▶ Advanced options		

Create compute

Cancel

Creating Databricks Cluster





Databricks Notebooks

Databricks Notebooks



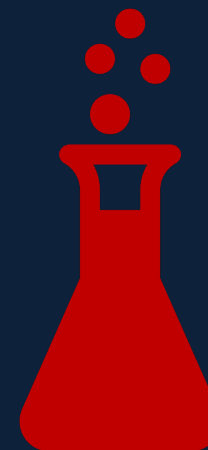
Introduction to notebooks

Magic commands

Databricks utilities

Debugging notebooks

Databricks Notebooks





Databricks Magic Commands



Databricks Utilities

Databricks Utilities



File System Utilities

Secrets Utilities

Widget Utilities

Notebook Workflow Utilities

Unity Catalog - Introduction



Unity Catalog Overview

Enable Unity Catalog Metastore

Cluster Configurations

Unity Catalog Object Model

Access External Data Lake

Introduction to Unity Catalog

Introduction to Unity Catalog



Users and
Applications



Compute

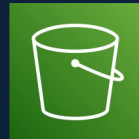
Read / Write



Data Storage



ADLS



AWS S3

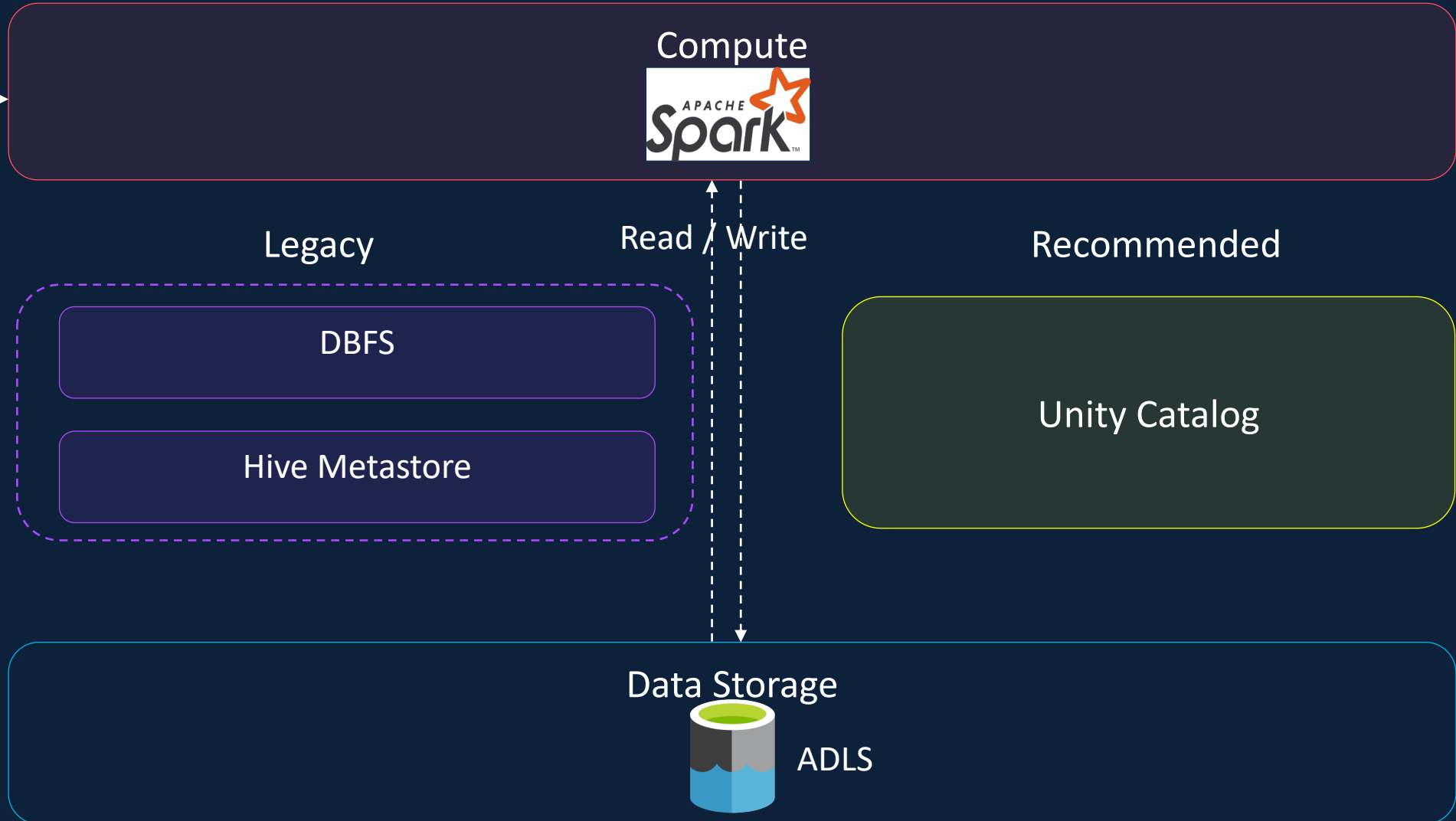


GCS

Introduction to Unity Catalog



Users and Applications



Introduction to Unity Catalog



Users and Applications

Compute



Read / Write

Unity Catalog



Volume



Table



View



Function

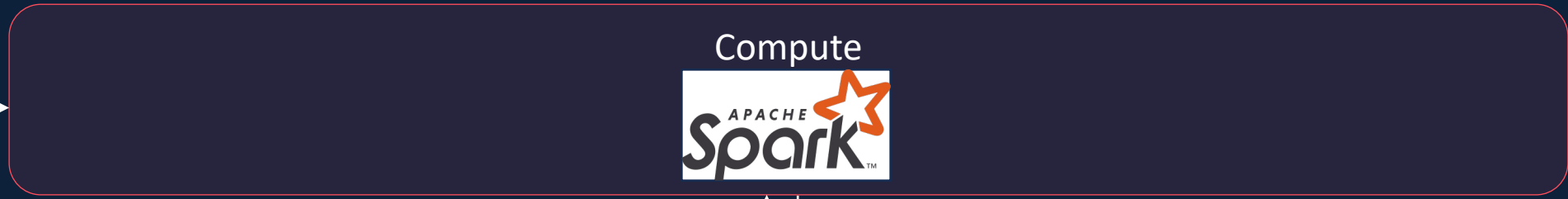
Data Storage



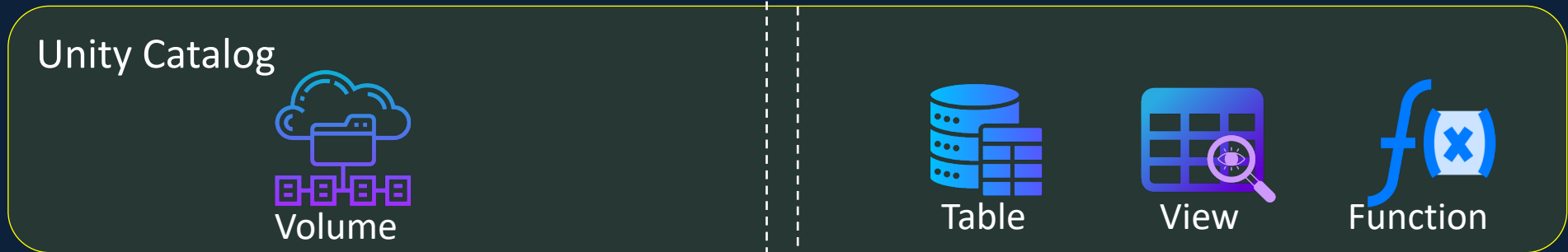
ADLS



Unity Catalog Object Model



Read / Write



Unity Catalog Object Model

A dark green rounded rectangle with a thin yellow border, containing the word "Metastore" in white text.

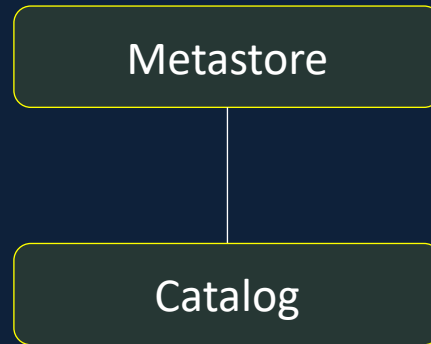
Metastore

Top level container

Only one per region

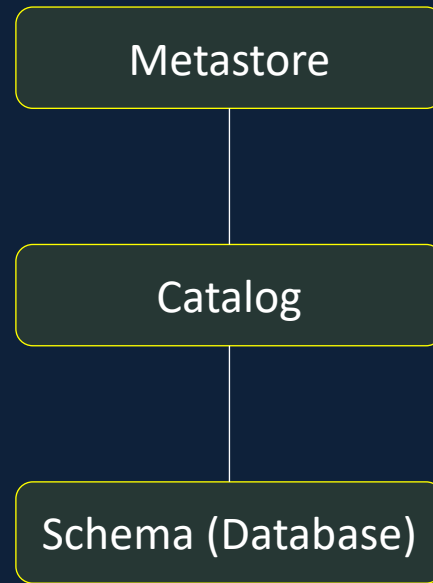
Paired with Default ADLS Storage

Unity Catalog Object Model



Logical container within Metastore

Unity Catalog Object Model

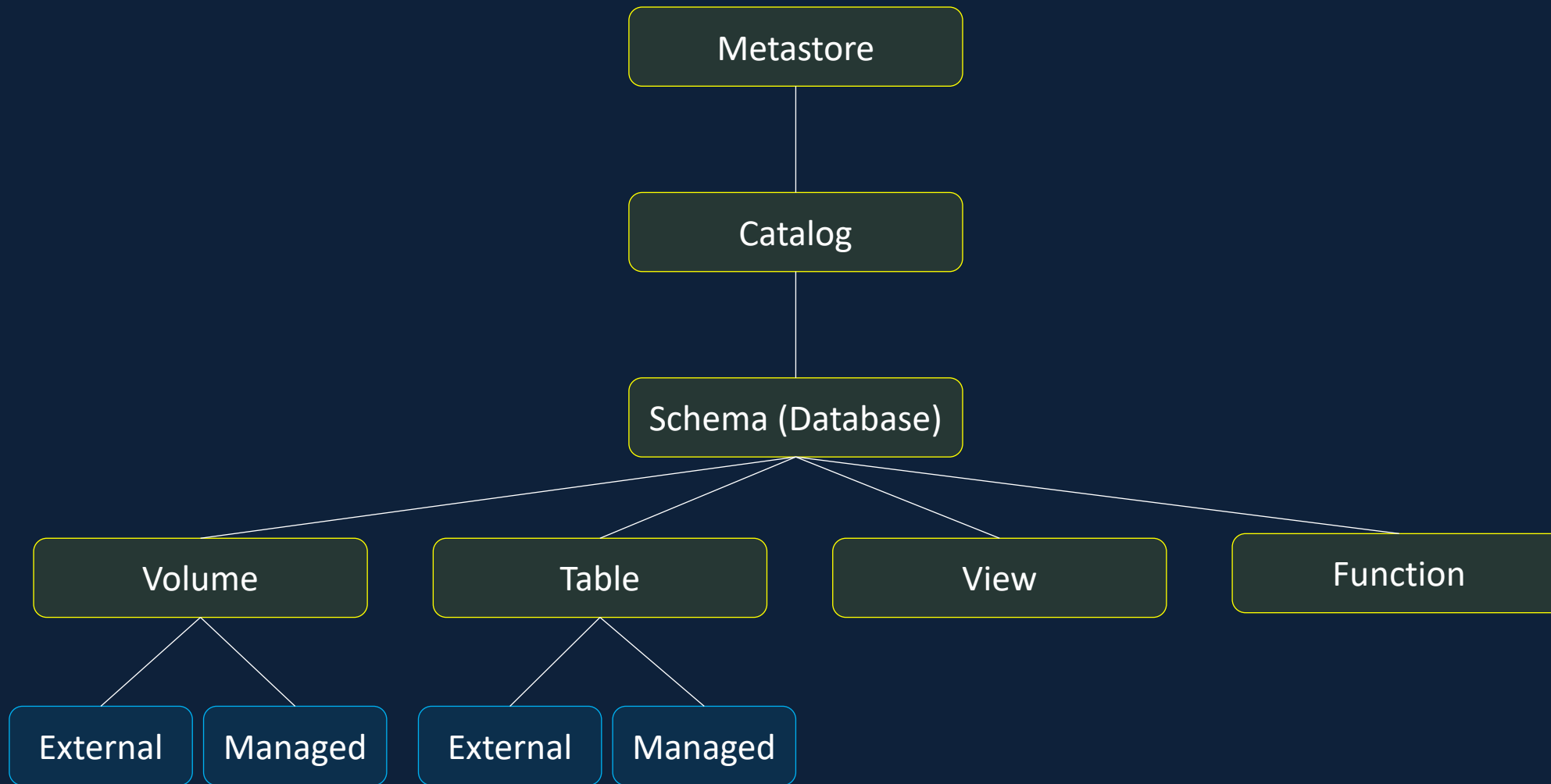


Next level container within Catalogs

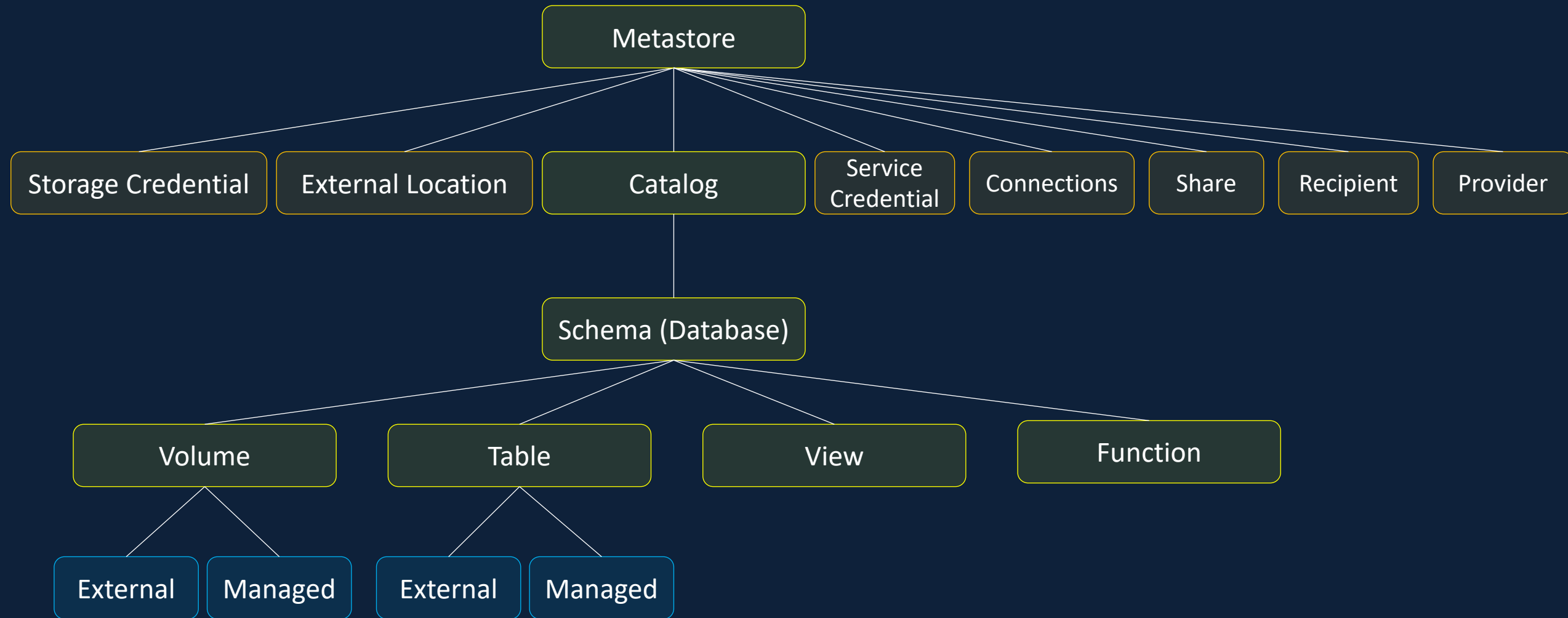
Schemas and Databases are the same

Use Schema instead of Database

Unity Catalog Object Model

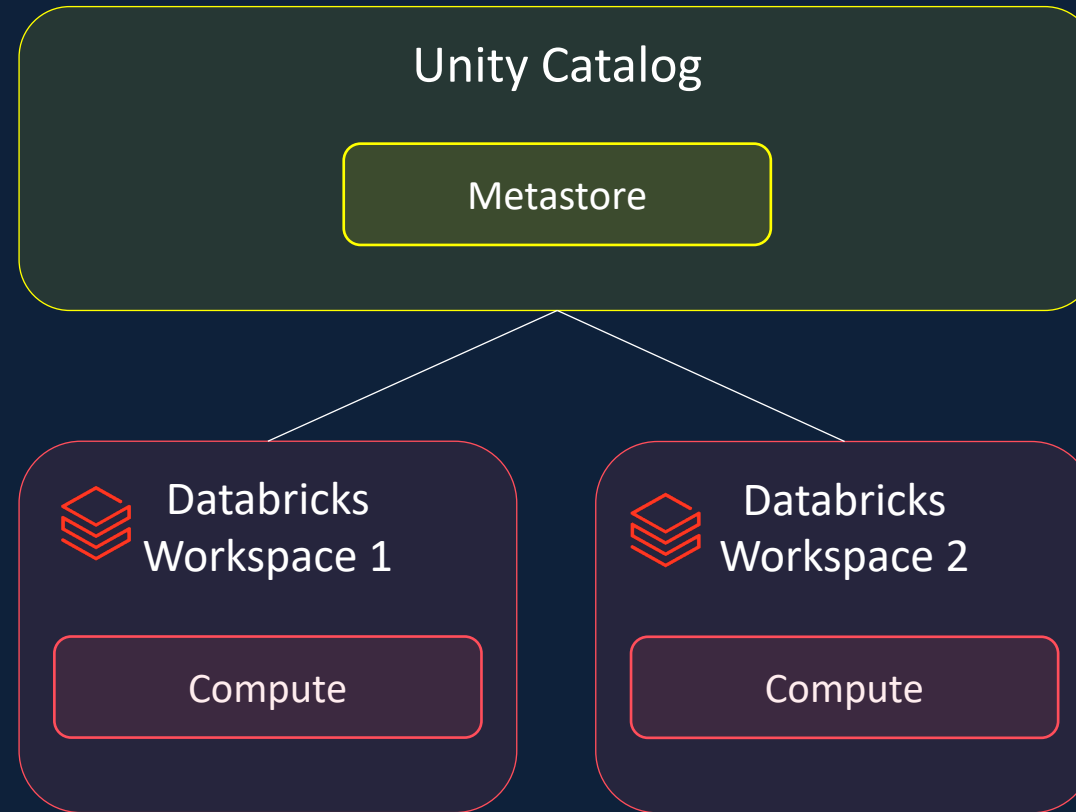


Unity Catalog Object Model

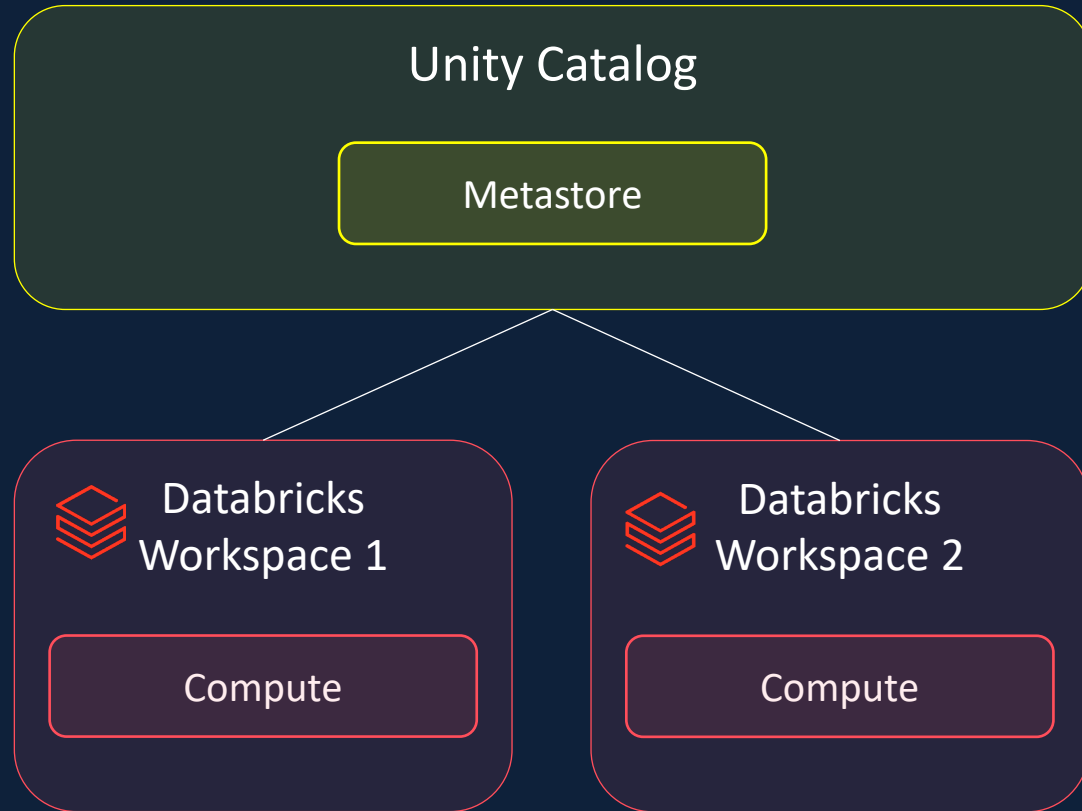


Unity Catalog Set-up

Unity Catalog Set-up



Unity Catalog Set-up



Login to Databricks Account Console

<https://accounts.azuredatabricks.net/>

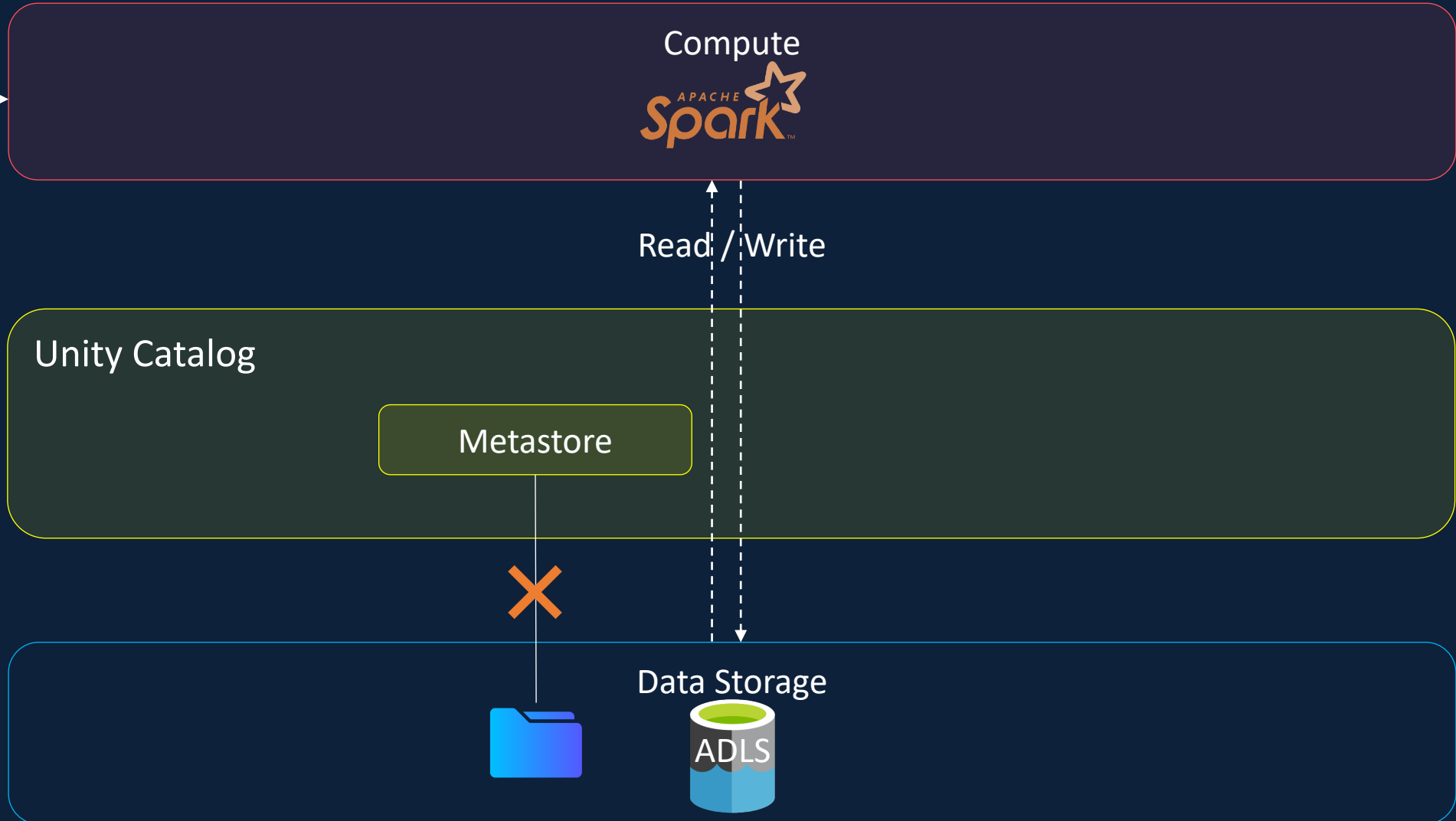
Account Console User Must

Exist in Microsoft Entra ID

Have Global Administrator Privileges

Azure Subscriptions Created on or after Nov 2023 have Unity Catalog Metastore Created by Default.

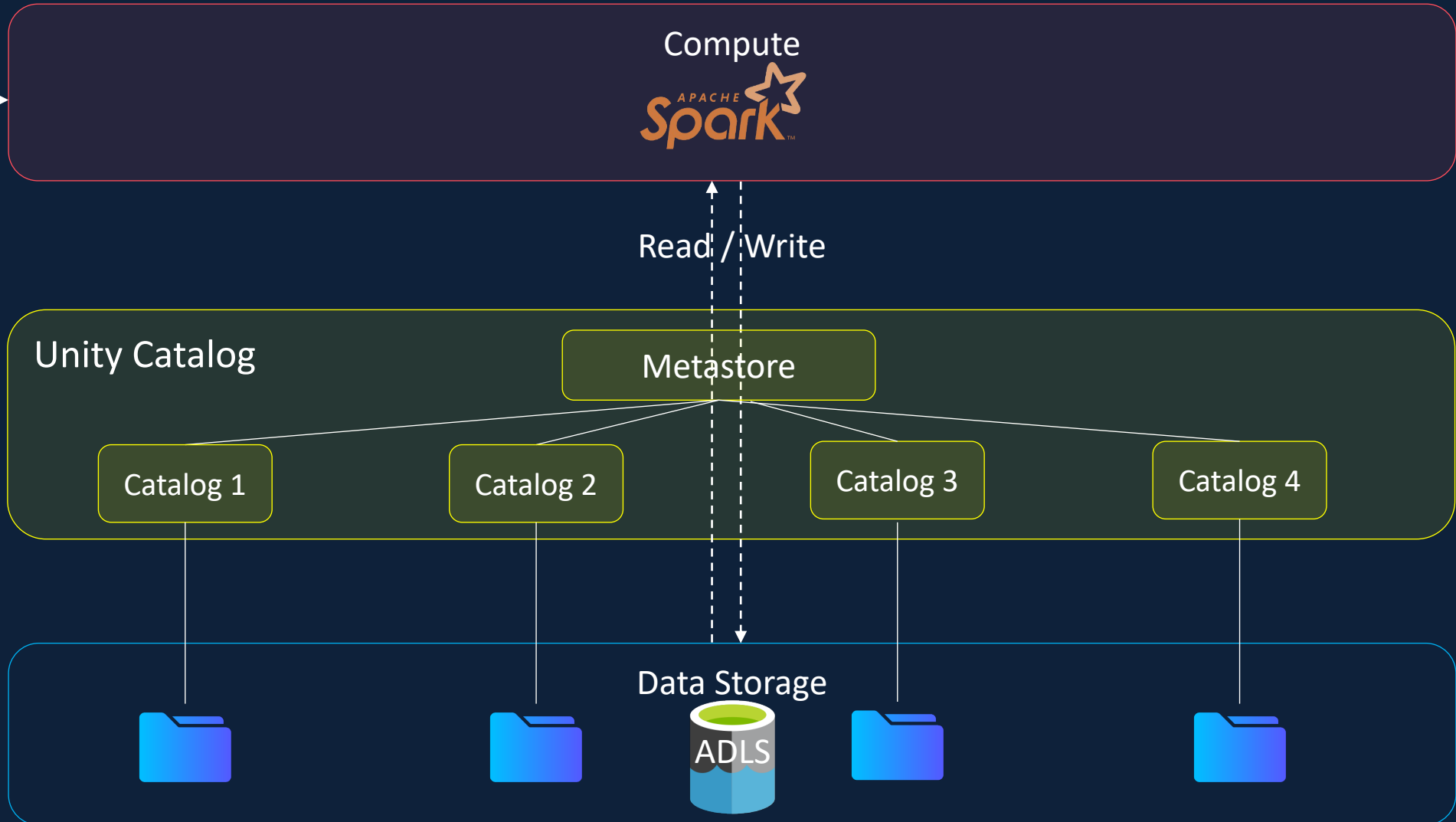
Configure Access to Cloud Storage



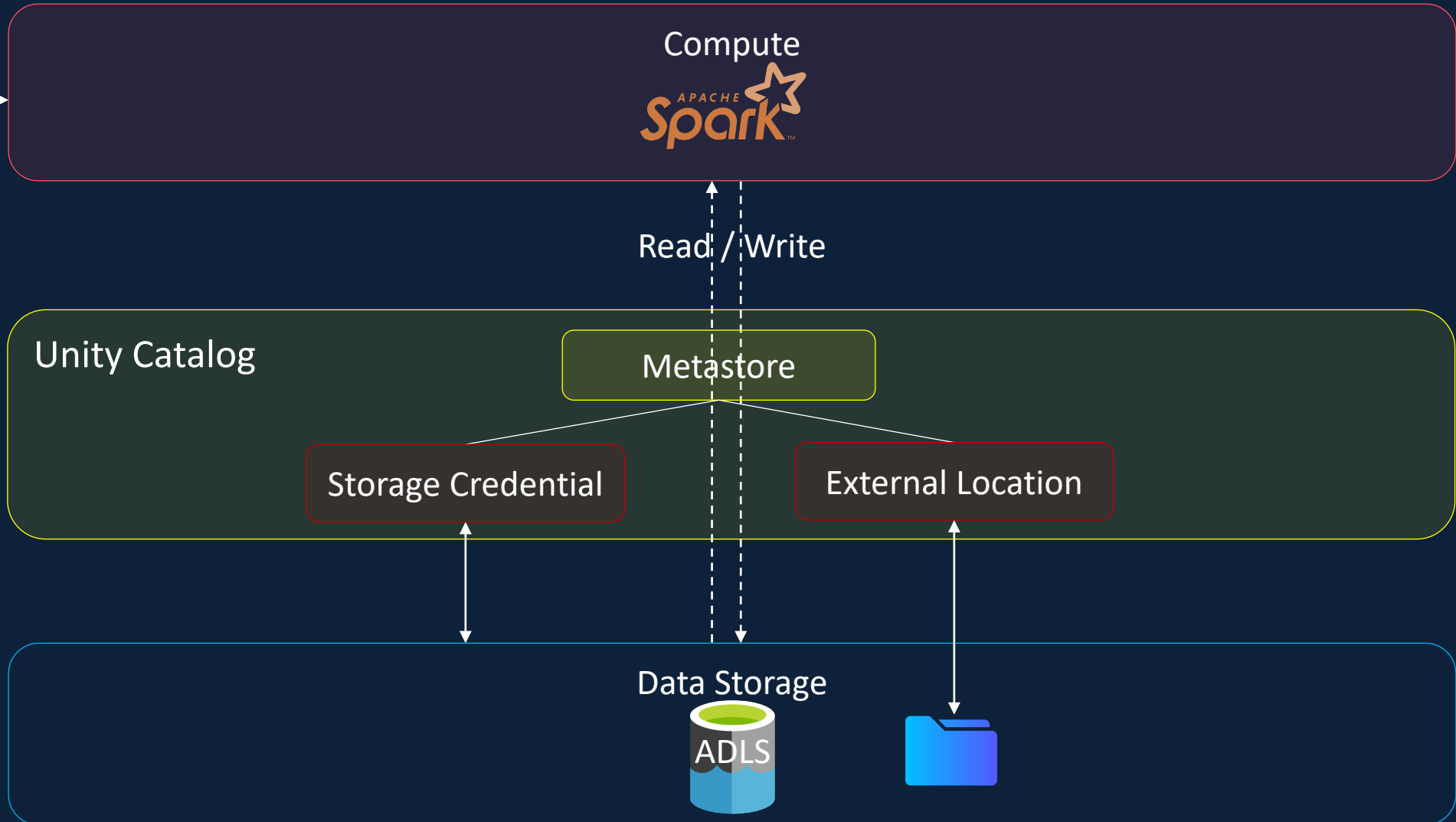
Configure Access to Cloud Storage



Users and Applications



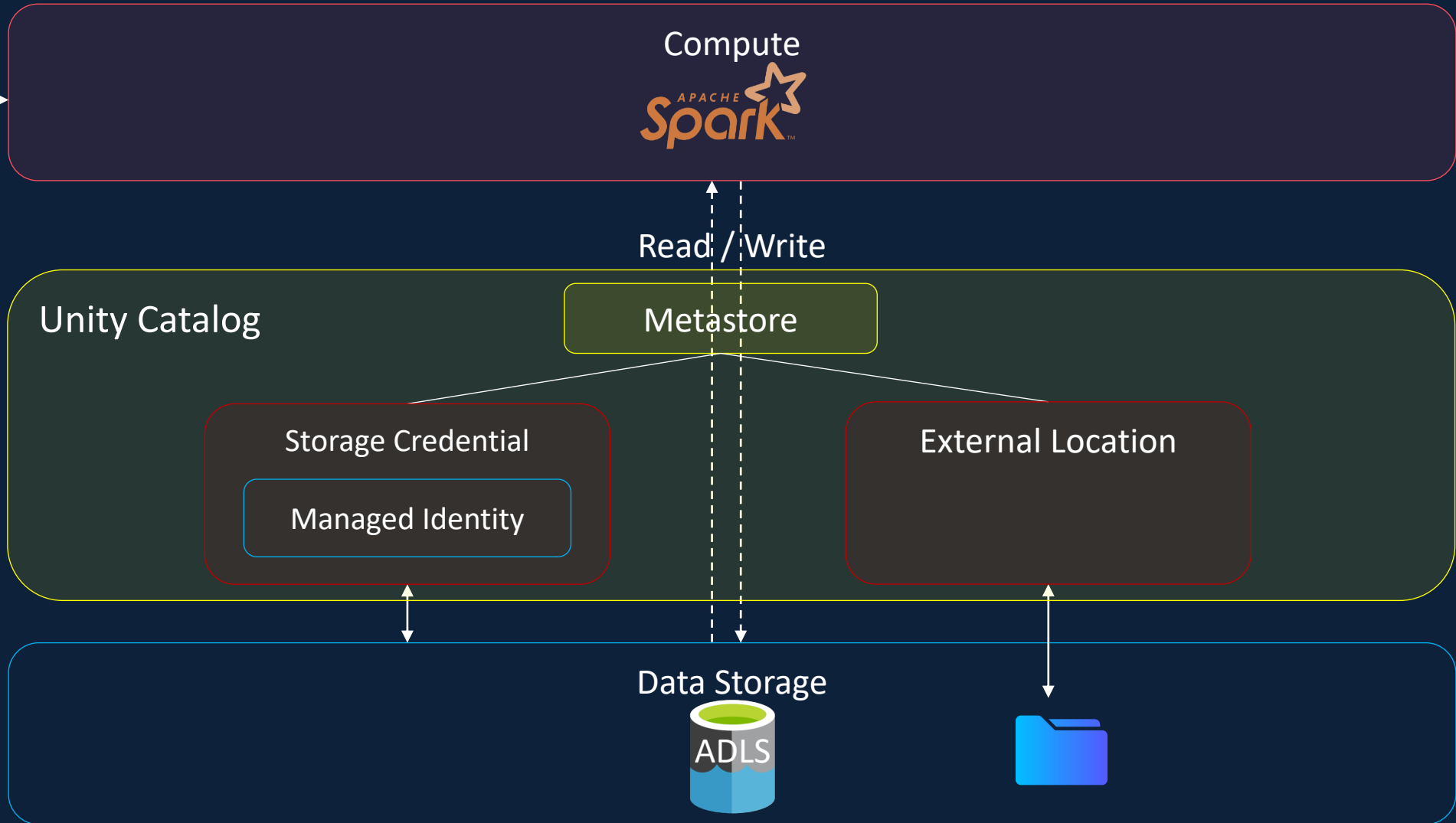
Configure Access to Cloud Storage



Configure Access to Cloud Storage



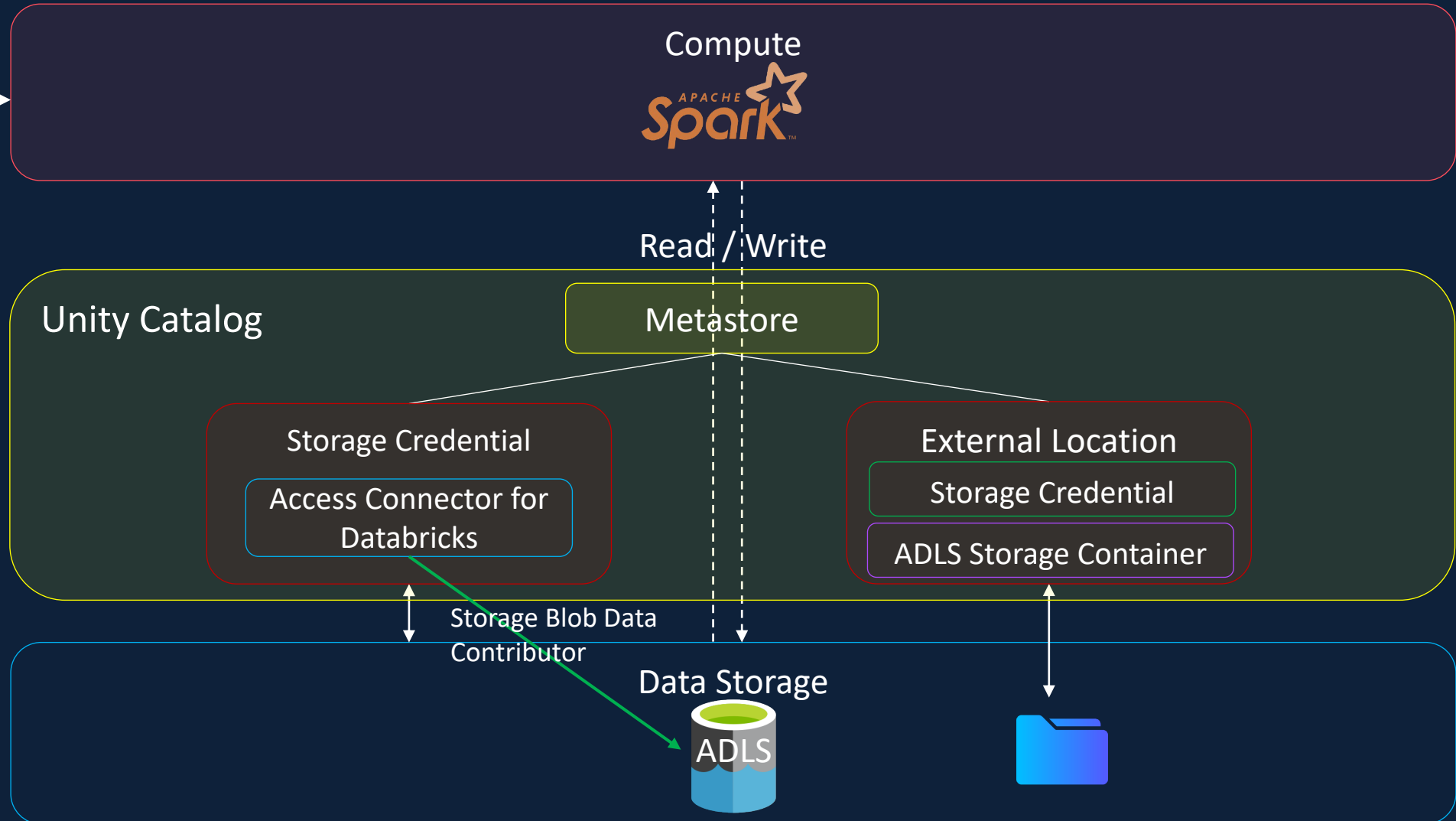
Users and Applications



Configure Access to Cloud Storage



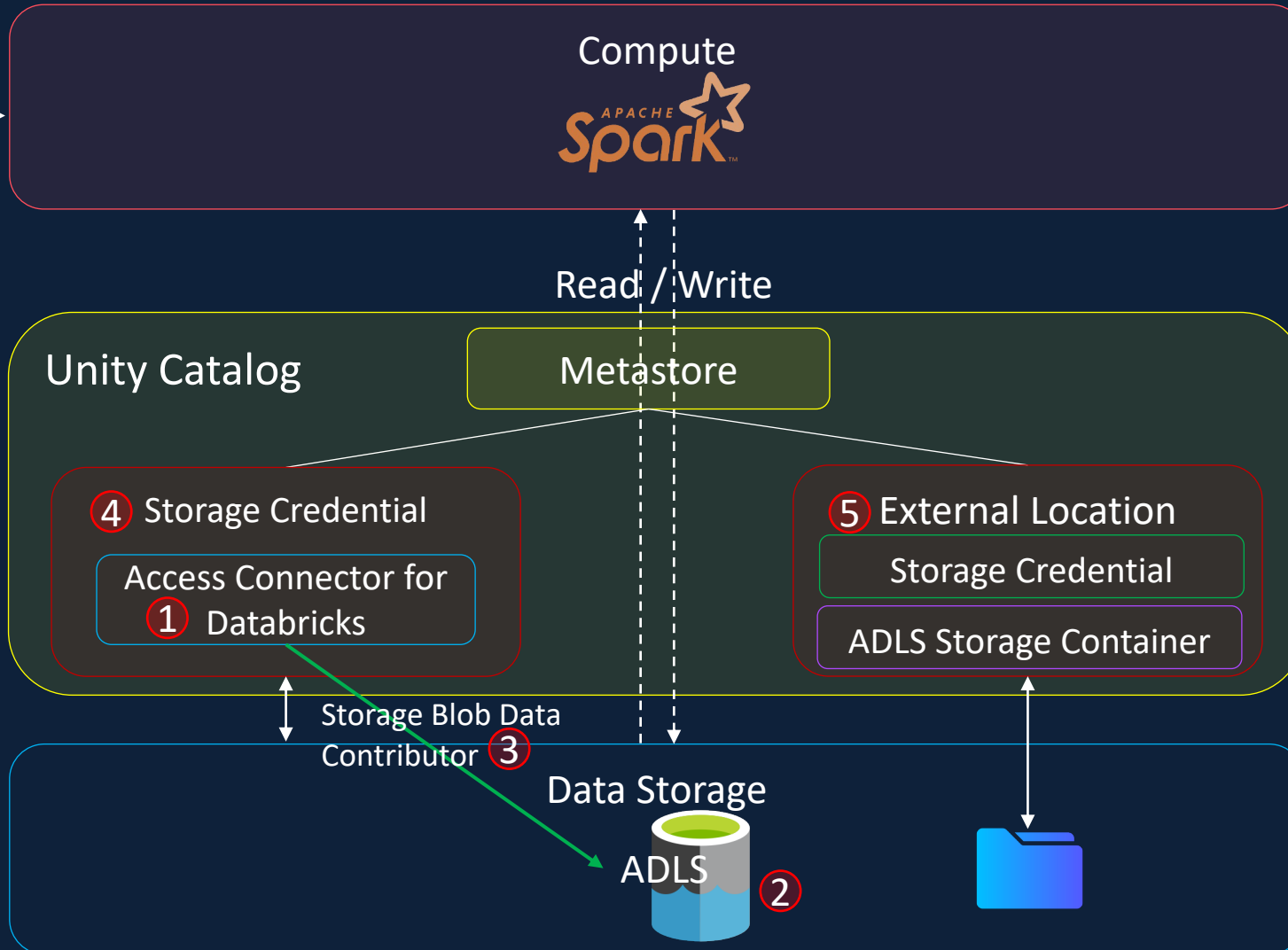
Users and Applications



Configure Access to Cloud Storage



Users and Applications



1. Create Access Connector

2. Create Azure Data Lake Storage

3. Assign Storage Blob Data Contributor role

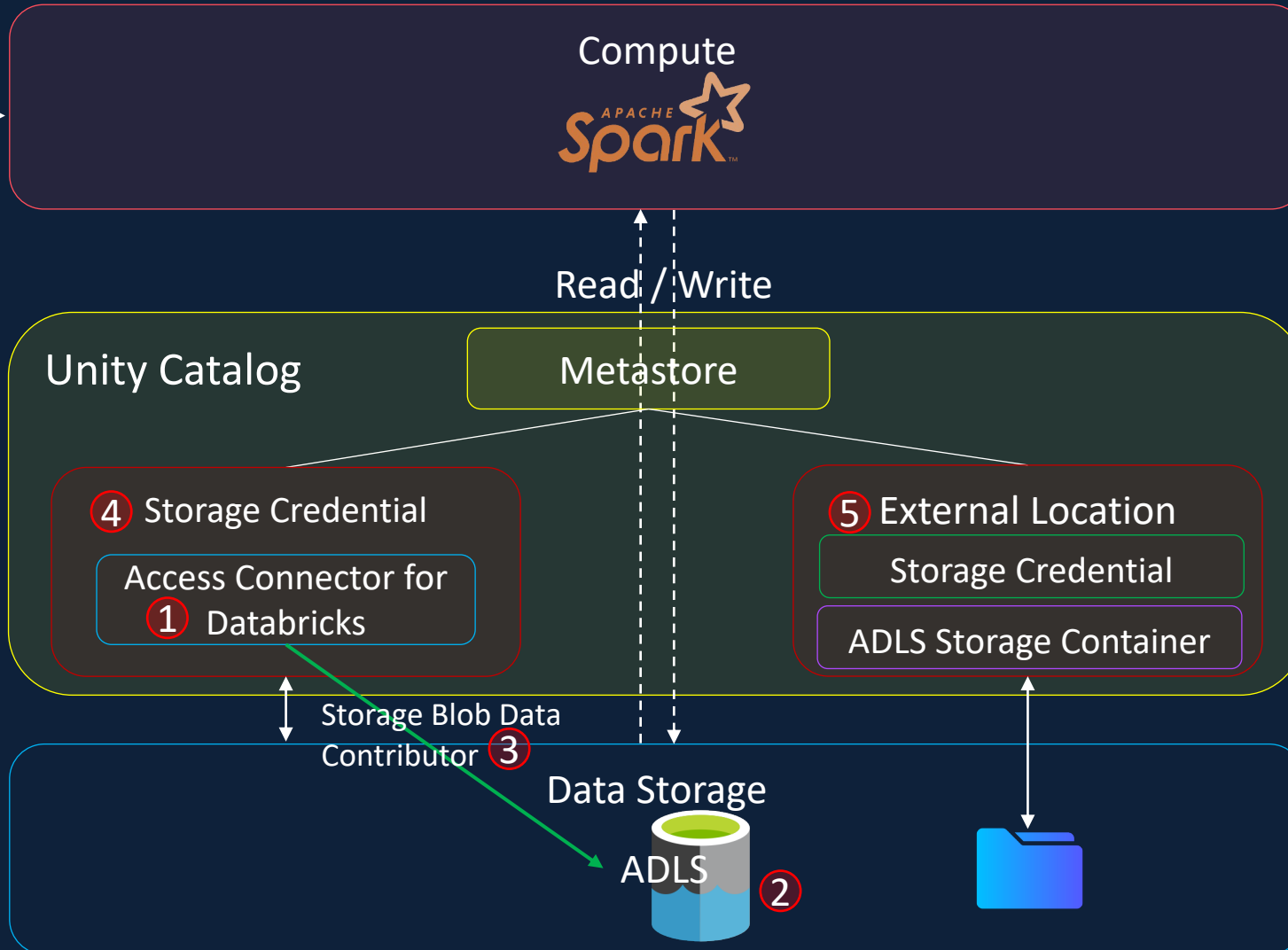
4. Create Storage Credential

5. Create External Location

Configure Access to Cloud Storage



Users and Applications



1. Create Access Connector

2. Create Azure Data Lake Storage

3. Assign Storage Blob Data Contributor role

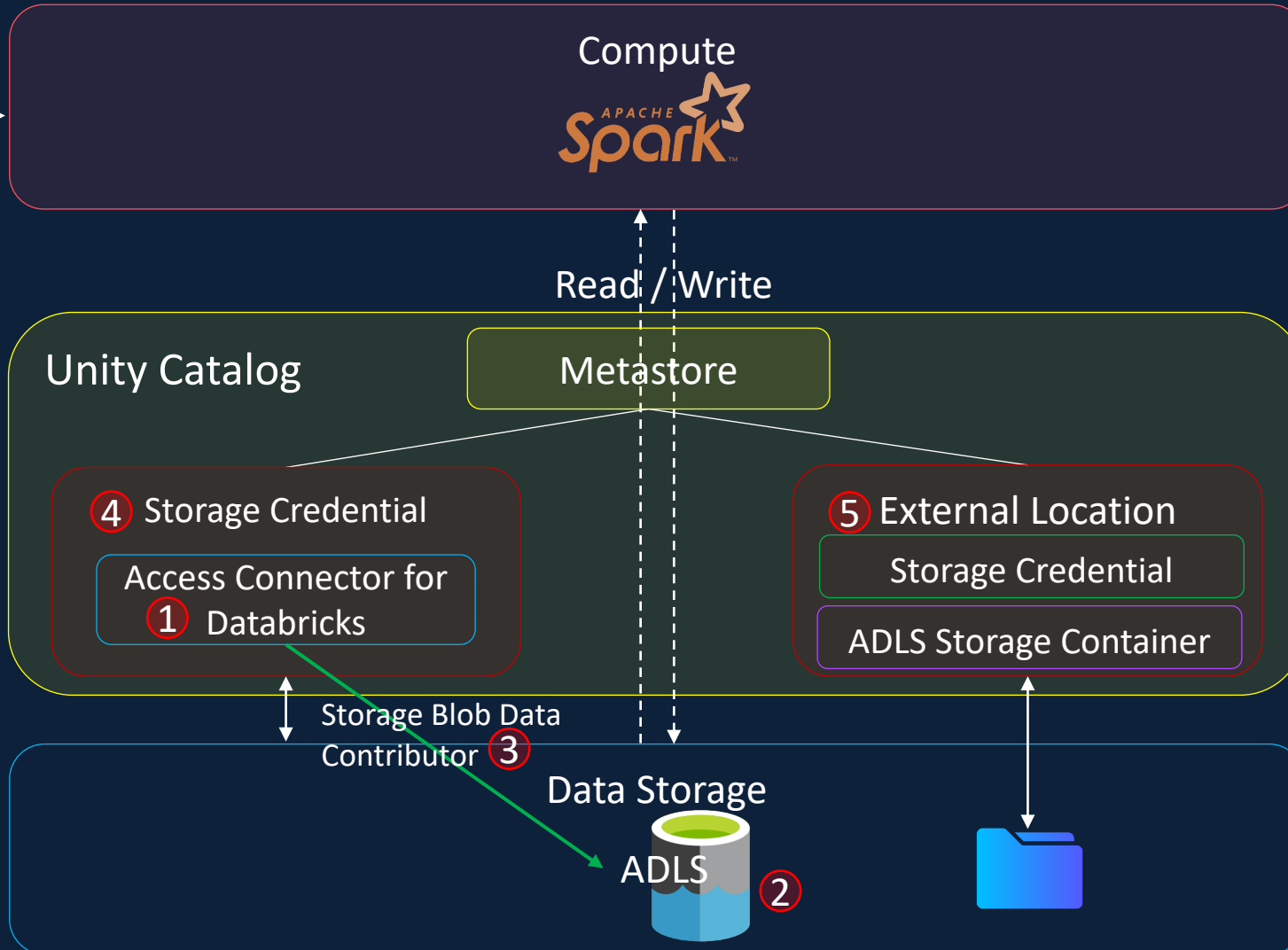
4. Create Storage Credential

5. Create External Location

Configure Access to Cloud Storage



Users and Applications



Azure

1. Create Access Connector

2. Create Azure Data Lake Storage

3. Assign Storage Blob Data Contributor role



Databricks

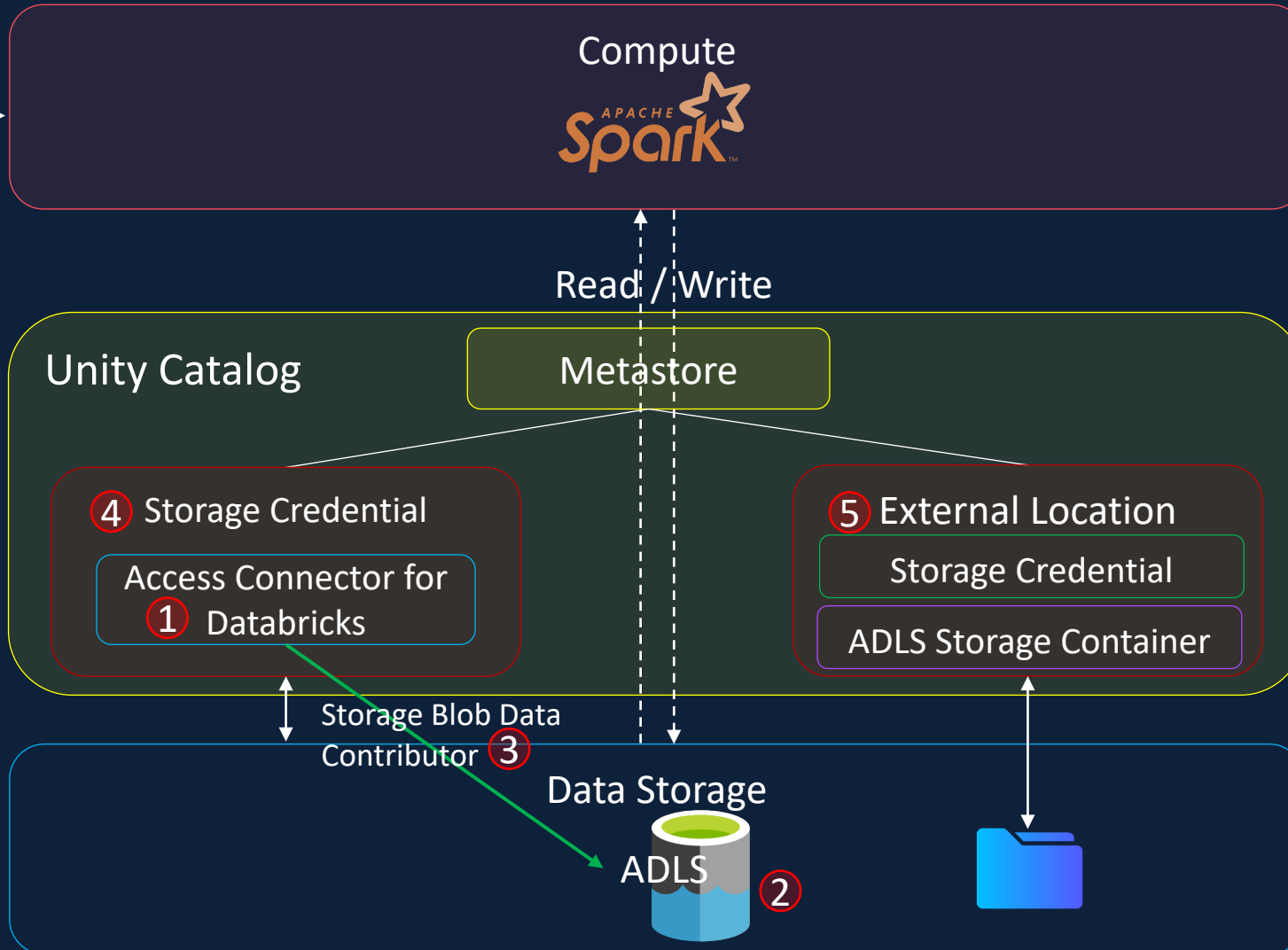
4. Create Storage Credential

5. Create External Location

Configure Access to Cloud Storage



Users and Applications



Azure

1. Create Access Connector

2. Create Azure Data Lake Storage

3. Assign Storage Blob Data Contributor role



Databricks

4. Create Storage Credential

5. Create External Location



Formula1 Project Overview

Formula1 Project Overview



Introduction to Formula1

Understanding Formula1 Data

Defining Project Requirements

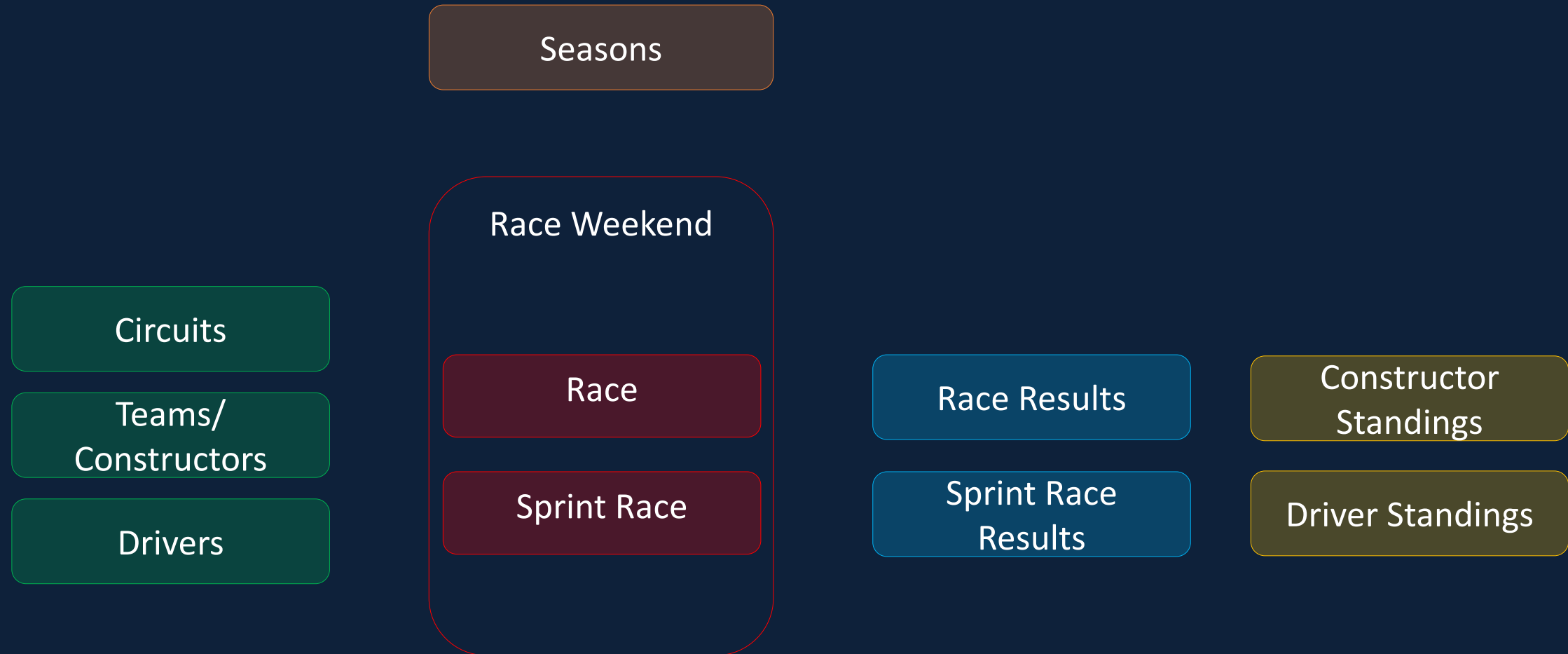


Formula1

Formula1 Overview



Formula1 Overview



Formula1 – Data Source



Data Sourced from Github Repo - [jolpica/jolpica-f1](https://github.com/jolpica/jolpica-f1)

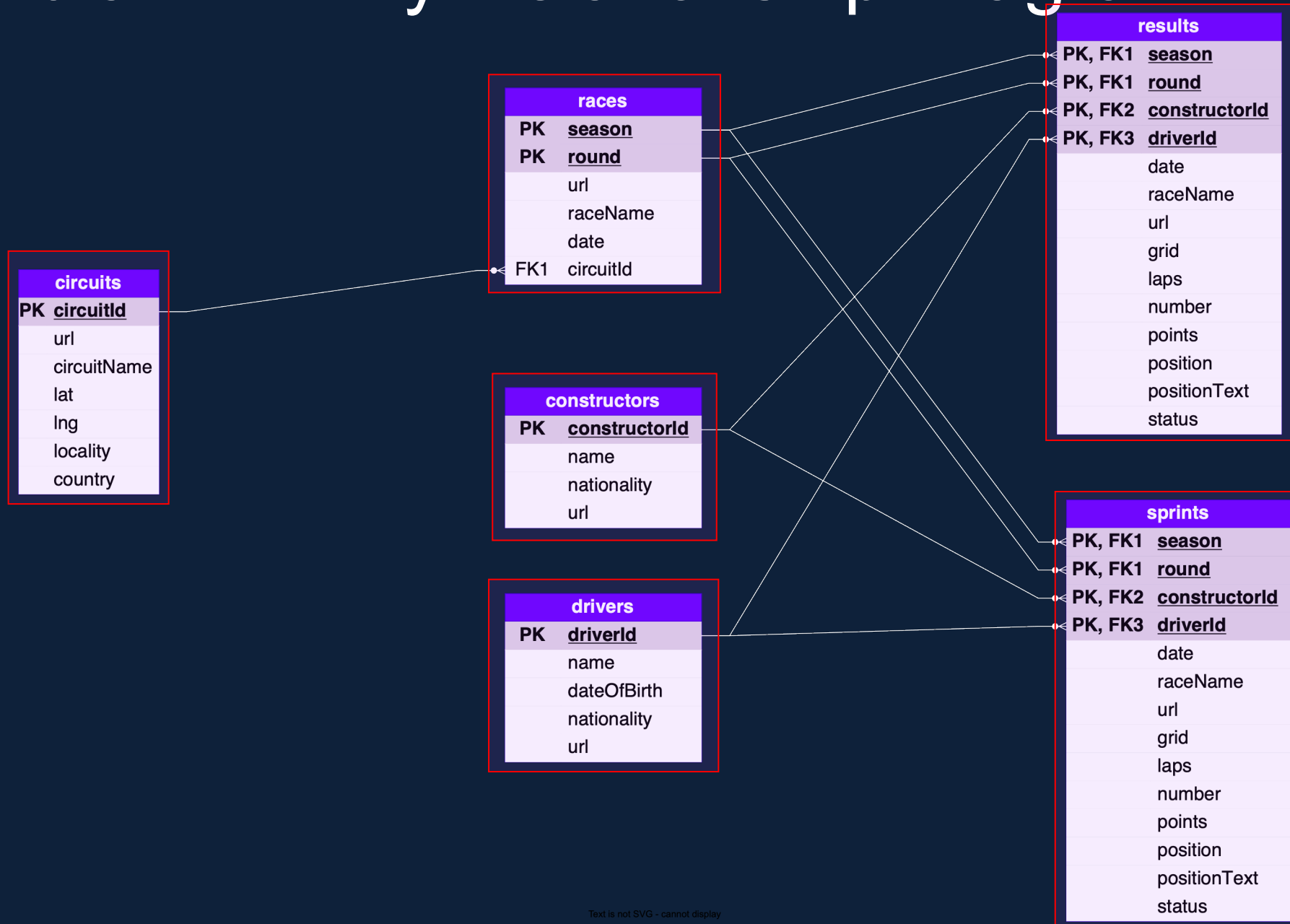
Open-source API with multiple formats

Using the relational style Ergast format

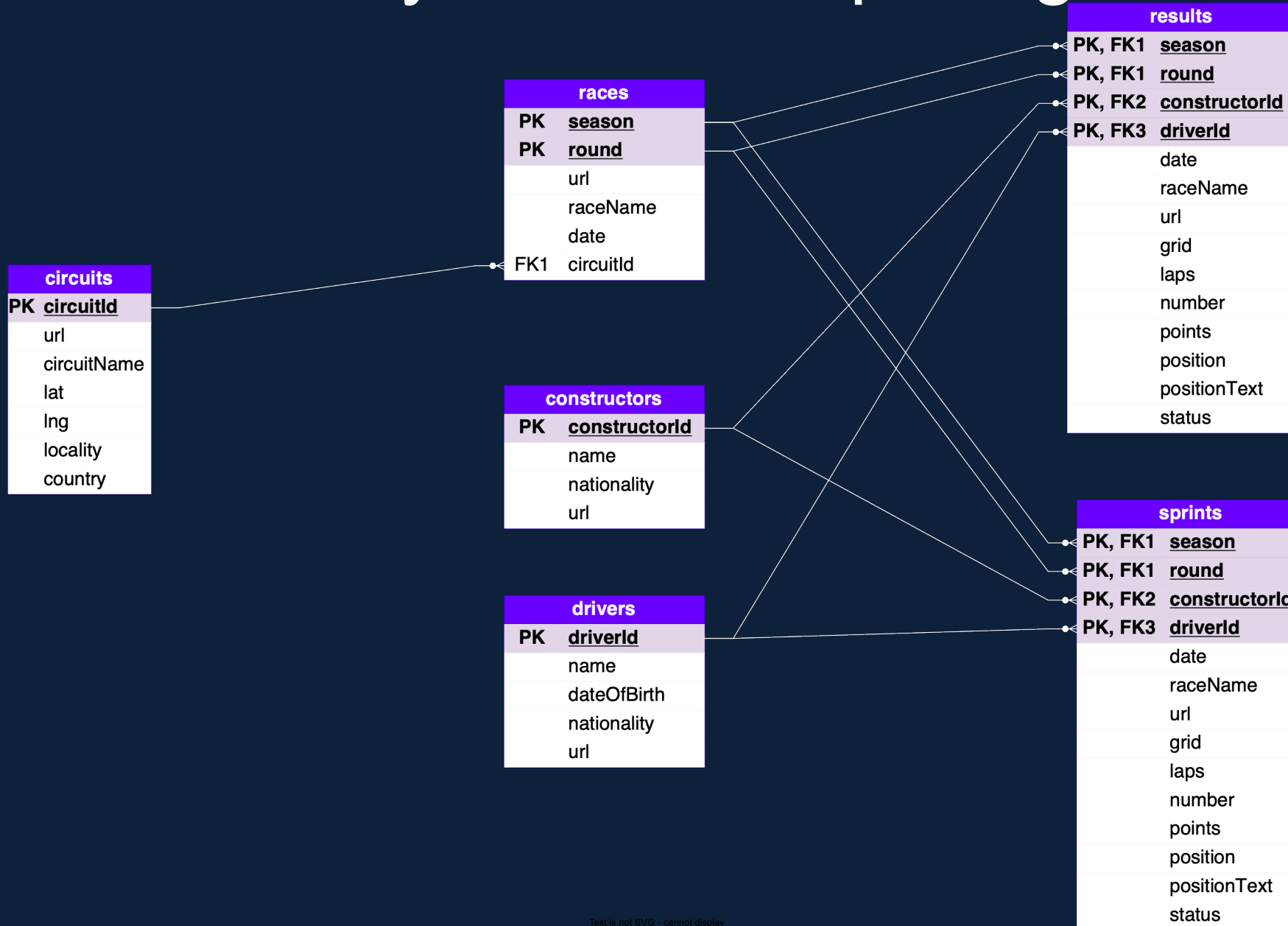
Apache 2.0 open-source license permitting educational use

Provided in multiple formats for learning purposes

Formula1 - Entity Relationship Diagram



Formula1 - Entity Relationship Diagram



Text is not SVG - cannot display

Formula1 – Data Files



Circuits	CSV
Races	CSV
Constructors	Single Line JSON
Drivers	Single Line Nested JSON
Results	Single Line JSON (Multiple Files)
Sprints	Multi Line JSON (Multiple Files)



Project Requirements

Project Requirements



Data Ingestion Requirements

Data Transformation Requirements

Reporting & Analytical Requirements

Non-Functional Requirements

Data Ingestion Requirements



Ingest all 6 datasets into the Data Lakehouse

Apply correct schema (columns and data types)

Add audit columns (ingestion timestamp, source file)

Store data in Delta format

Preserve Data Integrity and Reliability

Start with full load → extend to incremental load later

Data Transformation Requirements



Clean and standardise data

Apply consistent naming and reshape structures

Remove unnecessary columns and handle basic data quality

Preserve business keys across layers

Prepare datasets for Gold-layer analytics

Reporting & Analytical Requirements



Driver standings for each season

Constructor standings for each season

Analyse dominant drivers and teams

Support recent and historical analysis

Optimised for reporting and analytical queries

Scheduling Requirements



Scheduled and reliable pipeline execution

Monitoring, recovery, and alerting

GDPR compliance and data correction

Time travel, rollback, and Delta reliability

Project Requirements



Ingest multiple datasets using Delta

Transform data into structured, trusted layers

Produce analytical outputs for reporting

Design for reliability, governance, and versioning



Formula1 Solution Overview

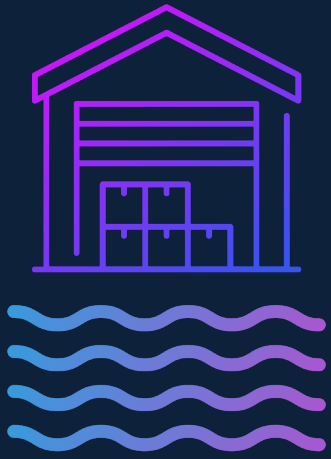
Formula1 Solution Overview



Introduction to Data Lakehouse

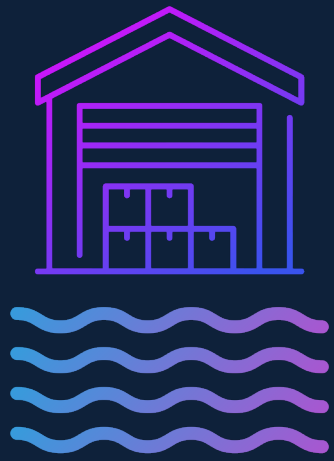
Understanding Medallion Architecture

Formula1 Project Solution Architecture



What is Data Lakehouse?

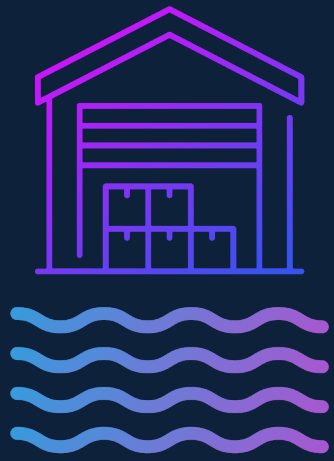
What is Data Lakehouse?



A Data Lakehouse is a new, open data management architecture that combines the flexibility, cost-efficiency, and scale of Data Lakes with the data management and ACID transactions of Data Warehouses, enabling business intelligence (BI) and machine learning (ML) on all data.

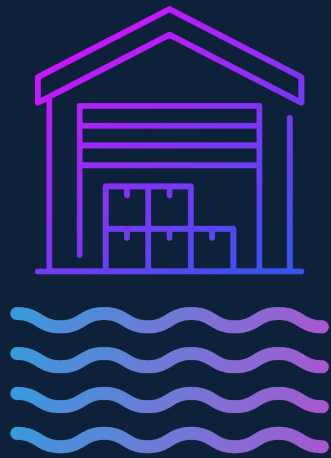
- *Databricks*

What is Data Lakehouse?



A Data Lakehouse is a new, open data management architecture that combines the flexibility, cost-efficiency, and scale of Data Lakes with the data management and ACID transactions of Data Warehouses, enabling business intelligence (BI) and machine learning (ML) on all data.

What is Data Lakehouse?



Data Lakehouse

=



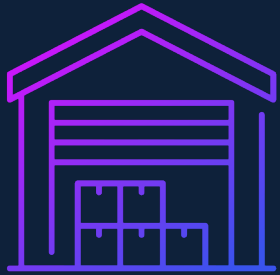
Data Lake

+



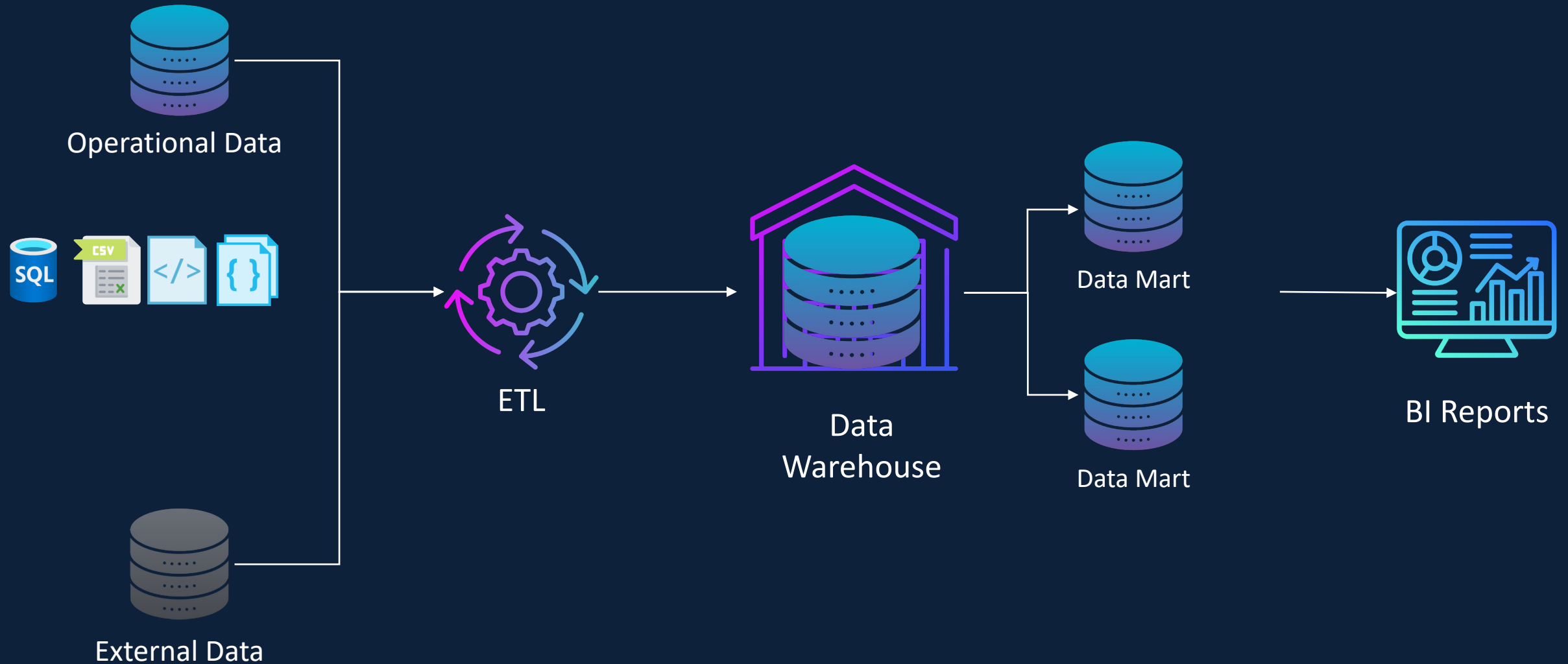
Data Warehouse

A Data Lakehouse is a new, open data management architecture that combines the flexibility, cost-efficiency, and scale of Data Lakes with the data management and ACID transactions of Data Warehouses, enabling business intelligence (BI) and machine learning (ML) on all data.

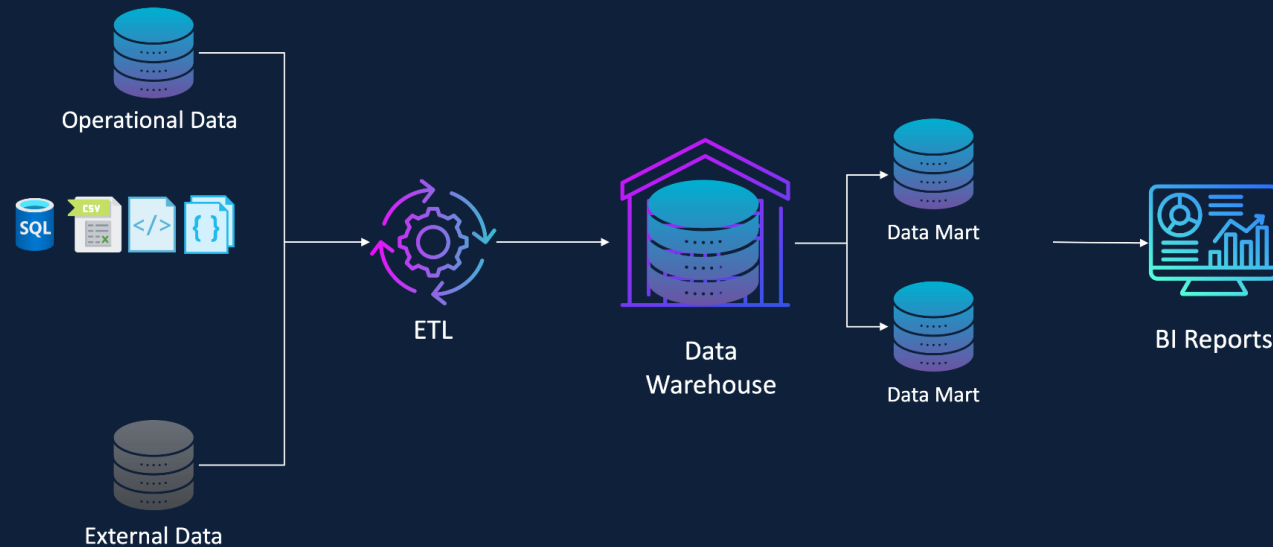


What is Data Warehouse?

What is Data Warehouse?



What is Data Warehouse?



Increased Data Volume & Variety

Longer Time to Ingest New Data

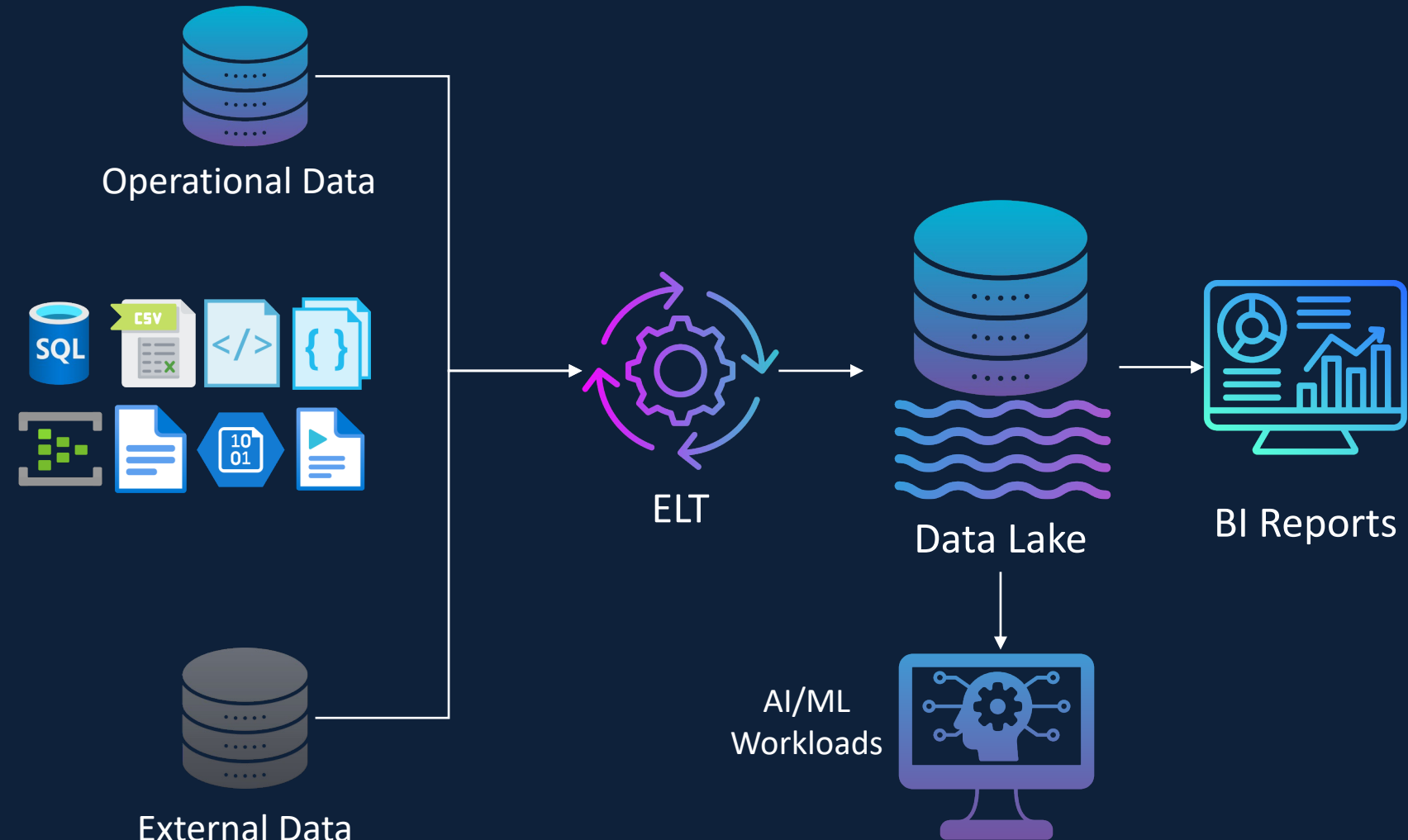
Proprietary Data Formats /Vendor Lock-in

Scalability Issues

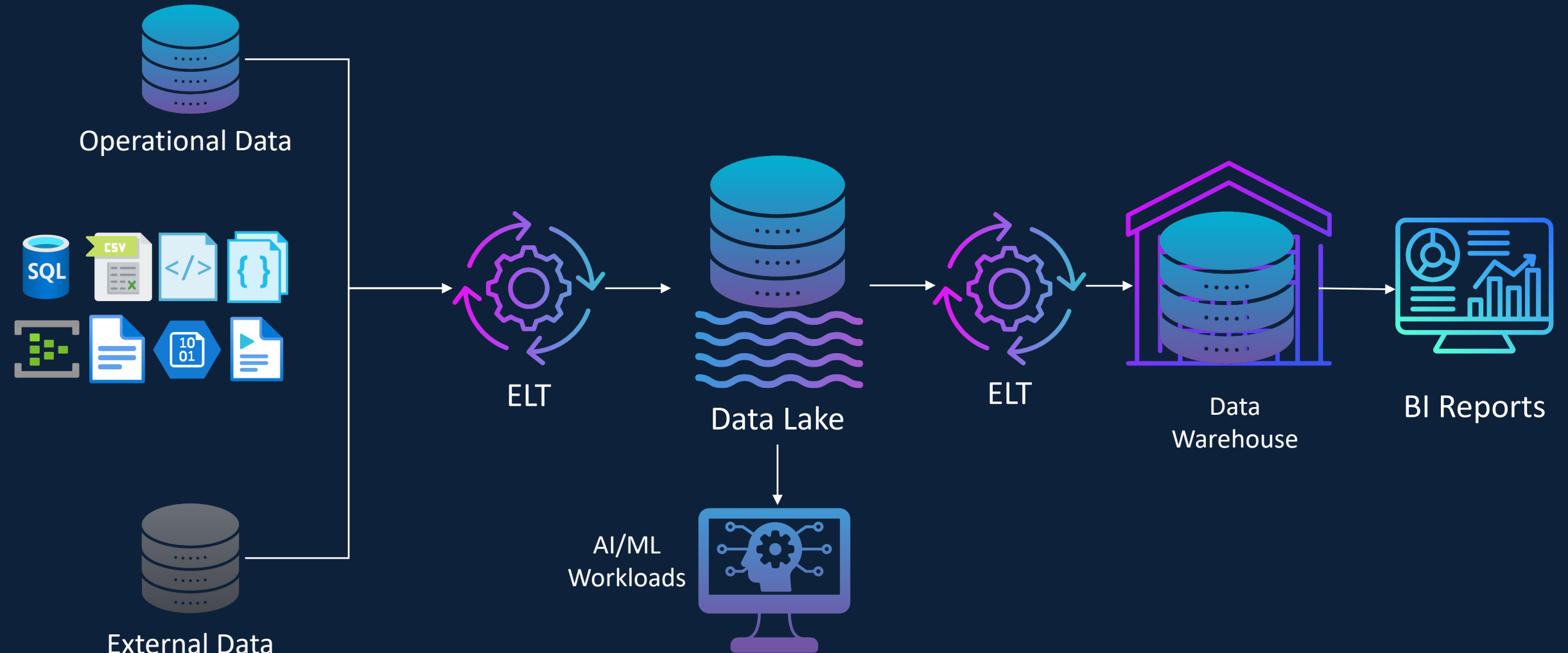
High Storage Costs

Limited Support for Advanced Analytics

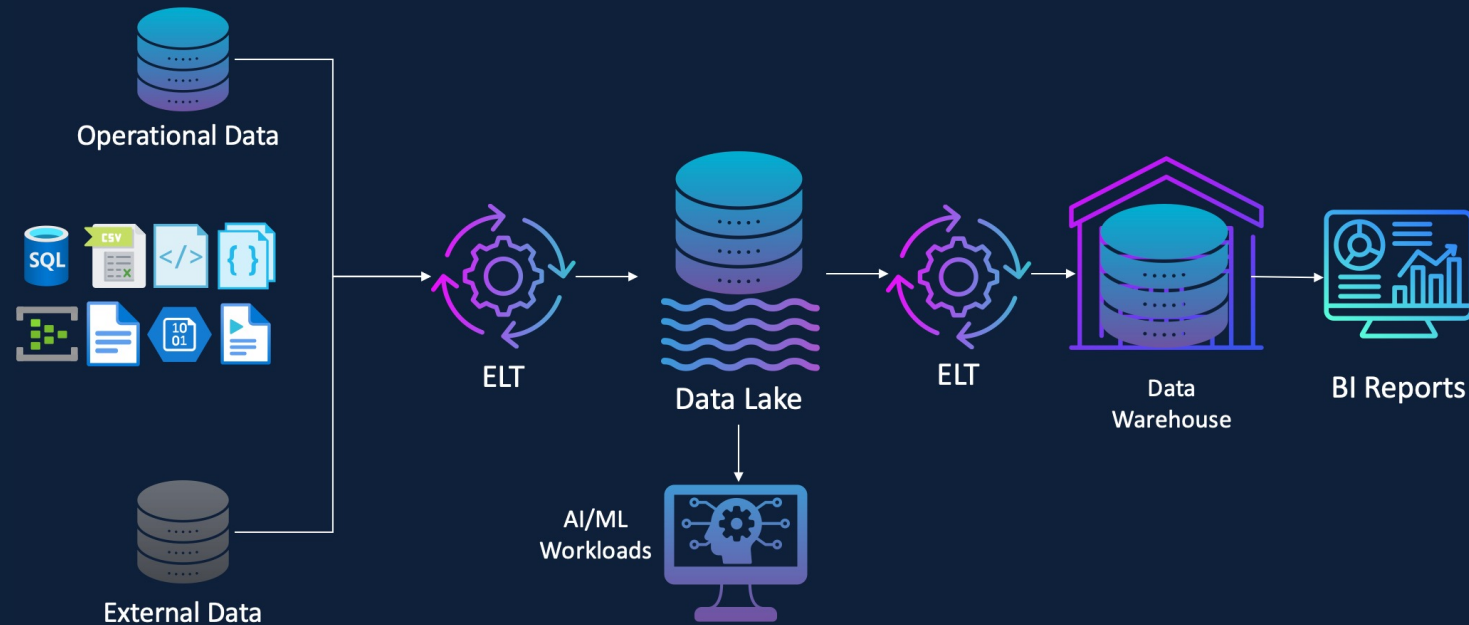
What is Data Lake?



What is Data Lake?



What is Data Lake?



No Support for ACID Transactions

Partial Data Loads

Inconsistent Reads

Complicated Data Corrections

No Rollback Capability

GDPR Challenges

No Data Versioning

Poor BI Performance and Support

Complex to set-up

Streaming vs. Batch Processing

What is Data Lakehouse?



Data Lake



Data Warehouse

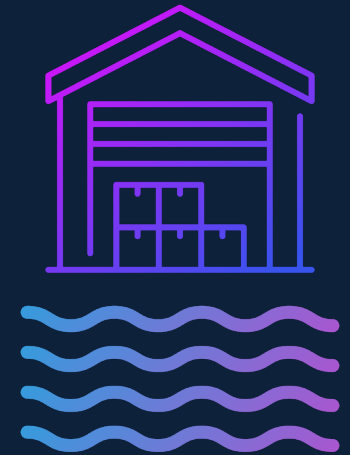
What is Data Lakehouse?



Data Lake

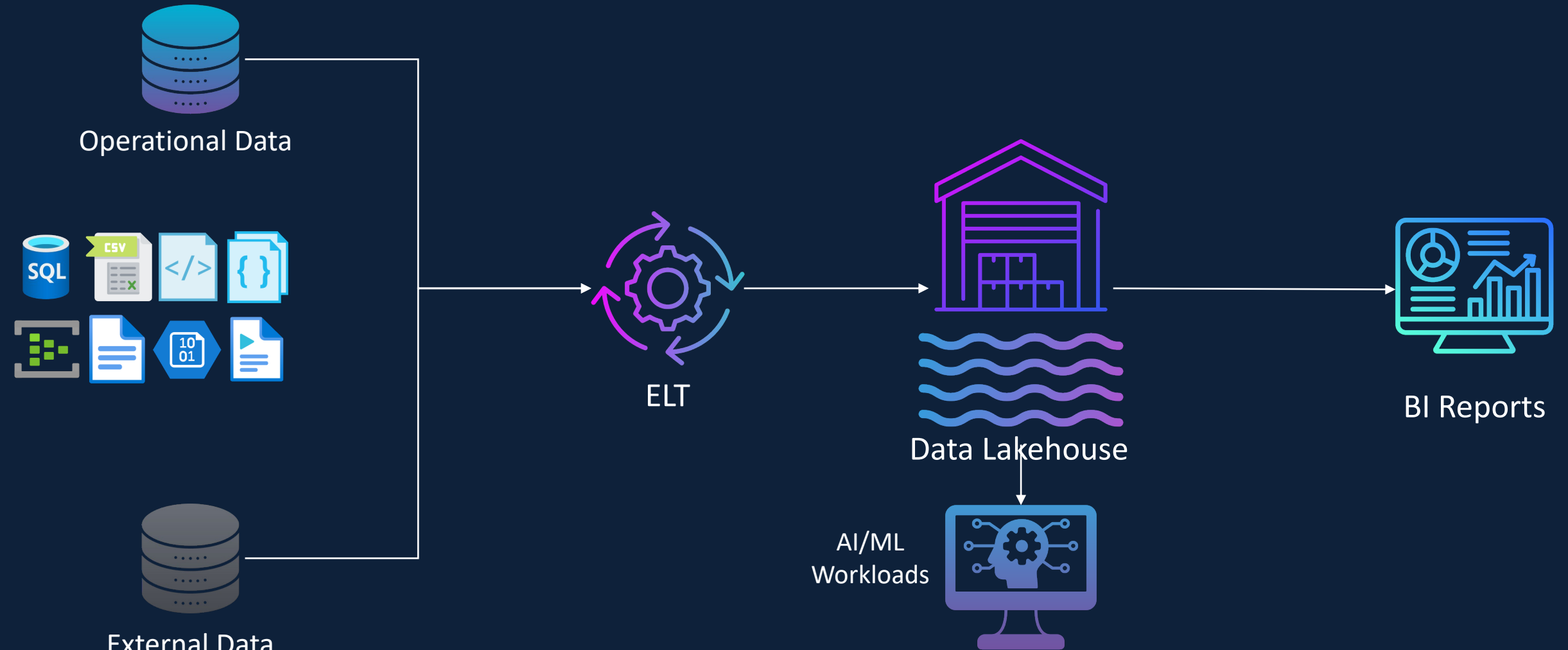


Data Warehouse

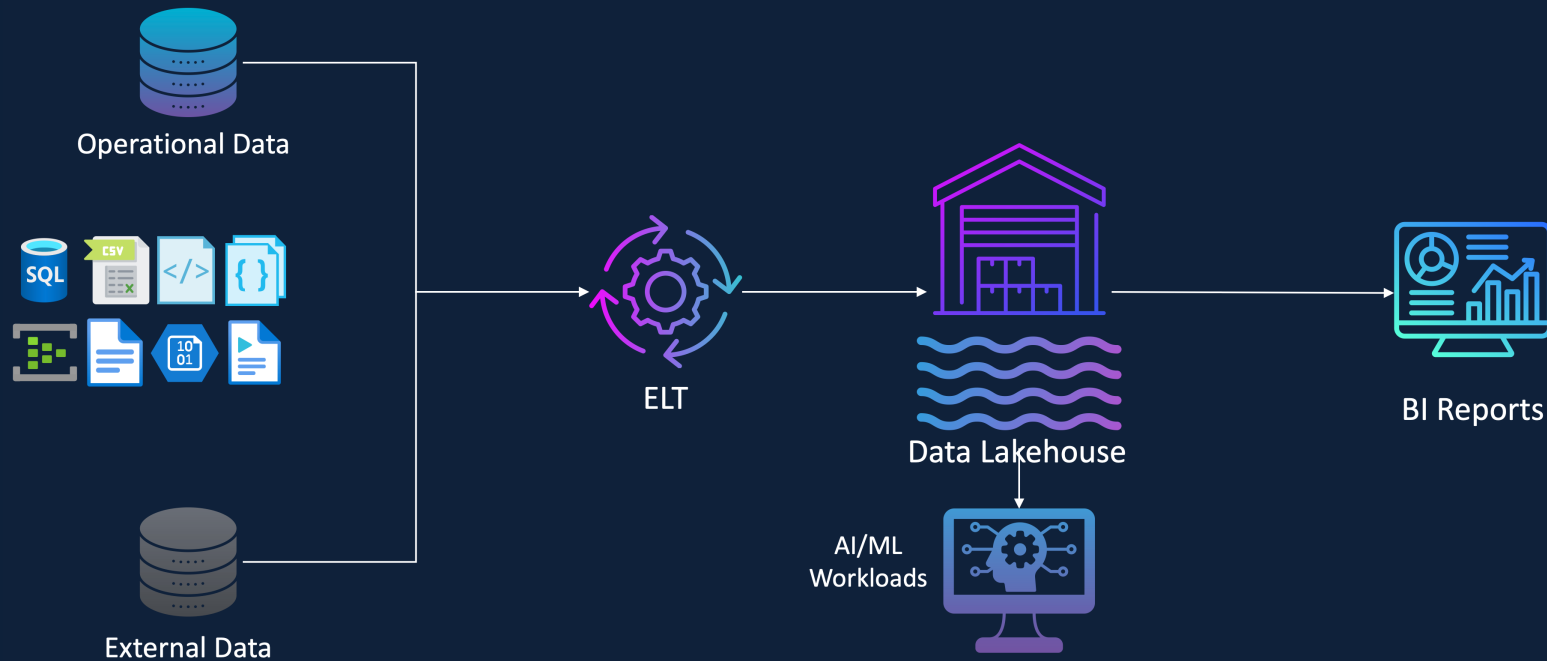


Data Lakehouse

What is Data Lakehouse?



What is Data Lakehouse?



Handles All Types of Data

Cheap Cloud Object Storage

Uses Open File Format

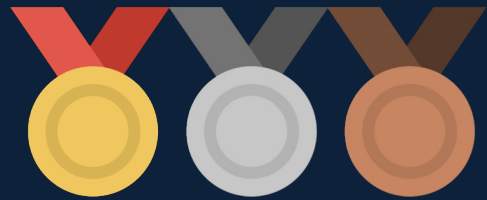
Support for All Workloads

Direct BI Integration

ACID Support & Version History

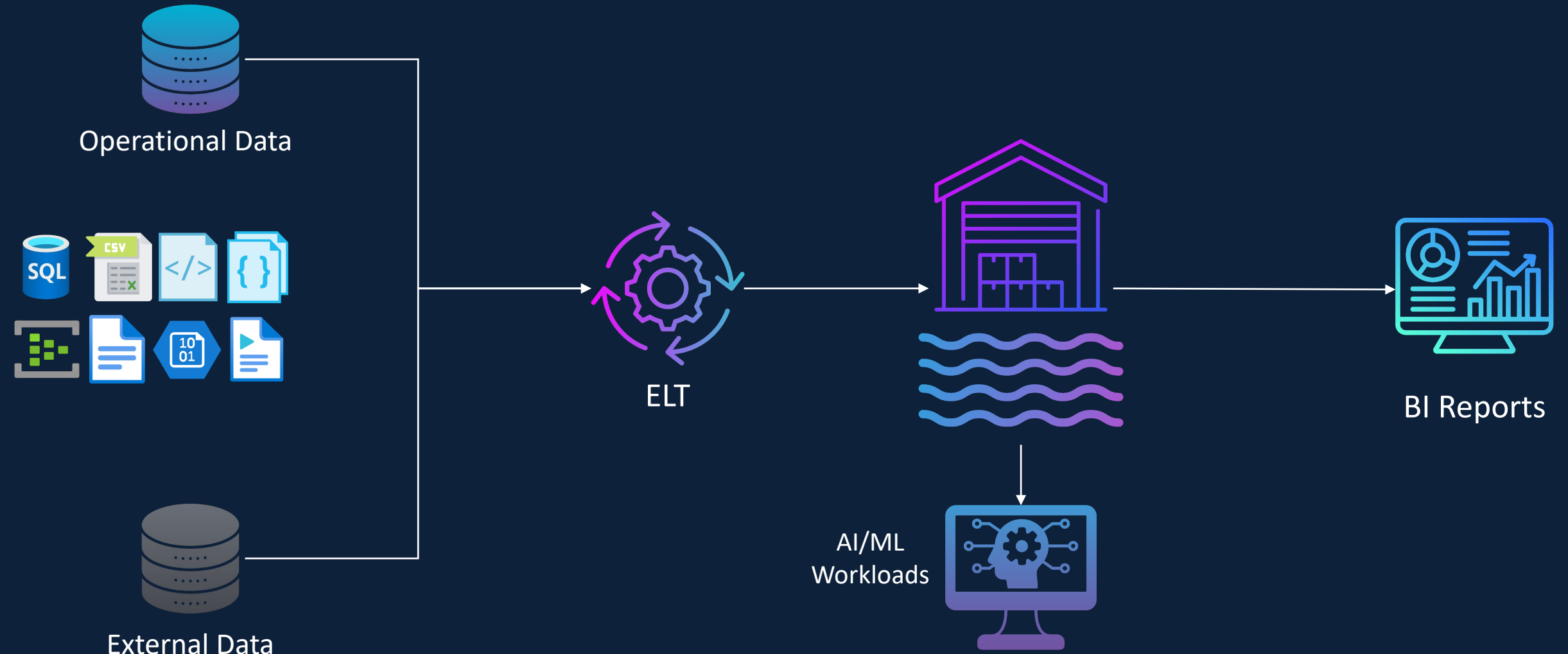
Improved Performance

Simple Architecture

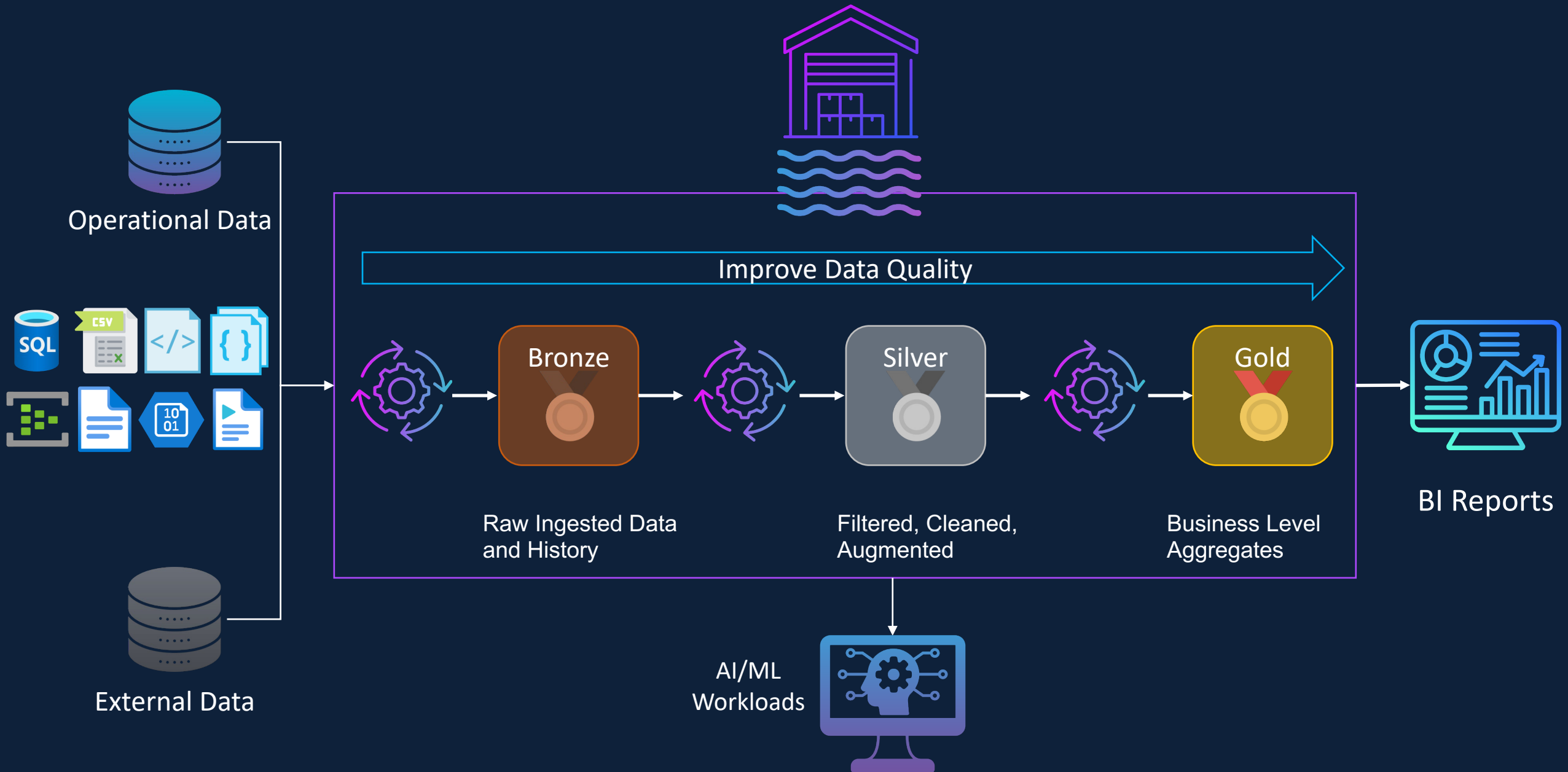


What is Medallion Architecture?

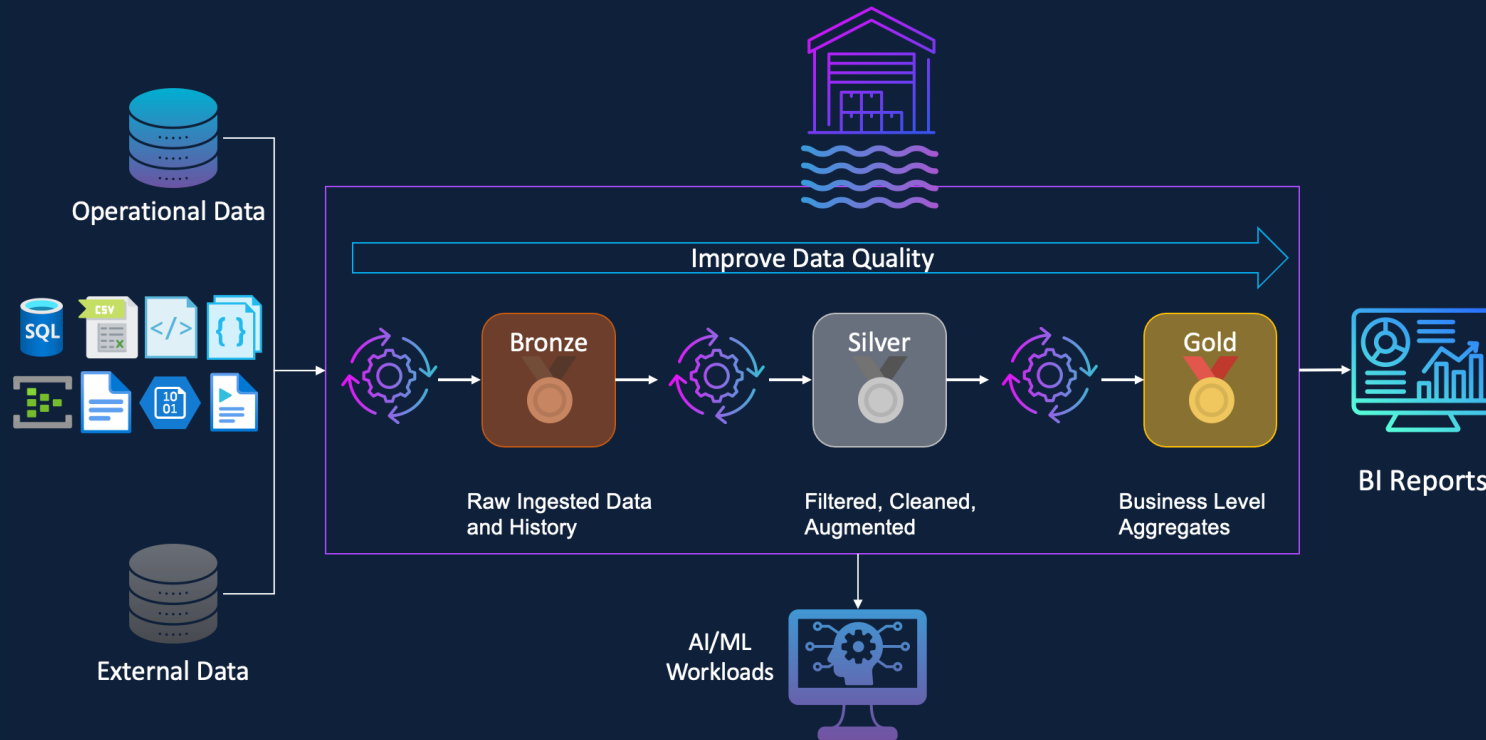
Medallion Architecture



Medallion Architecture



Medallion Architecture - Benefits



Data Lineage & Traceability

Data Governance & Compliance

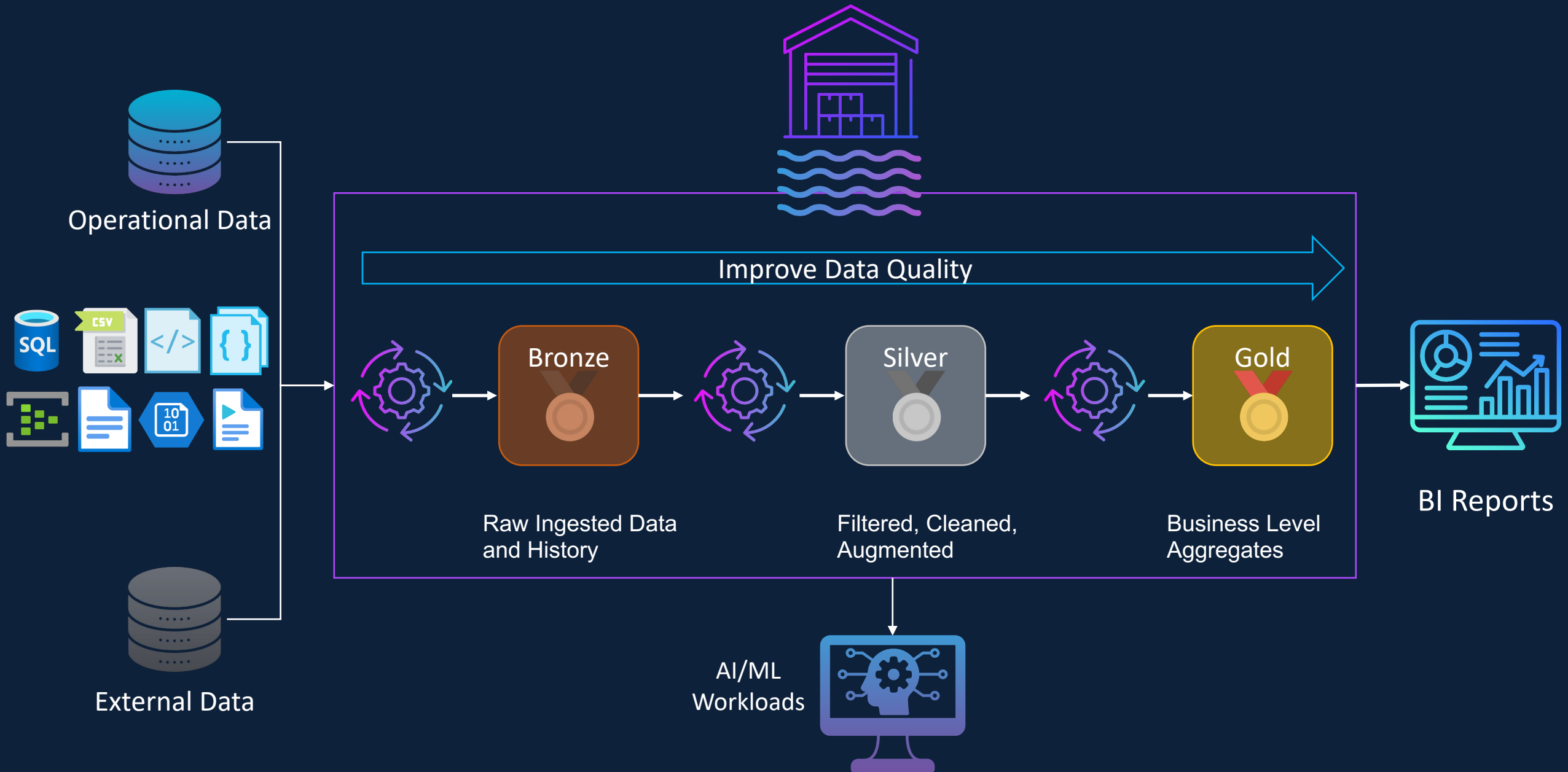
Support for Incremental Processing

Enhanced Performance & Scalability

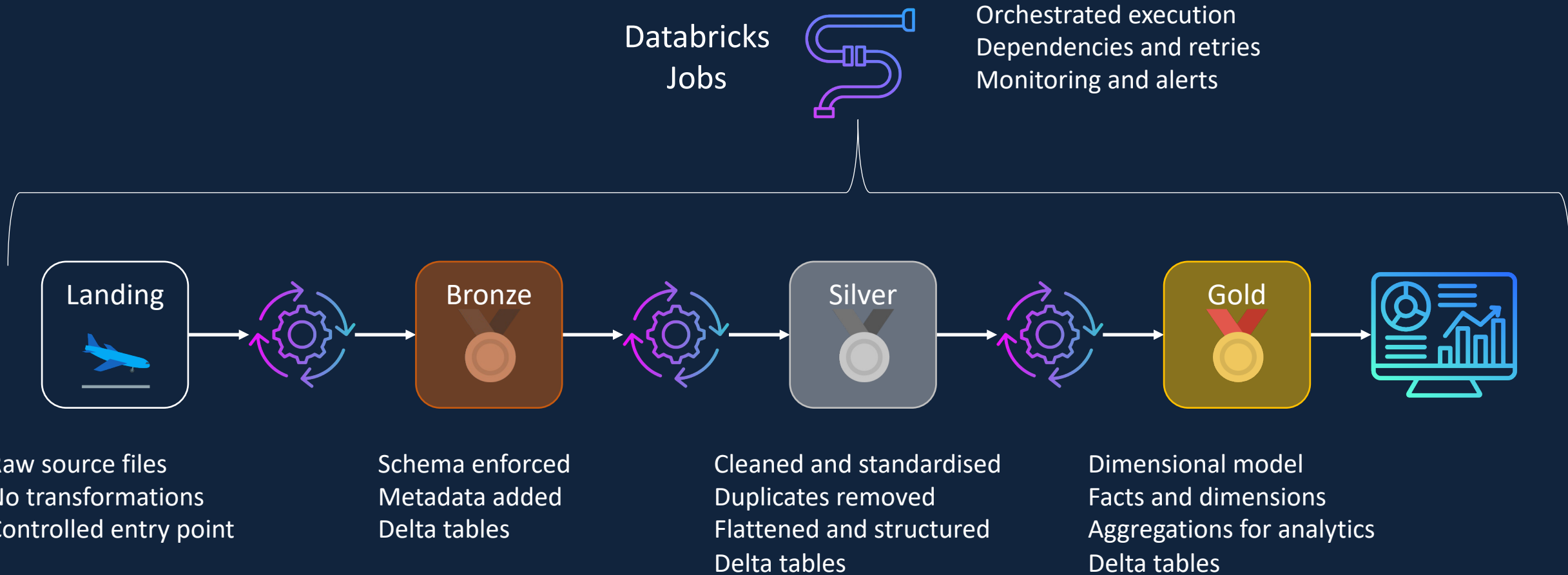
Data Security

Solution Architecture

Solution Architecture



Solution Architecture





Formula1 Project Environment Set-up

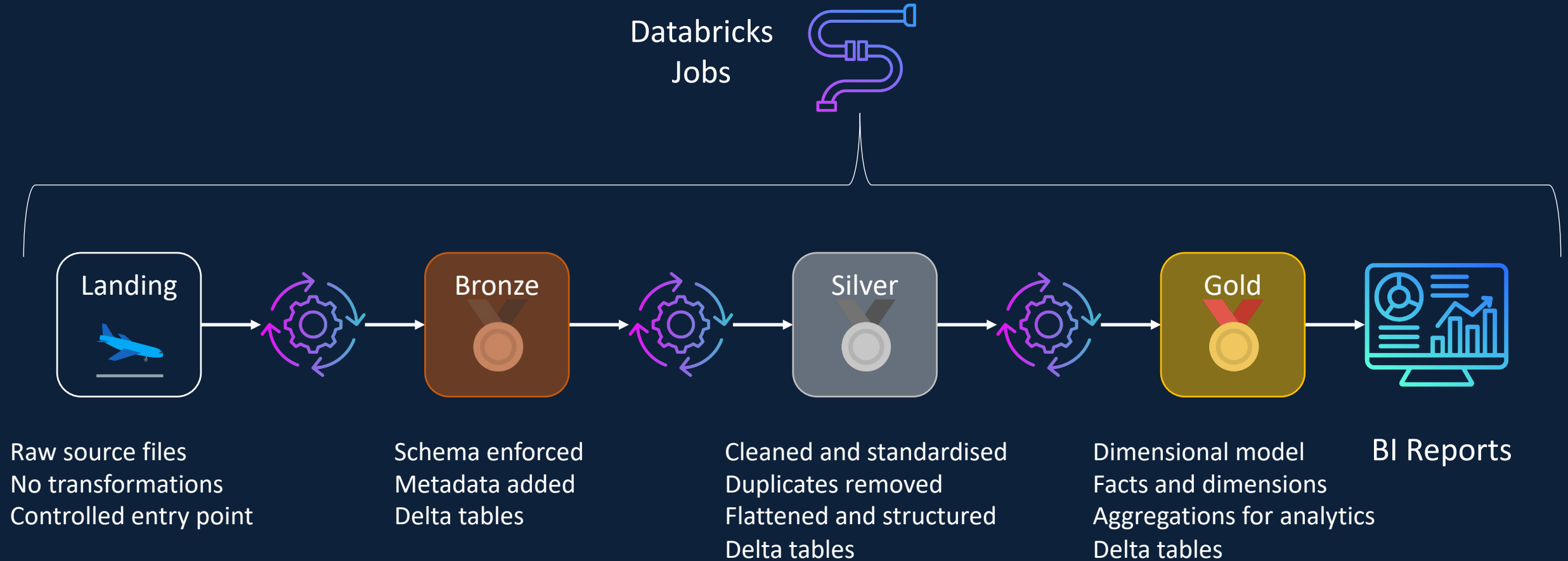
Formula1 Project Environment Set-up



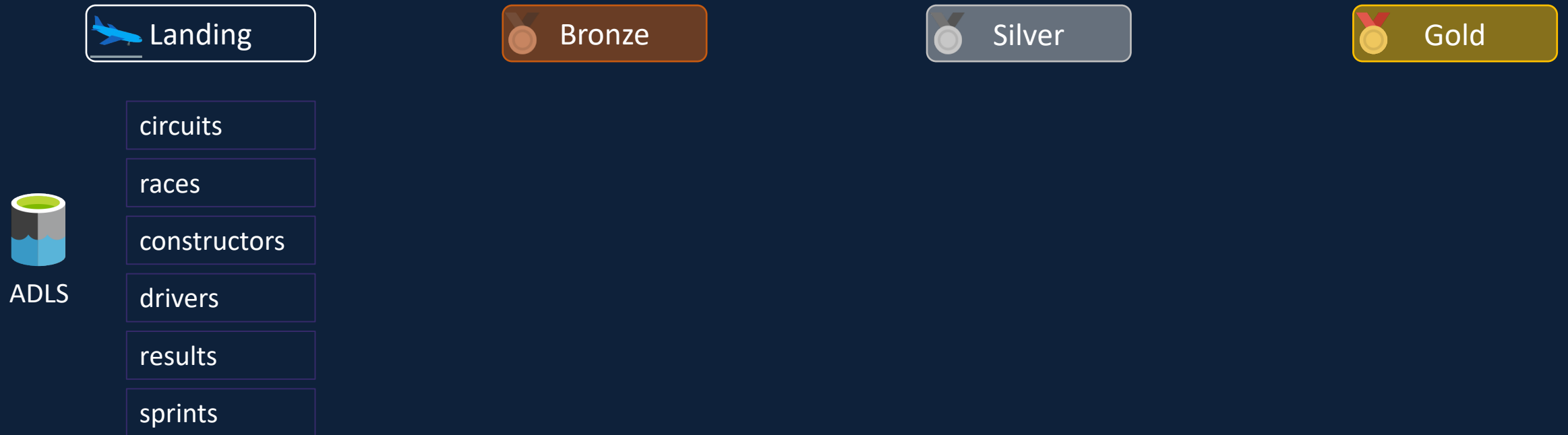
Set up Data Lake environment

Configure Unity Catalog

Data Architecture

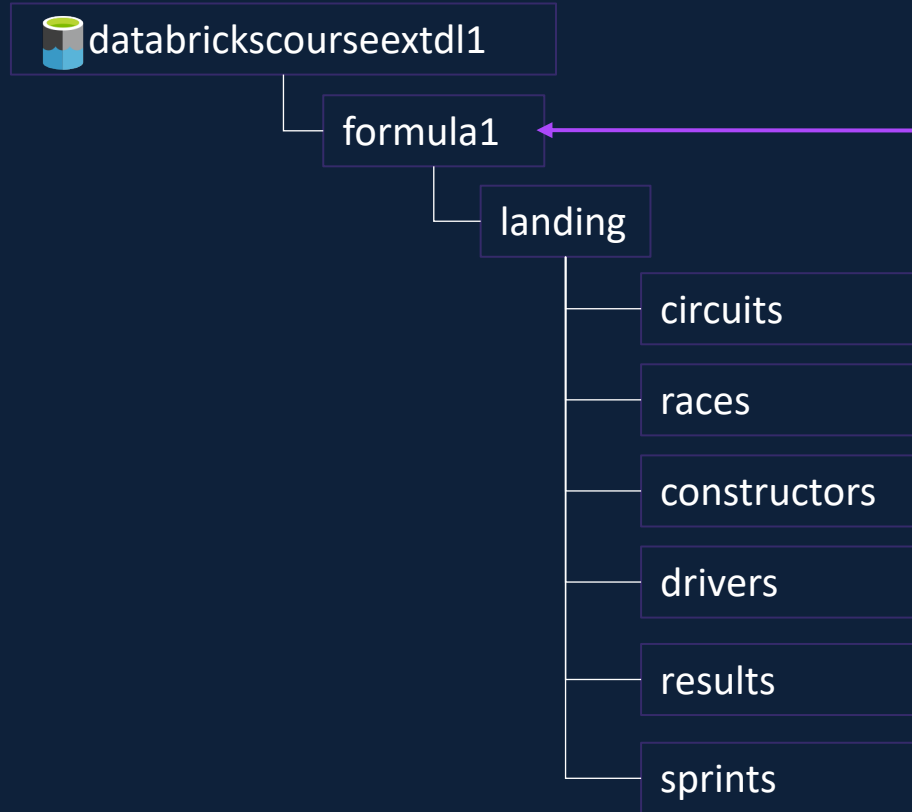


Data Architecture



Data Architecture

Azure Data Lake Storage Set-Up



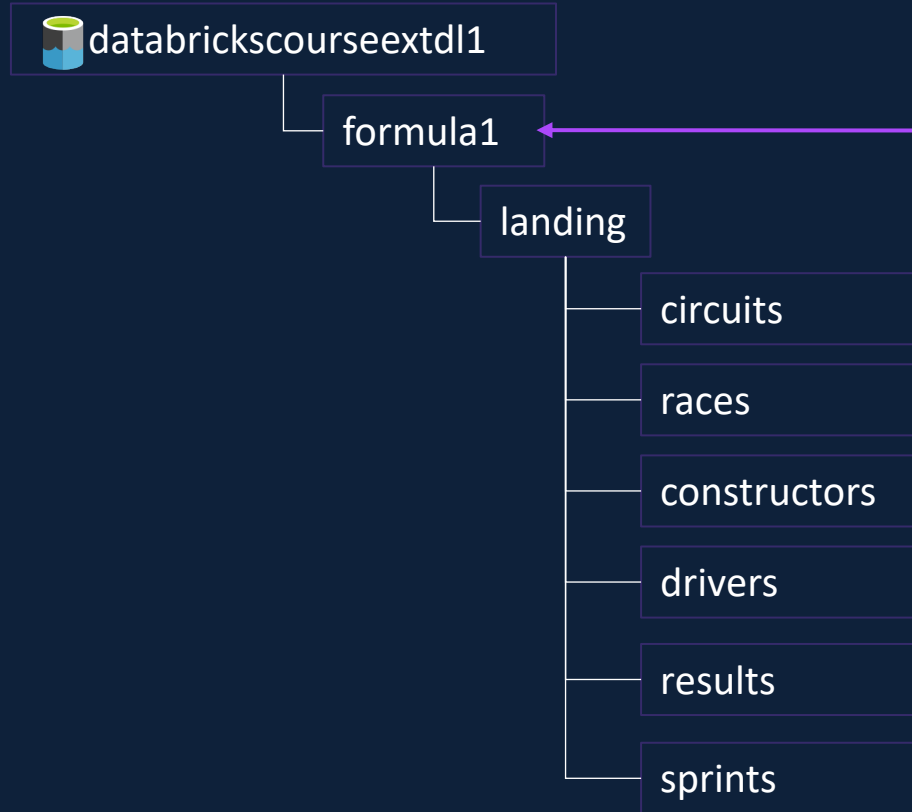
Unity Catalog Set-up

External Location

`databricks_course_ext_dl_formula1`

Data Architecture

Azure Data Lake Storage Set-Up

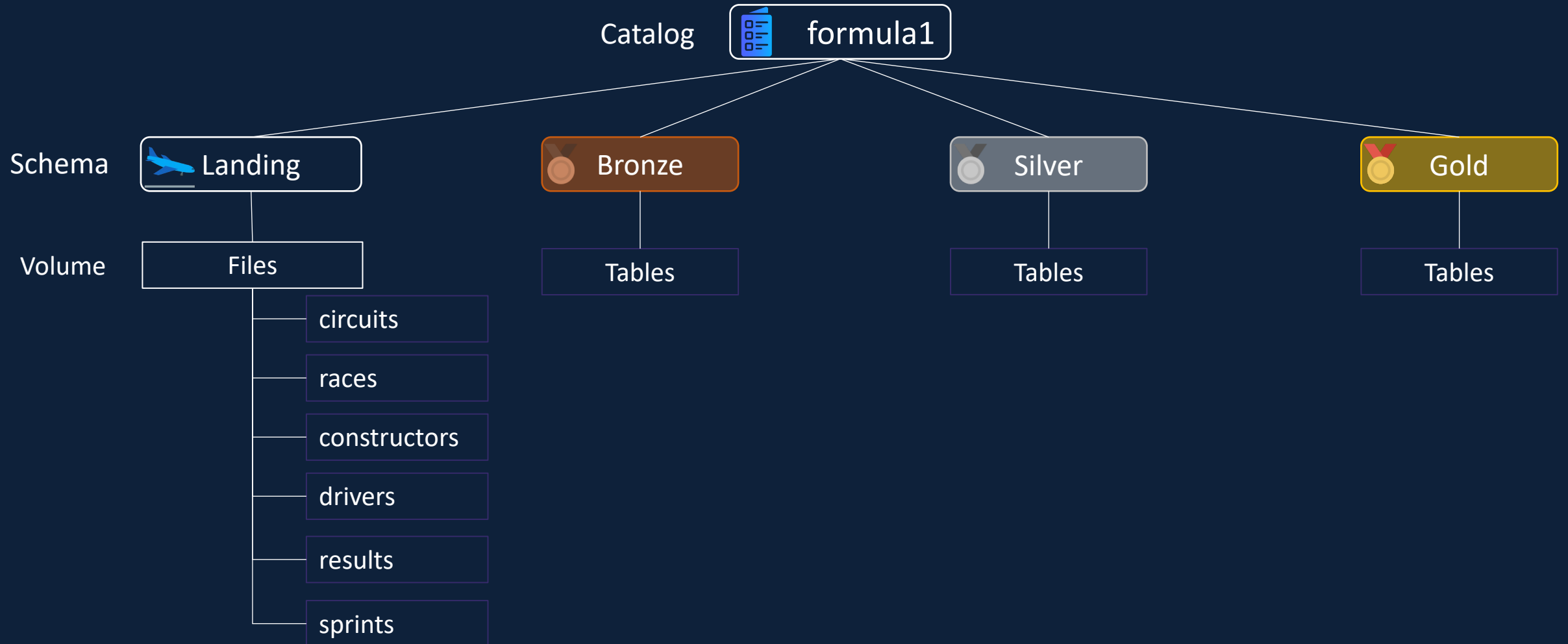


Unity Catalog Set-up

`databricks_course_ext_dl_formula1`

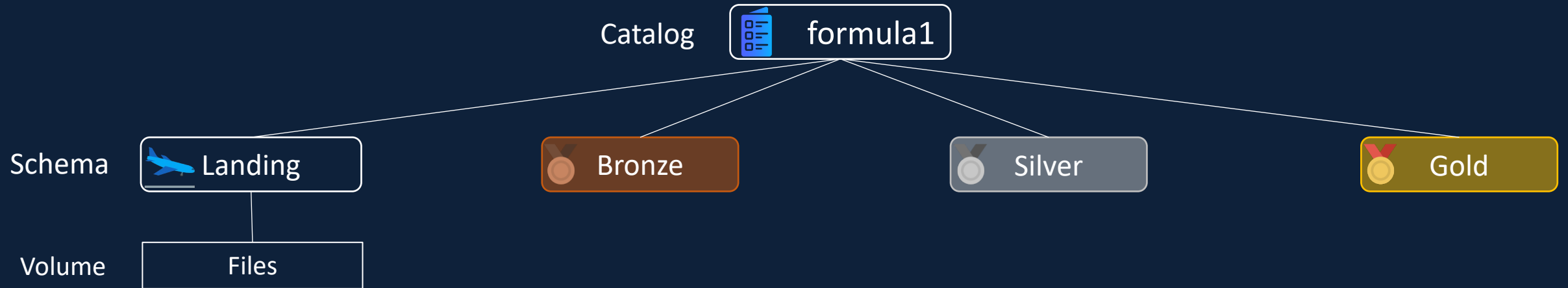
Data Architecture

Unity Catalog



Data Architecture

Unity Catalog



Data Ingestion Overview

Data Ingestion Overview



Ingest all 6 datasets into the Data Lakehouse

Apply correct schema (columns and data types)

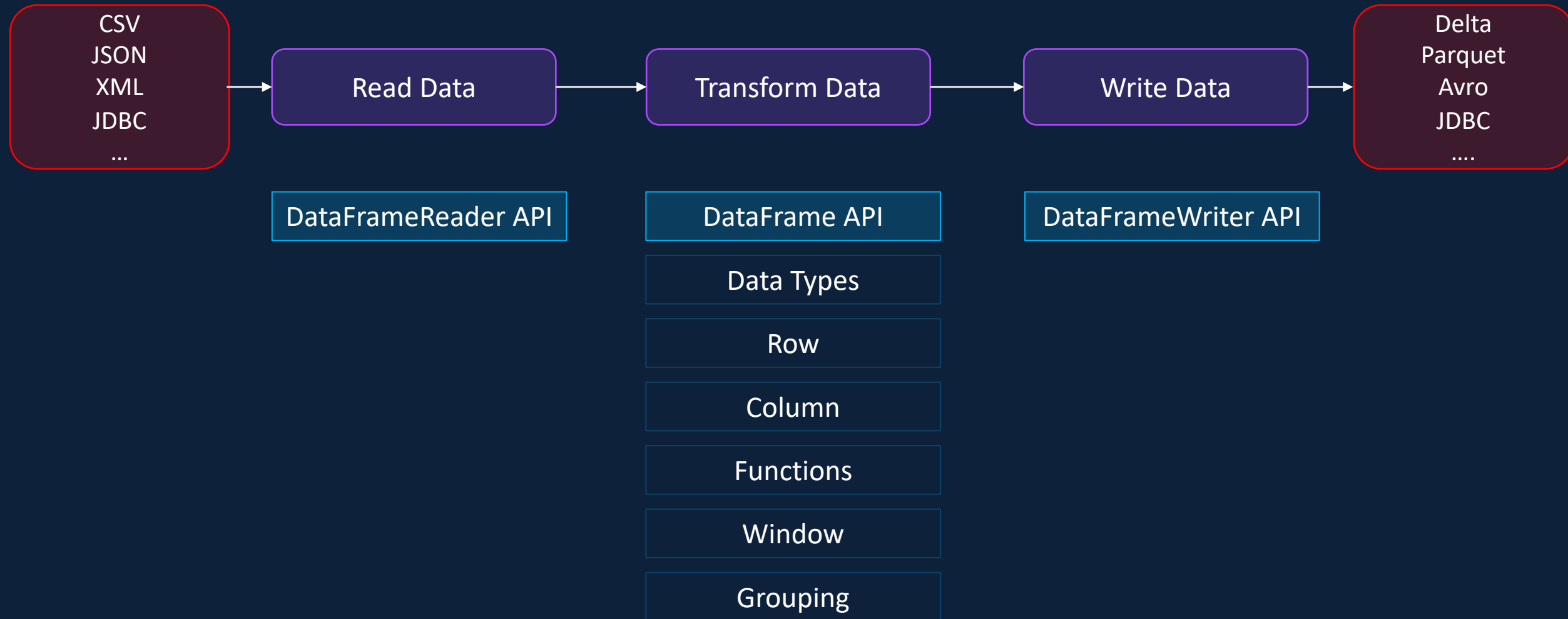
Add audit columns (ingestion timestamp, source file)

Store data in Delta format

Preserve Data Integrity and Reliability

Start with full load → extend to incremental load later

Data Ingestion Overview



Data Ingestion Overview

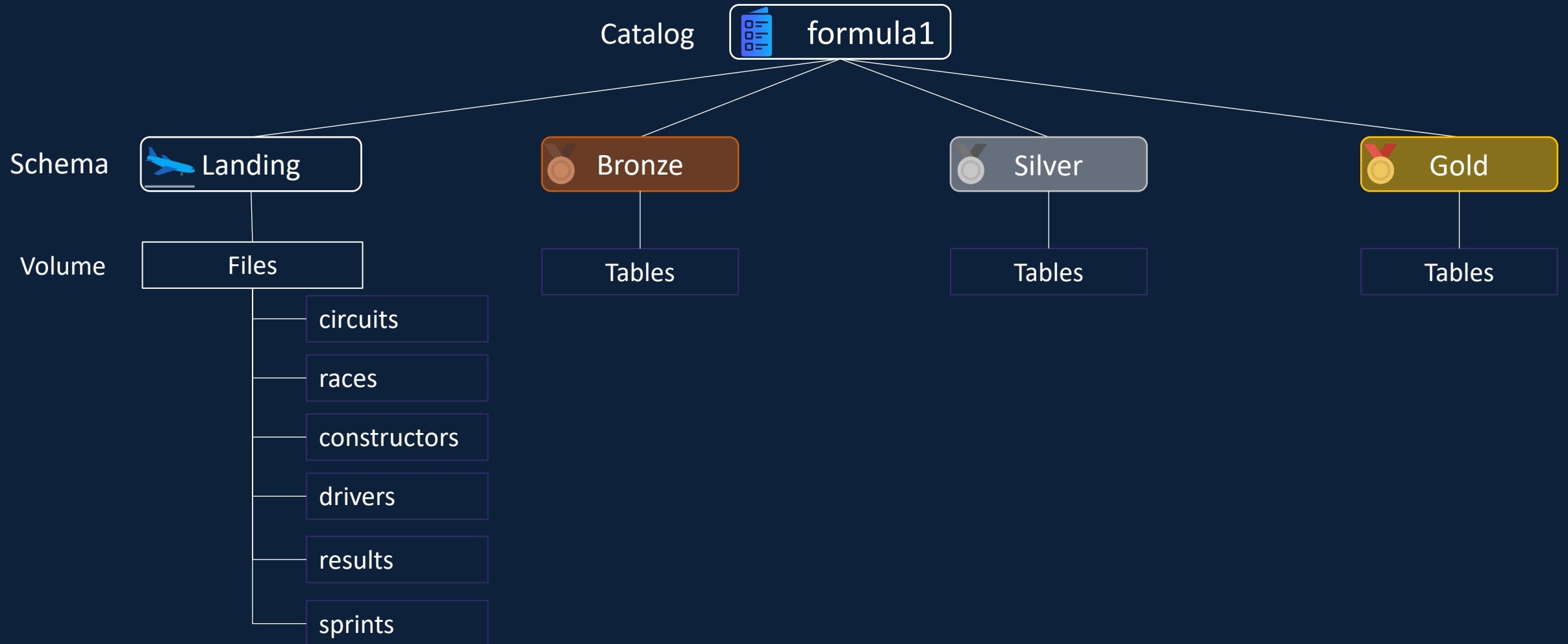


Circuits File Ingestion Step by Step

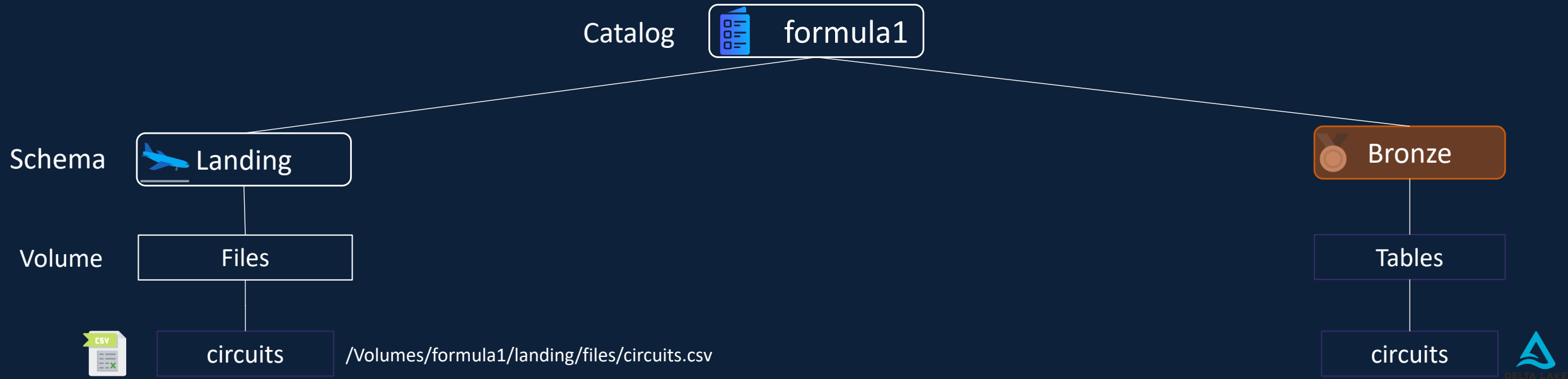
Refactor for reusable code

Apply pattern to all datasets

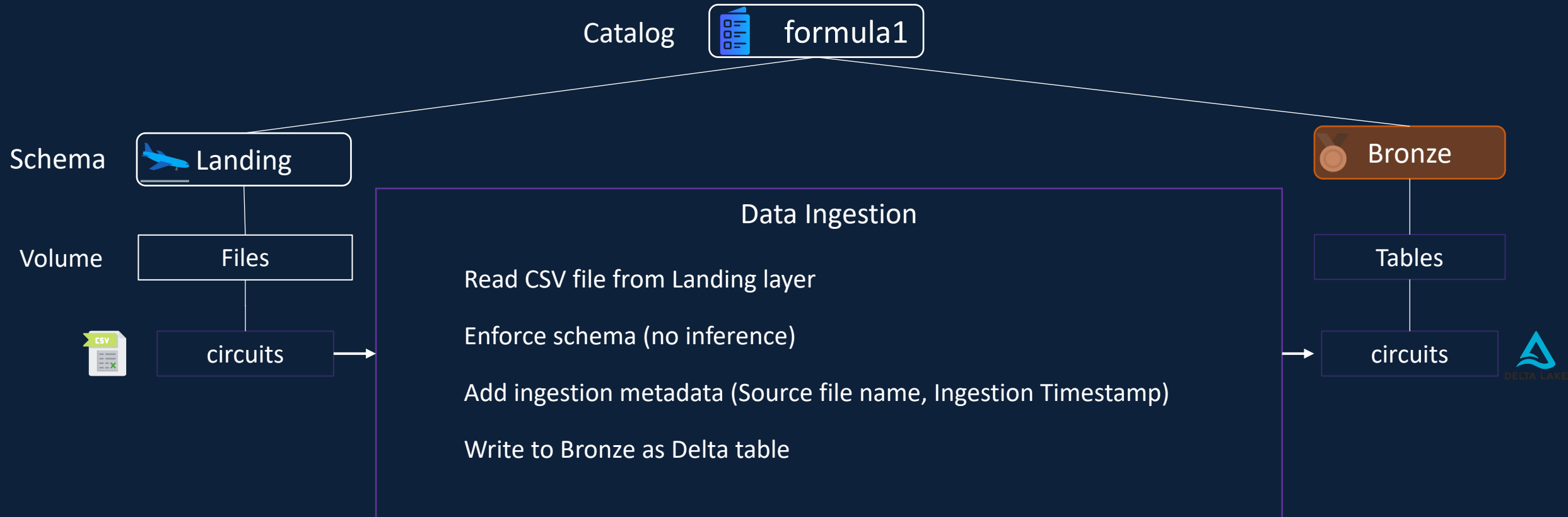
Ingesting Circuits Dataset



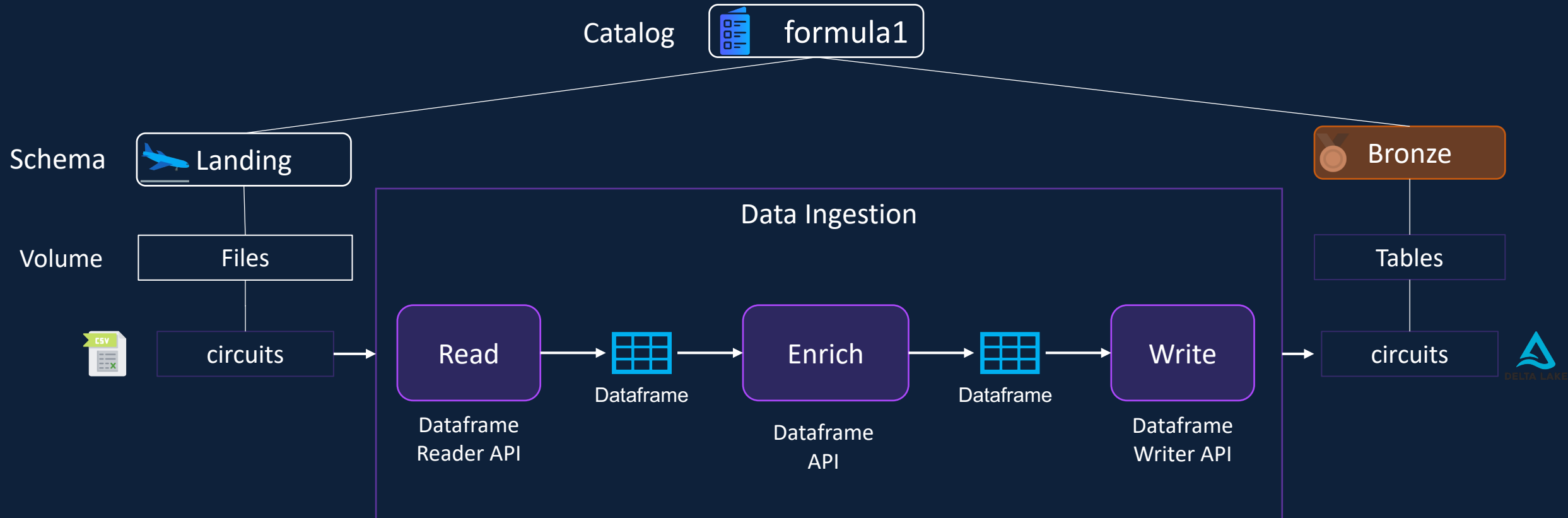
Ingesting Circuits Dataset



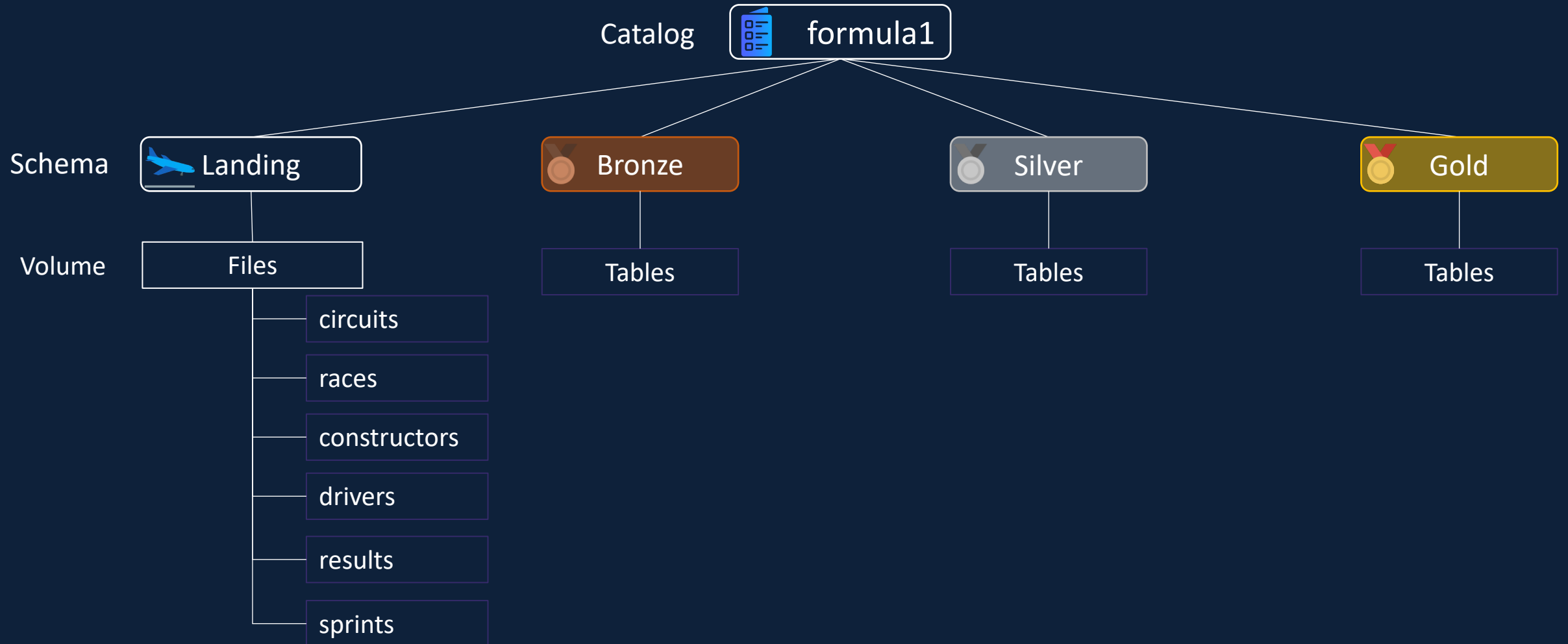
Ingesting Circuits Dataset



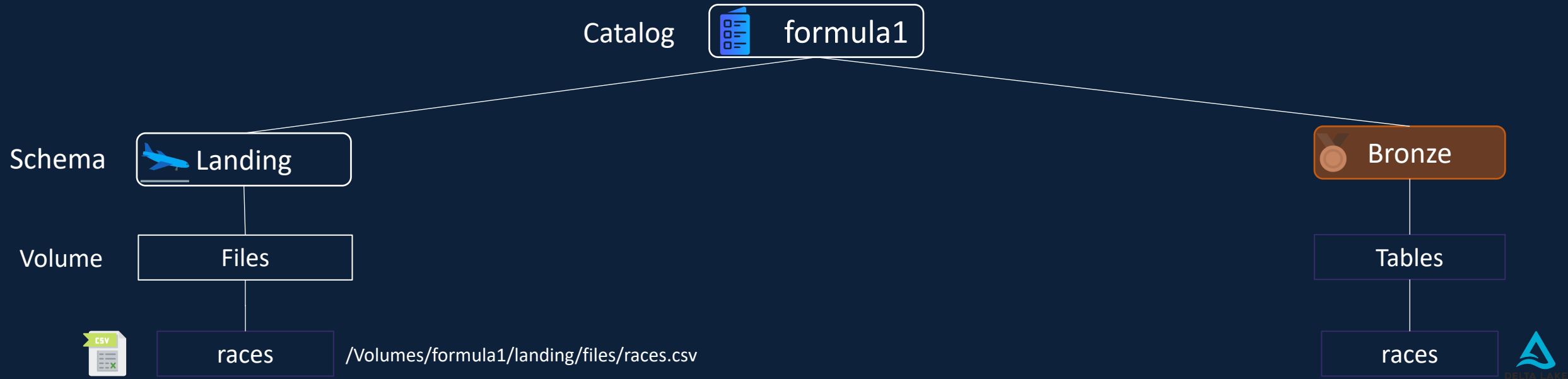
Ingesting Circuits Dataset



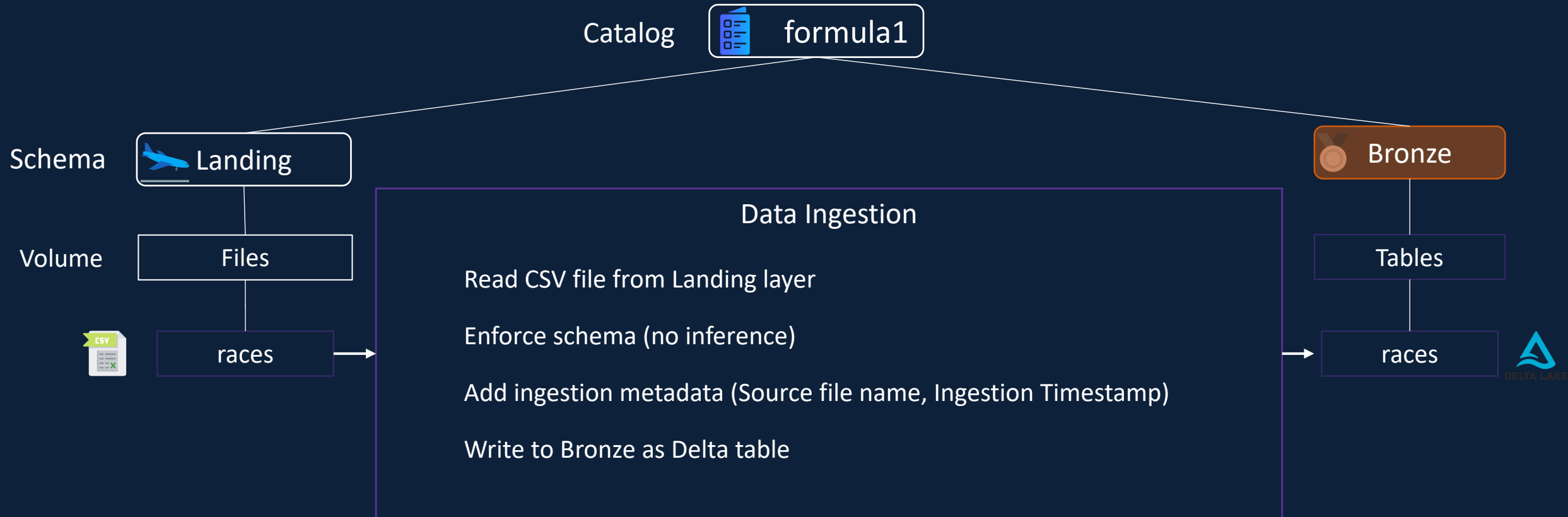
Ingesting Races Dataset (Assignment)



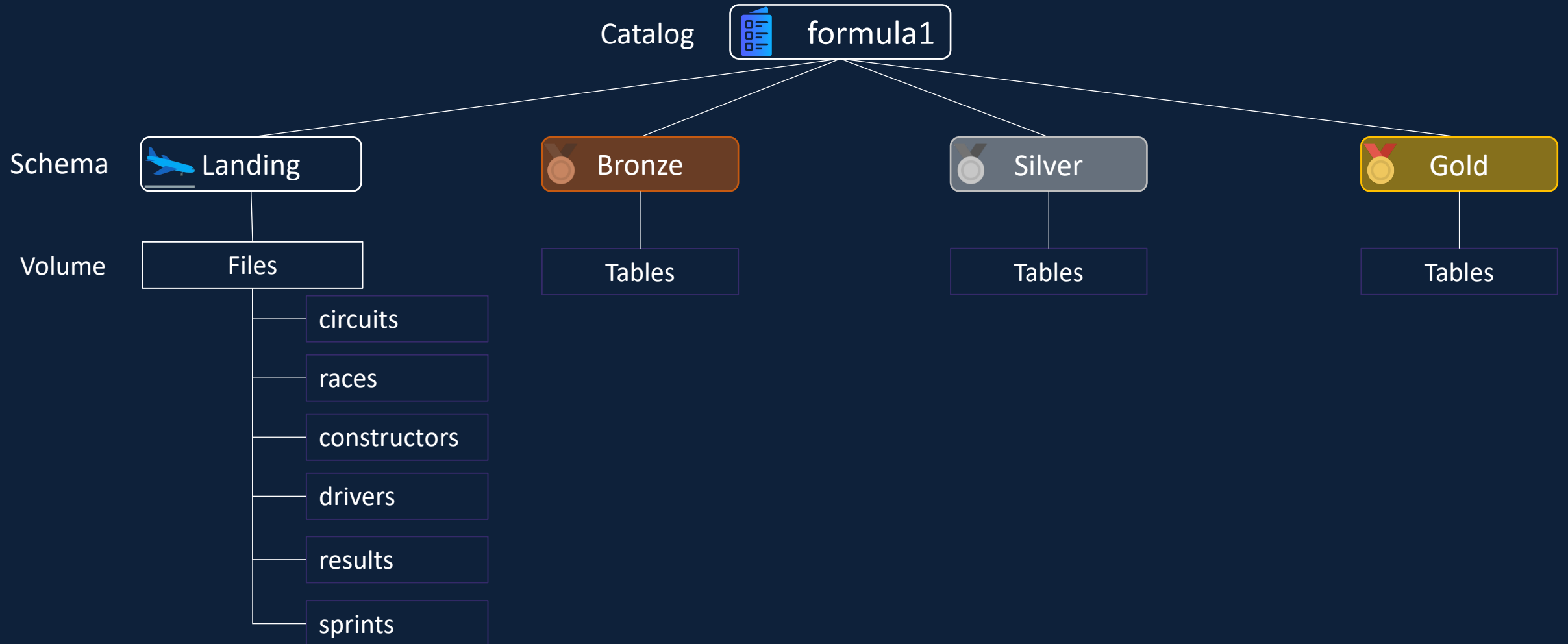
Ingesting Races Dataset (Assignment)



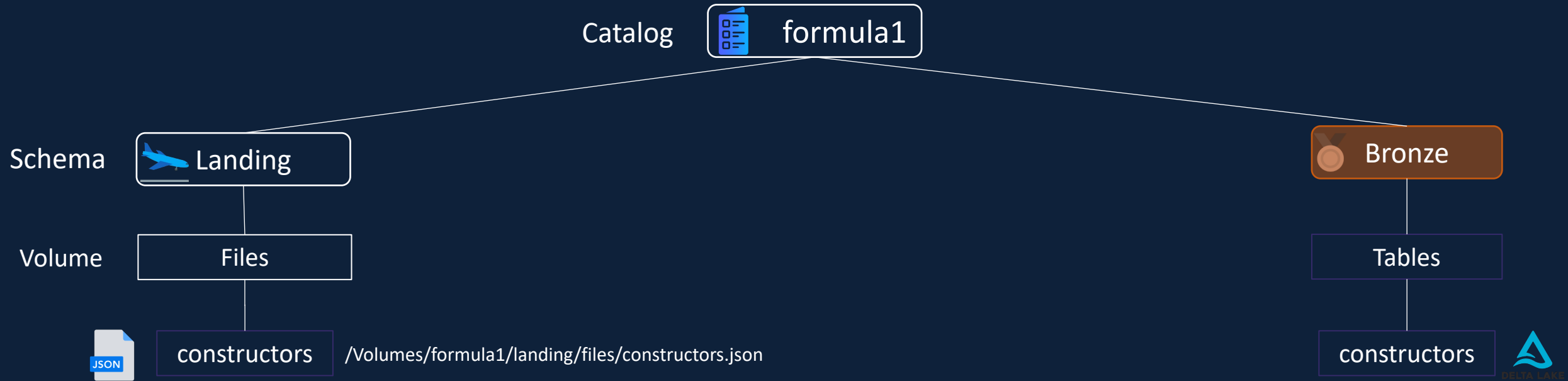
Ingesting Races Dataset (Assignment)



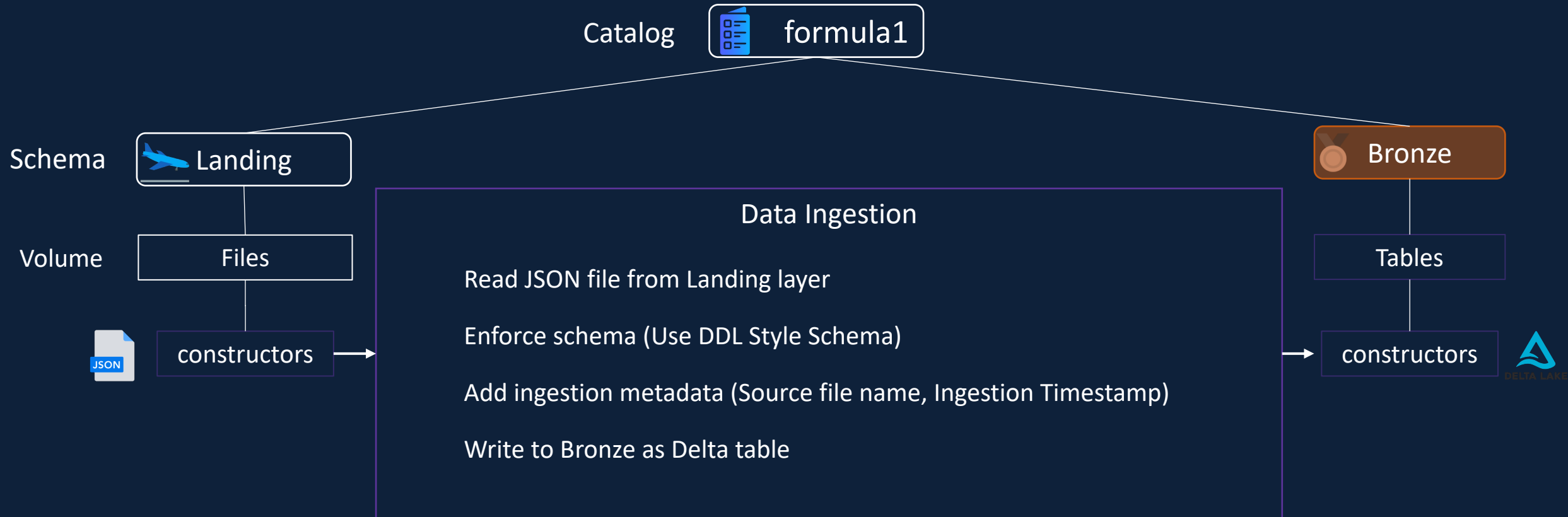
Ingesting Constructors Dataset



Ingesting Constructors Dataset



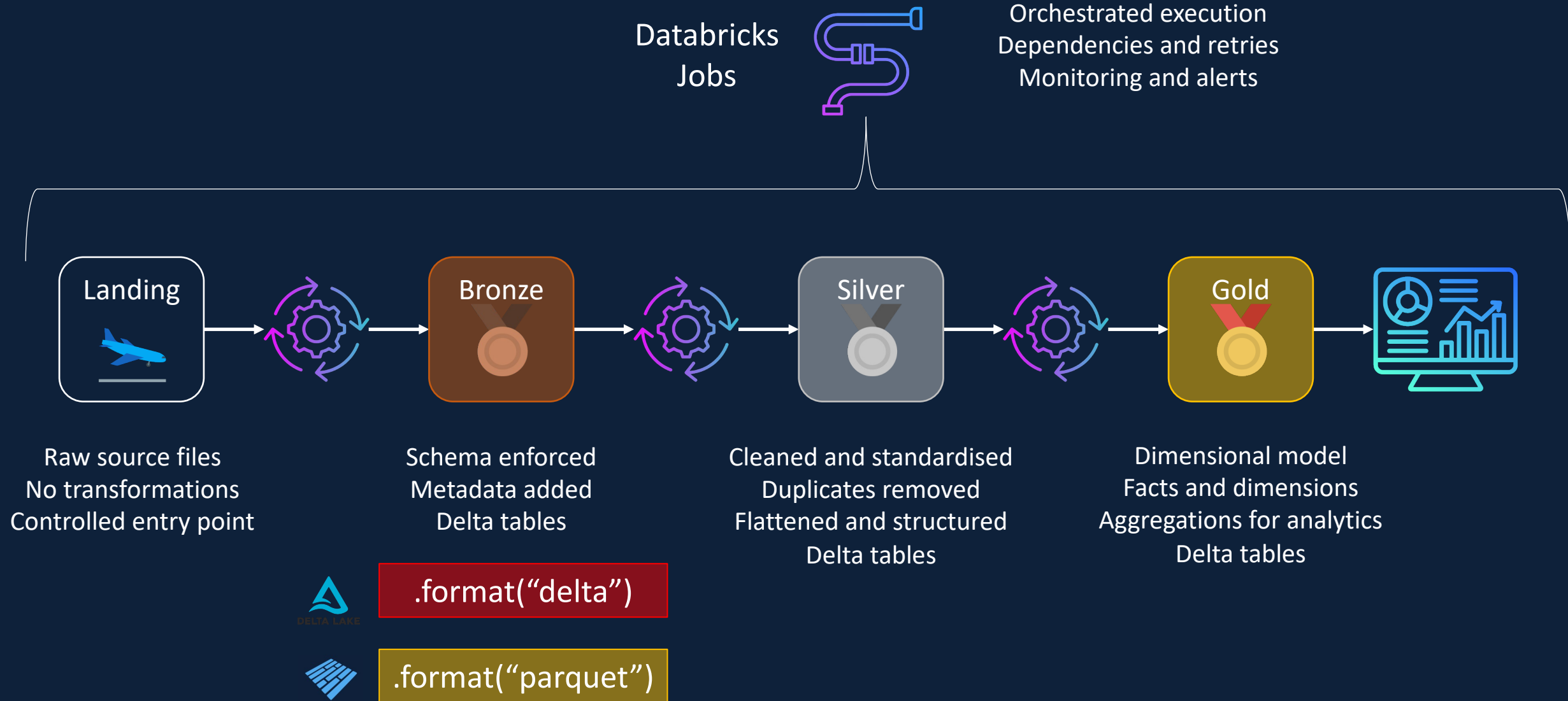
Ingesting Constructors Dataset





Introduction to Delta Lake

Introduction to Delta Lake



Introduction to Delta Lake



Columnar File Format

Efficient for storing and querying large datasets

Failed jobs leave partial files

Concurrent writes can overwrite each other

Inconsistent reads

No history or versioning

Doesn't support rolling back changes

Lack of support for updates and deletes

Delta Lake



Delta Lake



Transaction Log



Parquet Data Files

Open file format

History, Versioning, ACID, Time Travel

Delta Lake – Key Characteristics



ACID Transactions

Atomicity

Consistency

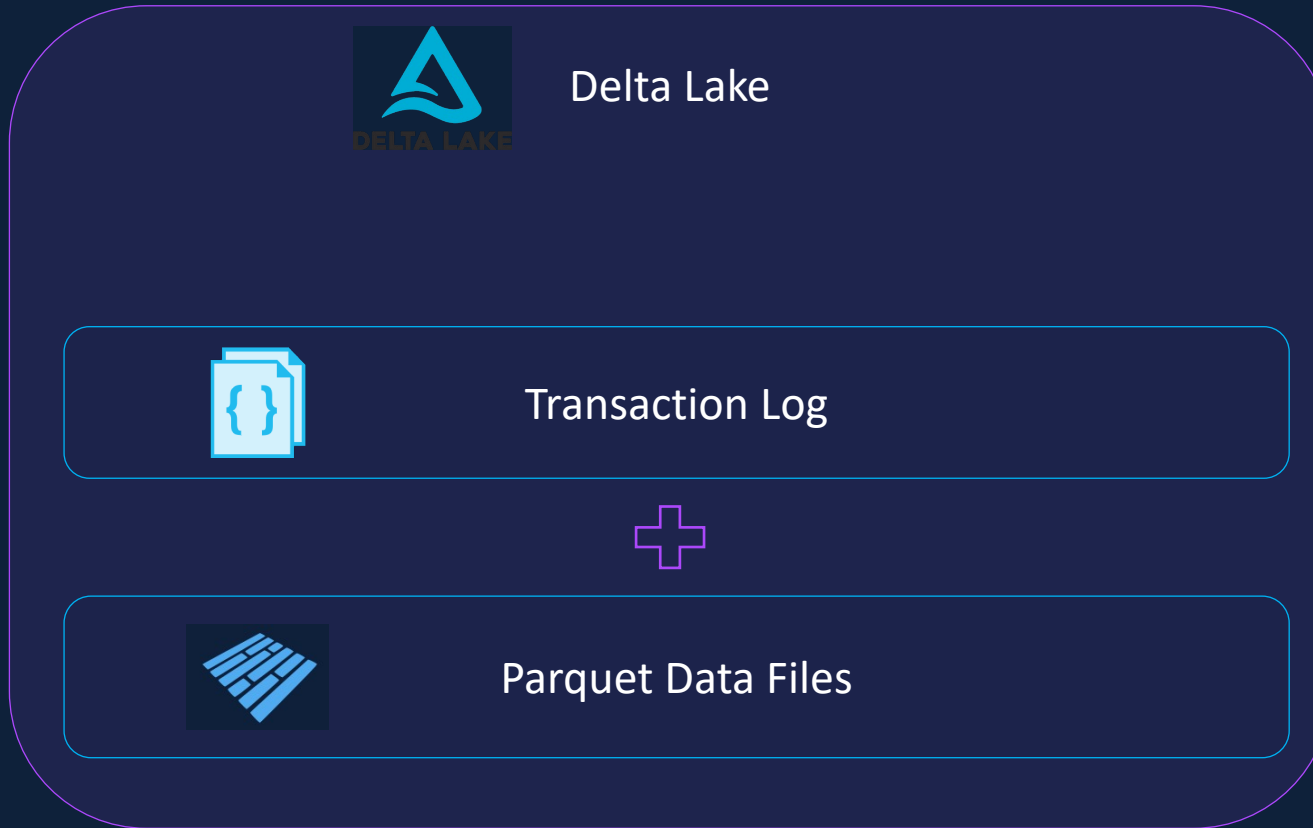
Isolation

Durability

Version History

Support for Data Modification

Delta Lake – Key Characteristics



Open file format

History, Versioning, ACID, Time Travel

Delta Lake



Delta Lake



Transaction Log

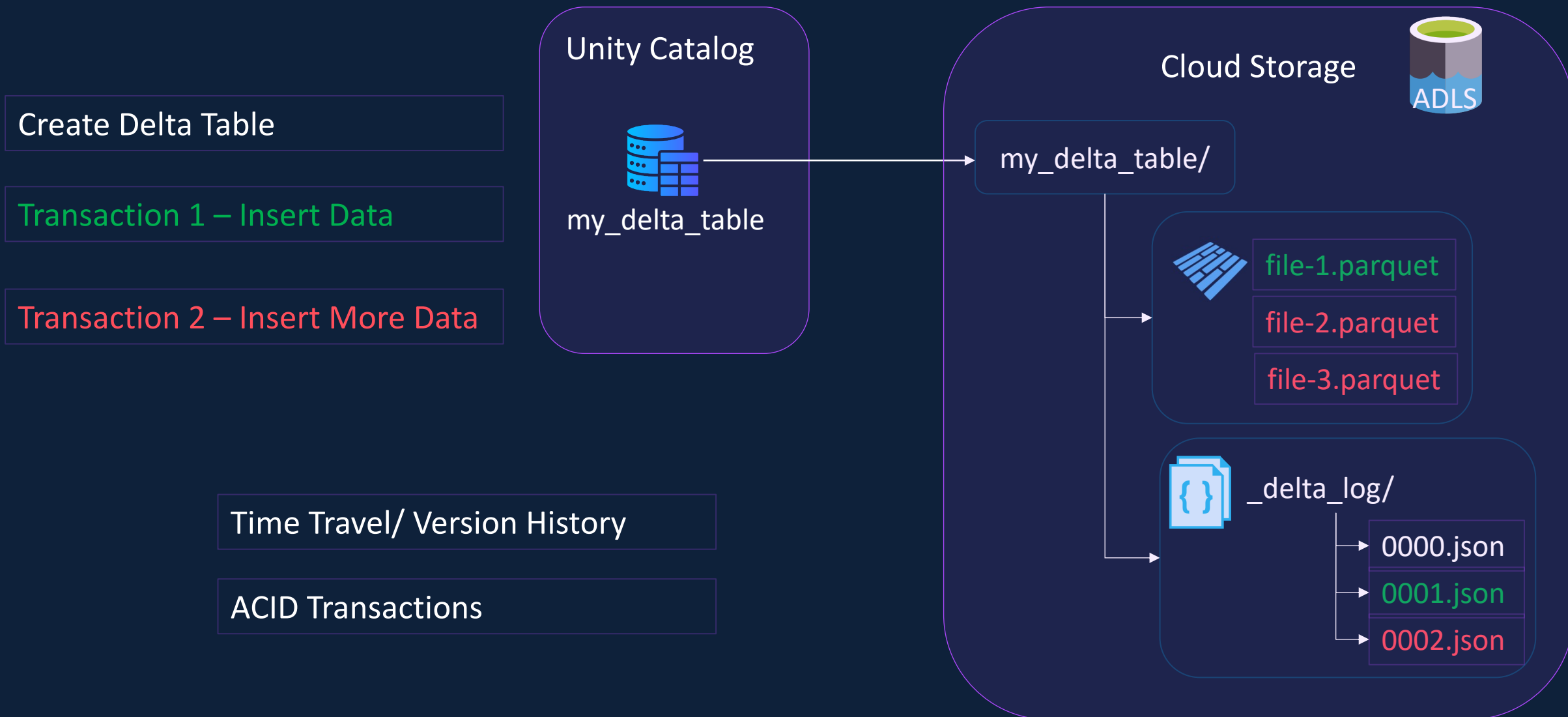


Parquet Data Files

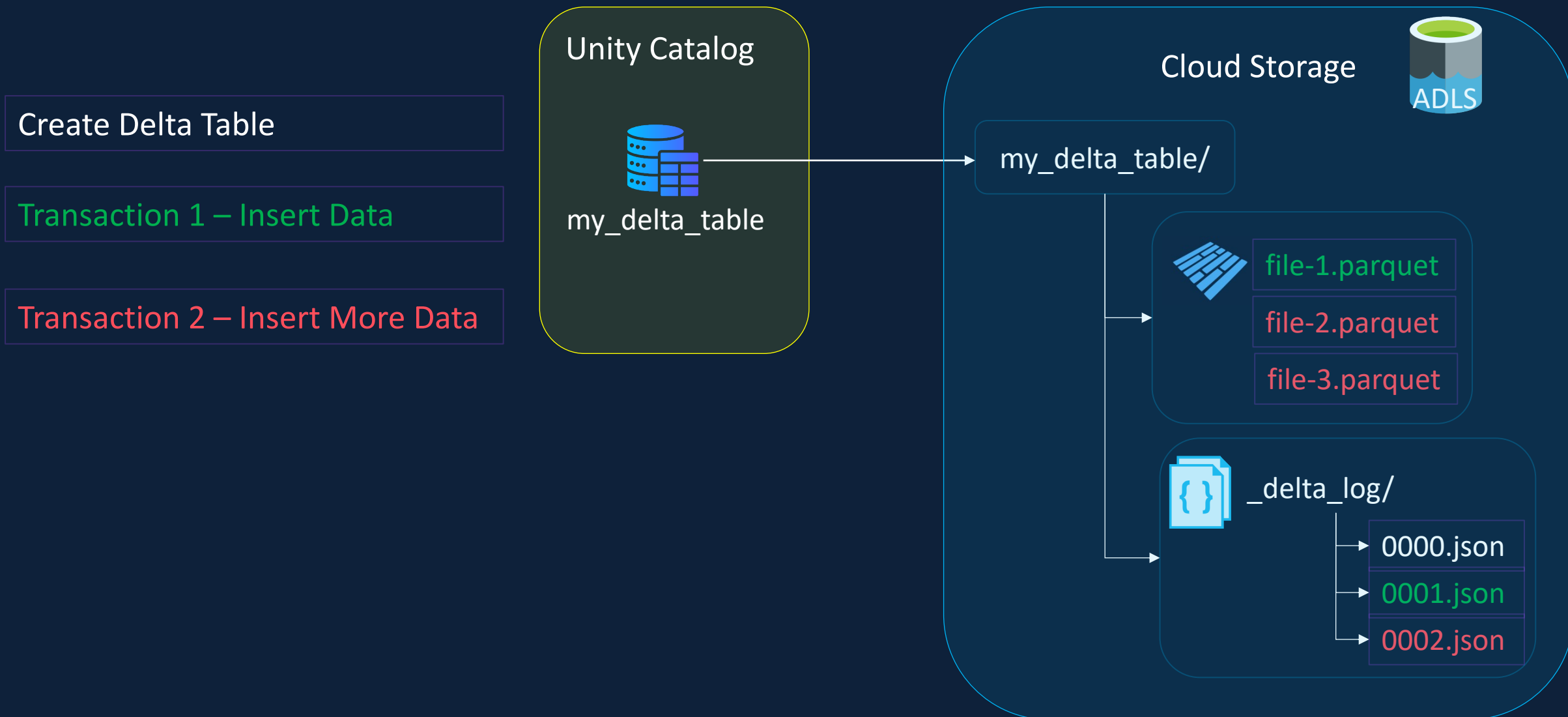
Open file format

History, Versioning, ACID, Time Travel

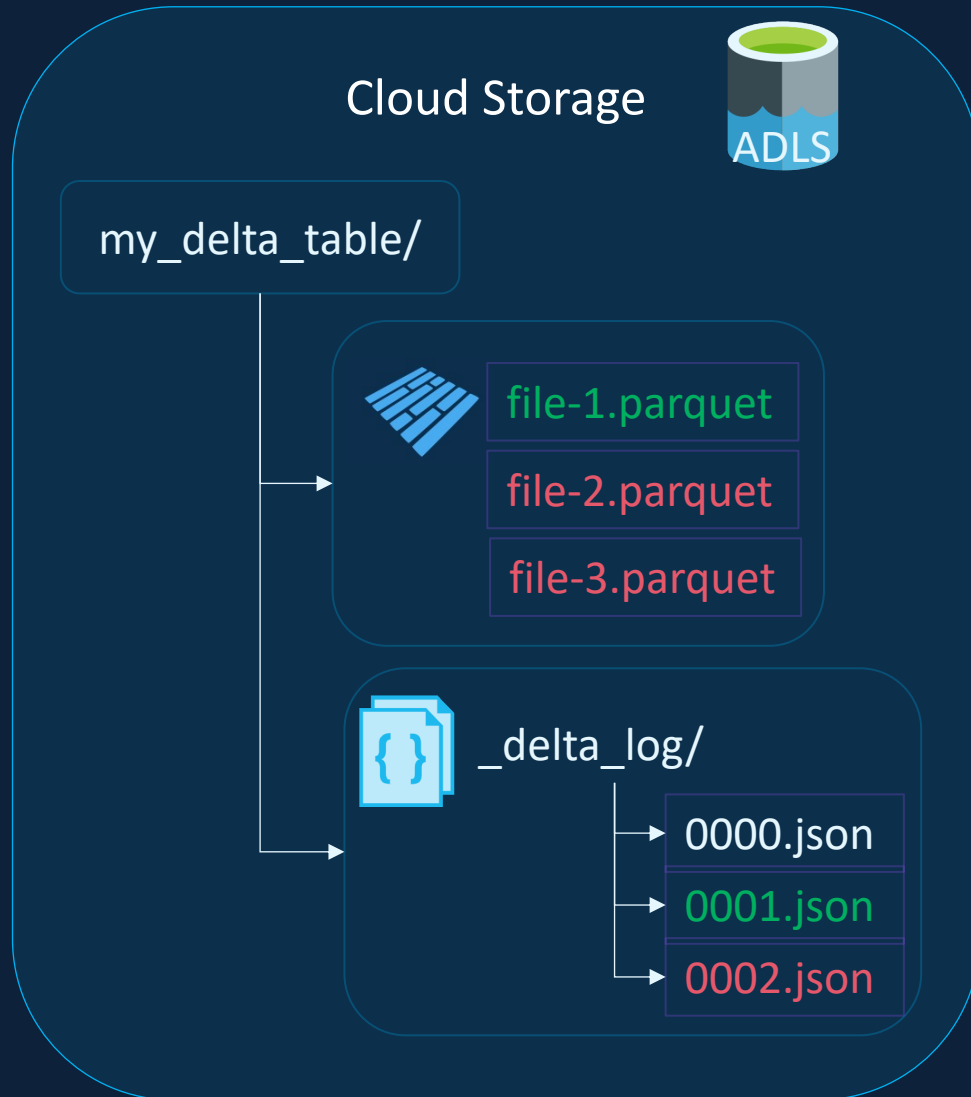
Delta Transaction Log



Delta Lake – ACID Transactions



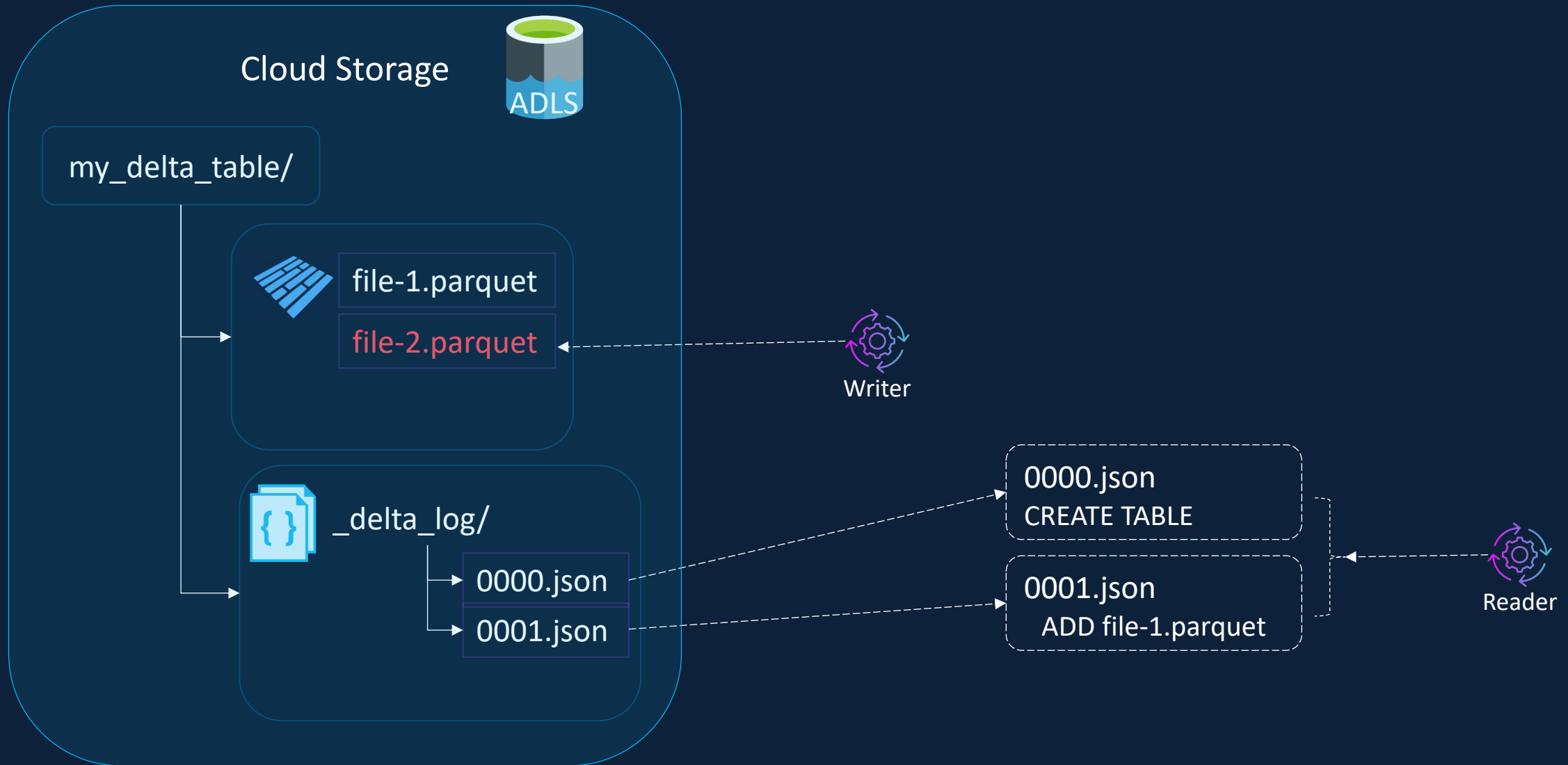
Delta Lake – ACID Transactions



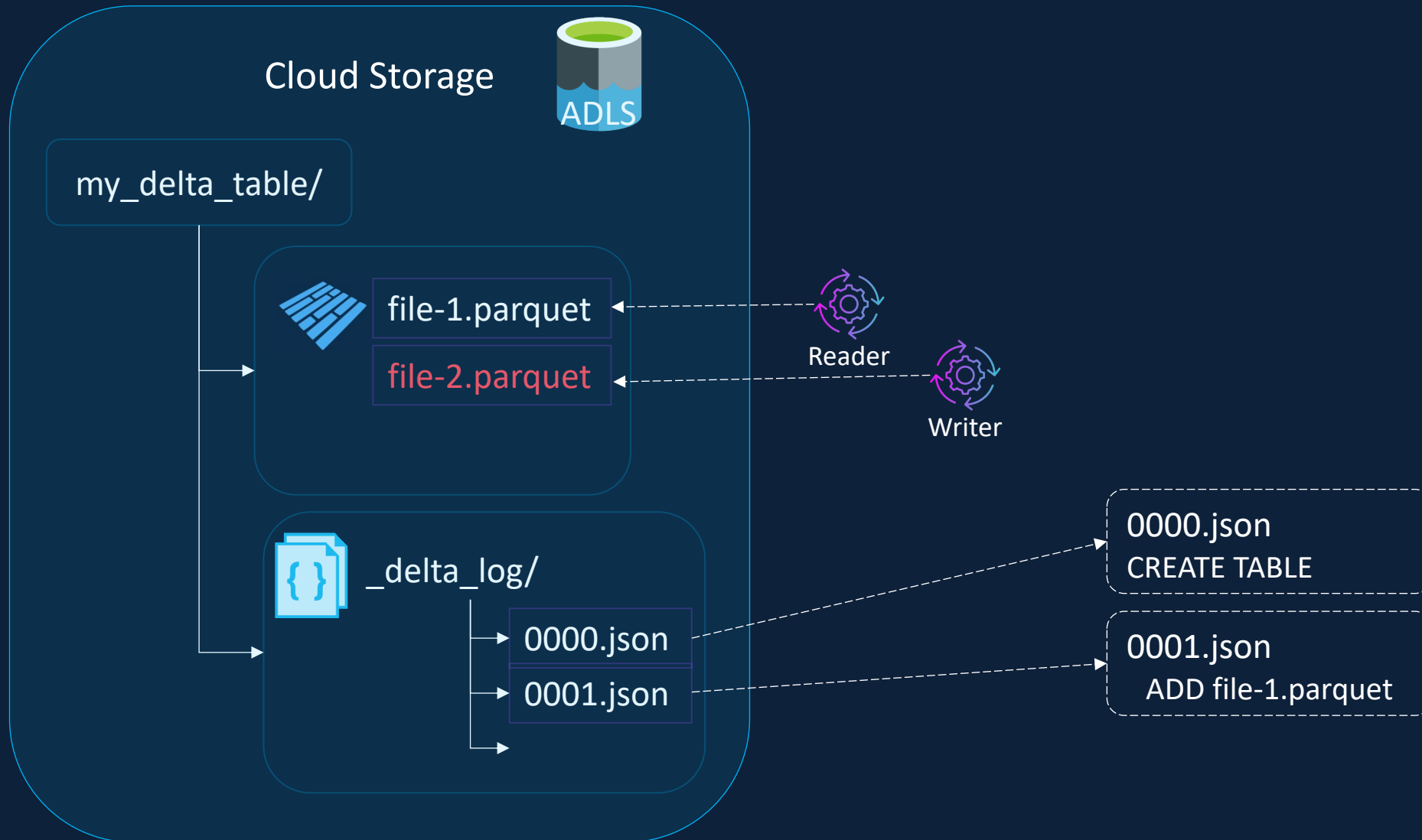
1. Transaction Logs are written at the end of the transaction

2. Readers will always read the transaction logs first to identify the list of data files to read

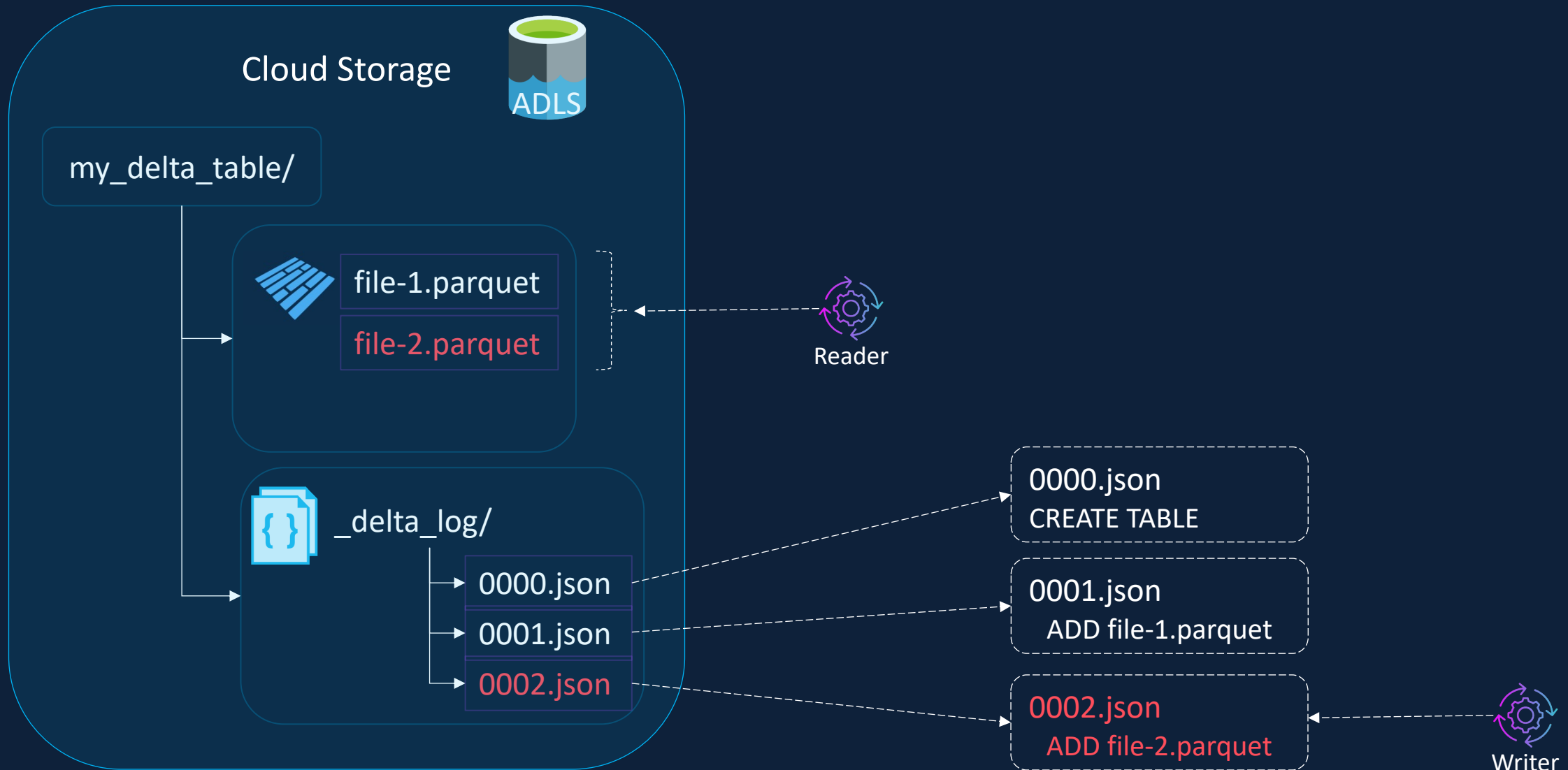
Delta Lake – ACID Transactions



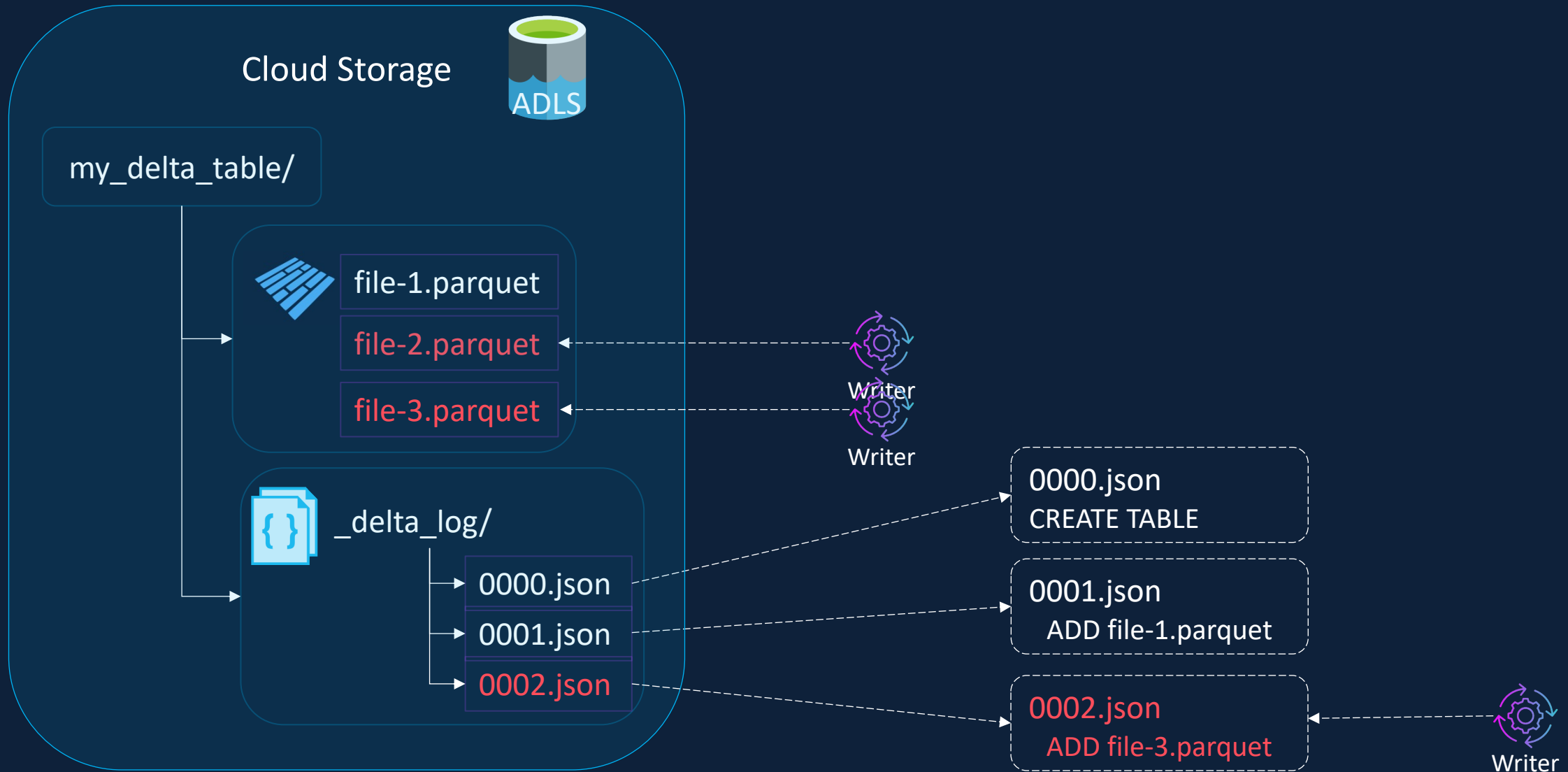
Delta Lake – ACID Transactions



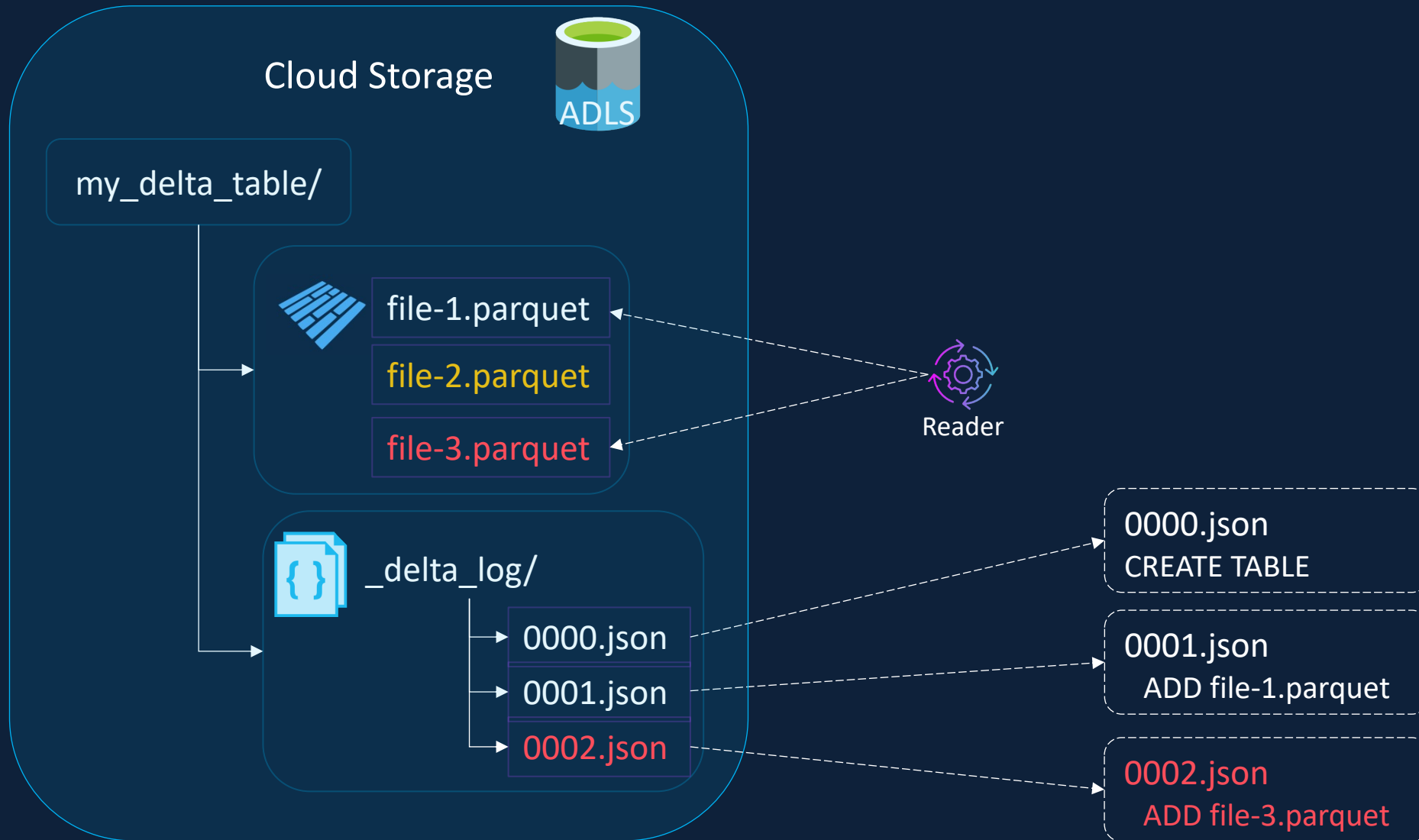
Delta Lake – ACID Transactions



Delta Lake – ACID Transactions



Delta Lake – ACID Transactions



Data Transformation Overview

Data Transformation Overview



Clean and standardise data

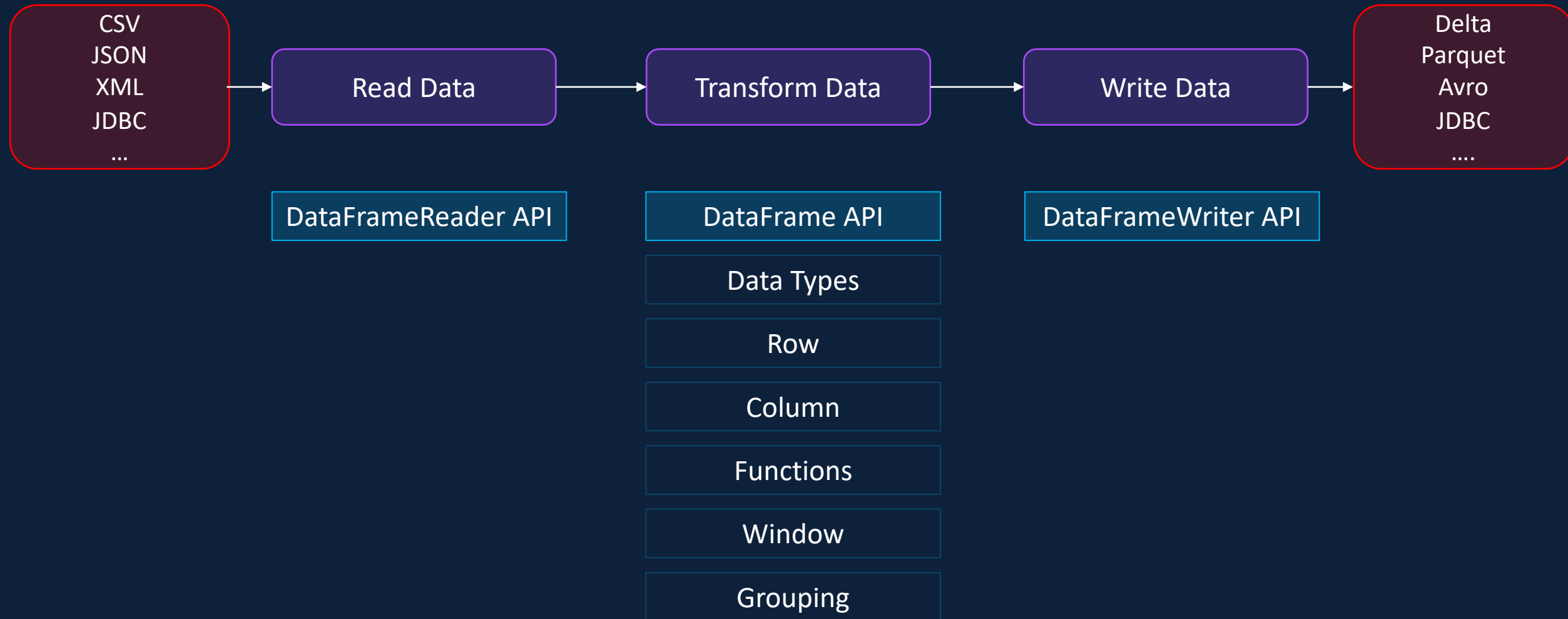
Apply naming conventions and reshape data

Perform data quality checks

Preserve business keys

Prepare data for Gold layer

Data Transformation Overview



Data Transformation Overview

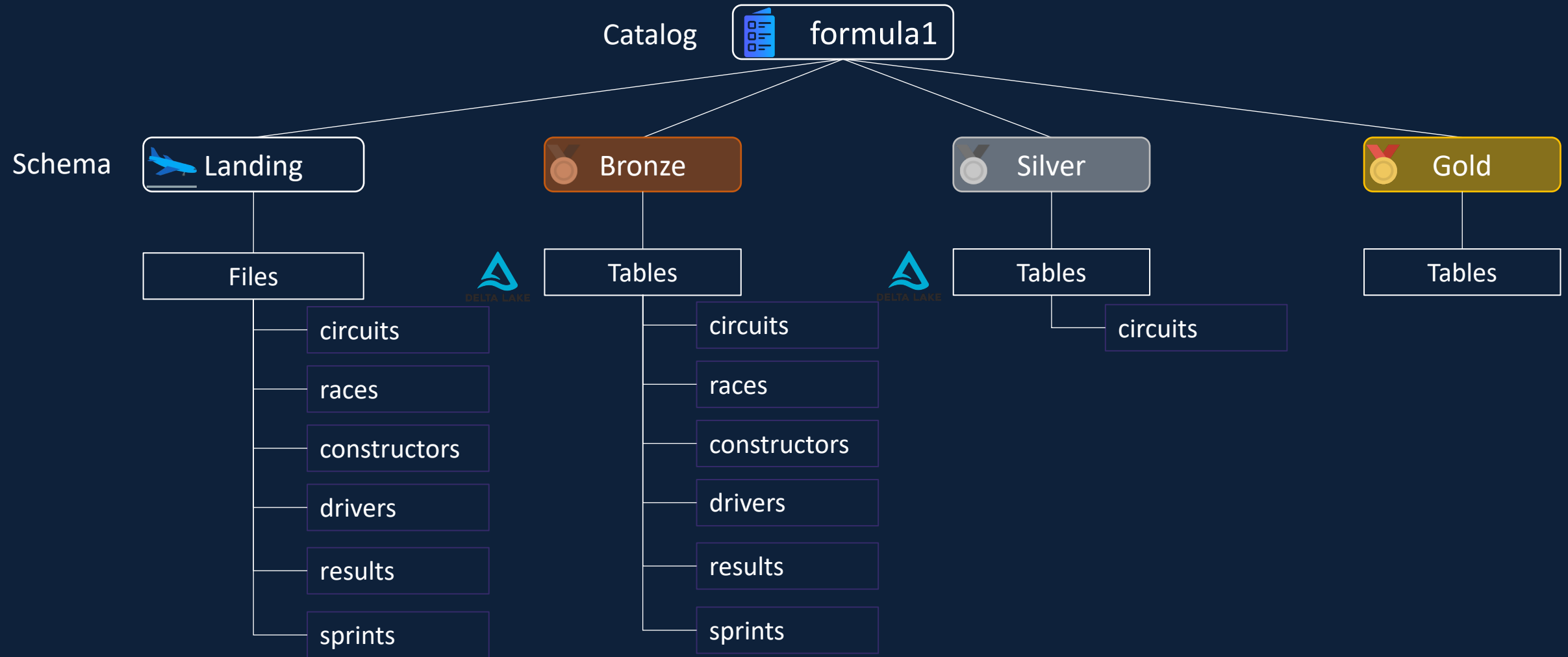


Transform circuits dataset step by step

Apply transformation pattern to all datasets

Store data in Silver layer

Transforming Circuits Dataset



Transforming Circuits Dataset

Catalog

 formula1

 Bronze

Tables

circuits

racers

constructors

drivers

results

sprints

Data Transformation

Keep only the columns required for analytics (Drop URL)

Standardize column names using snake case
(circuitId → circuit_id, circuitName → circuit_name)

Rename columns to make them more meaningful
(lat → latitude, long → longitude)

Remove duplicate records

Remove records with NULL circuit_id

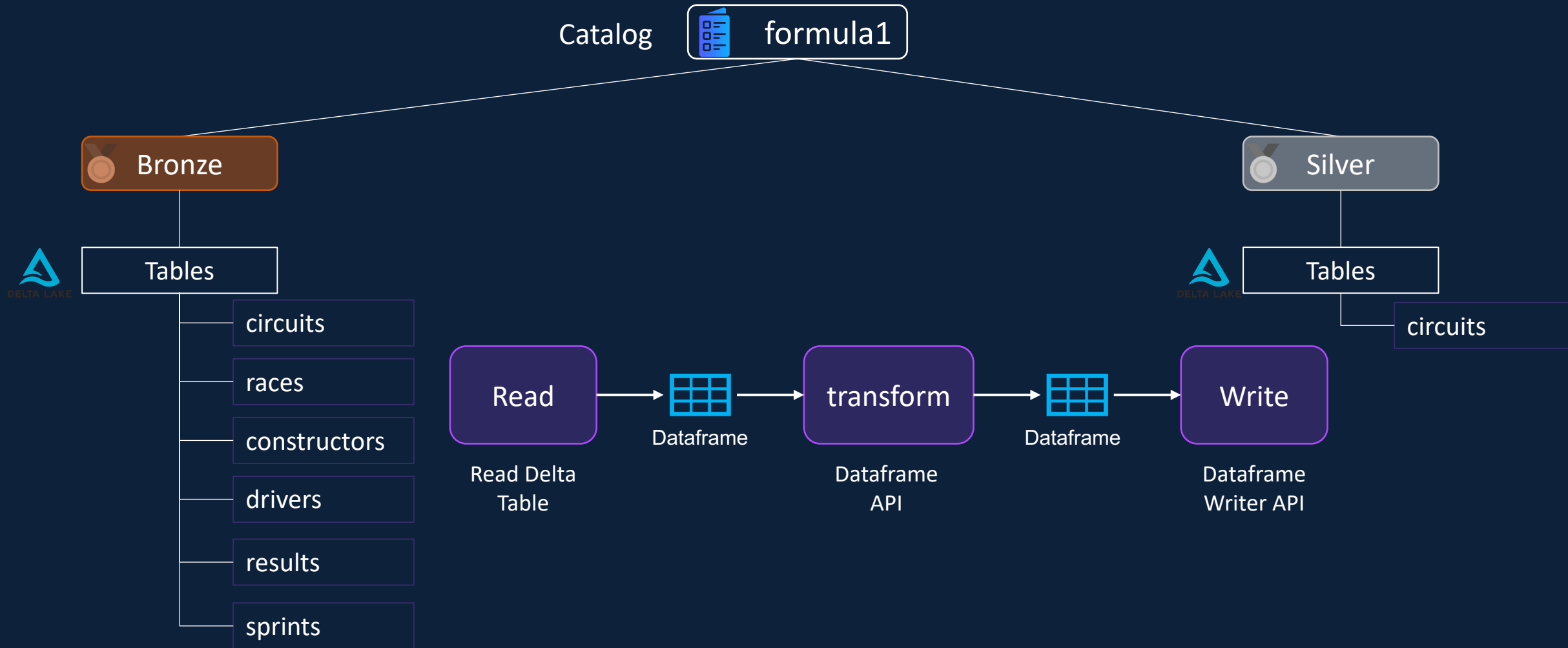
Transform column values of circuit_name and locality to Title Case

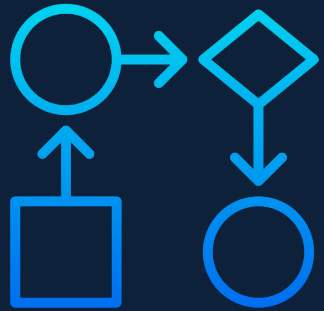
 Silver

Tables

circuits

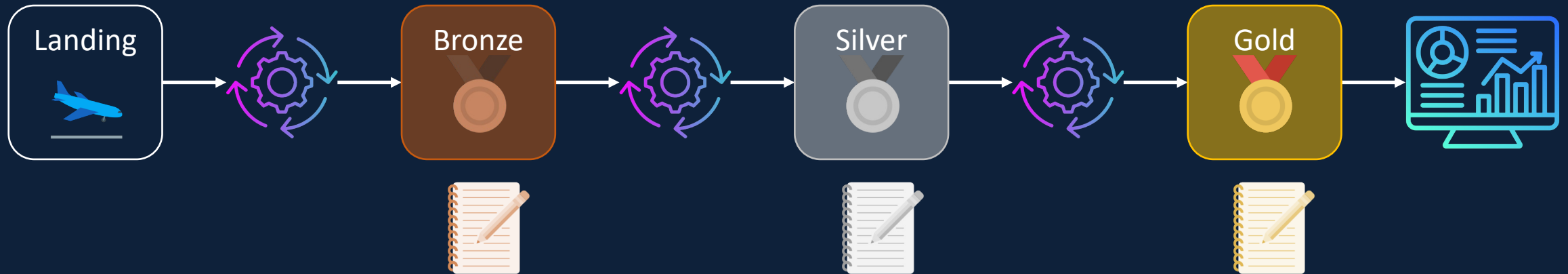
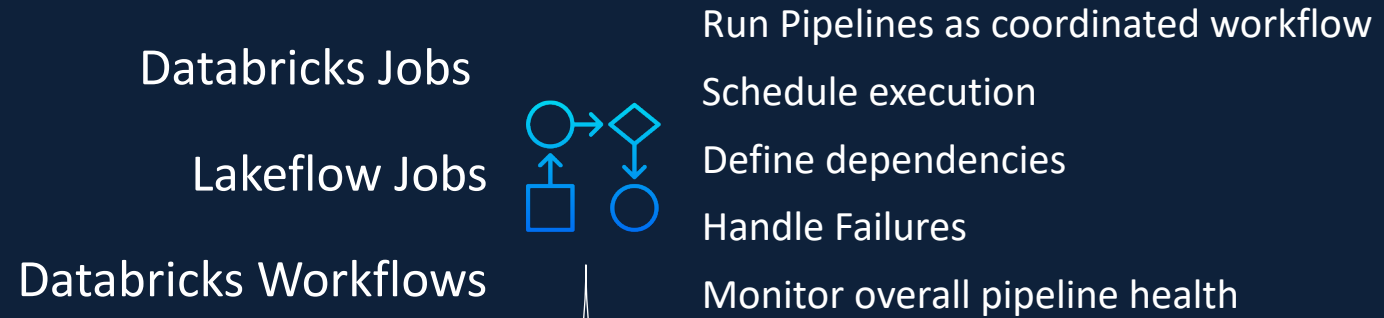
Transforming Circuits Dataset



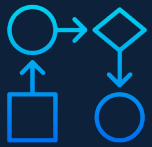


Introduction to Lakeflow Jobs

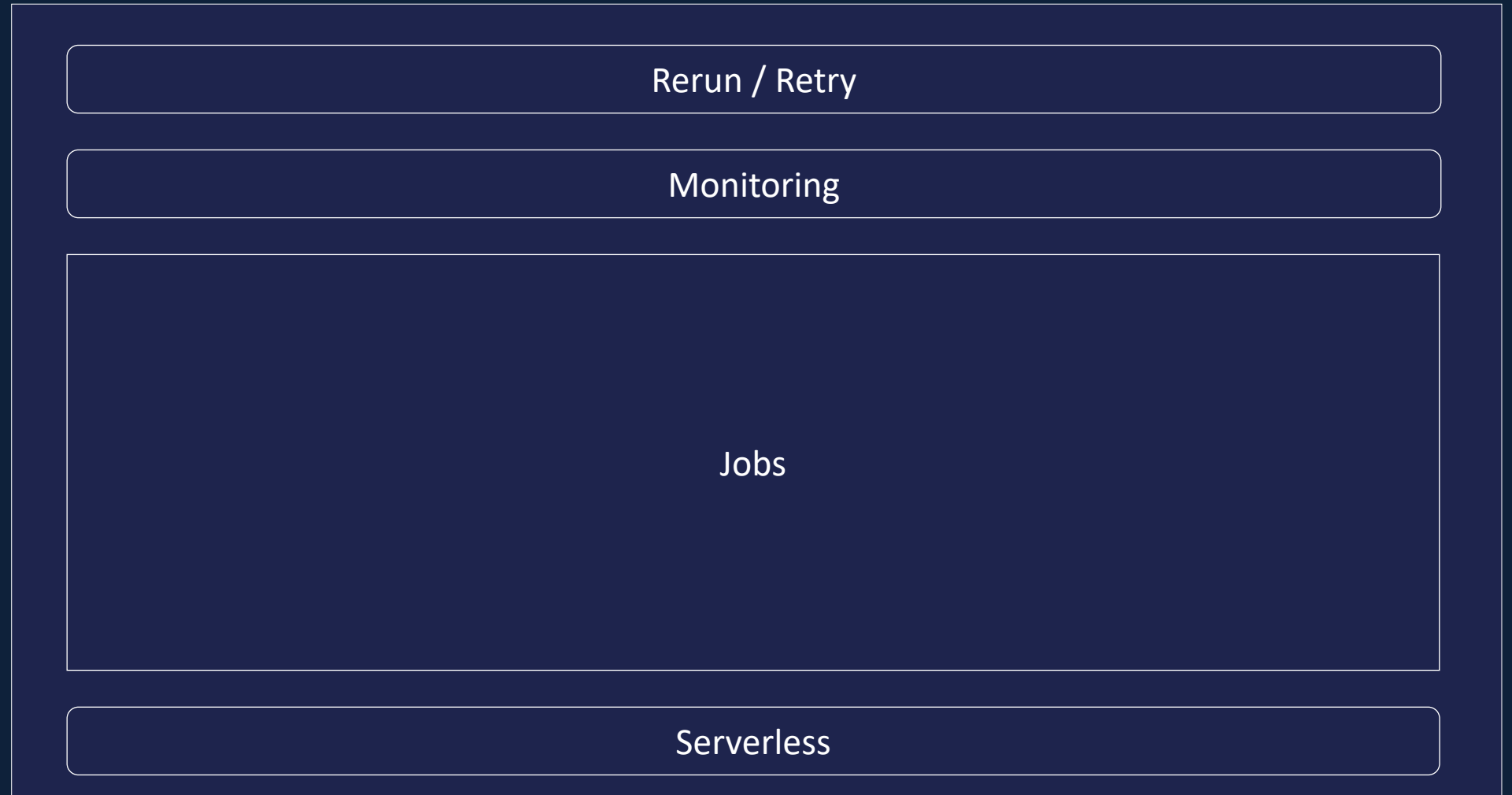
Introduction to Lakeflow Jobs



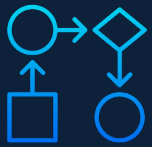
Introduction to Lakeflow Jobs



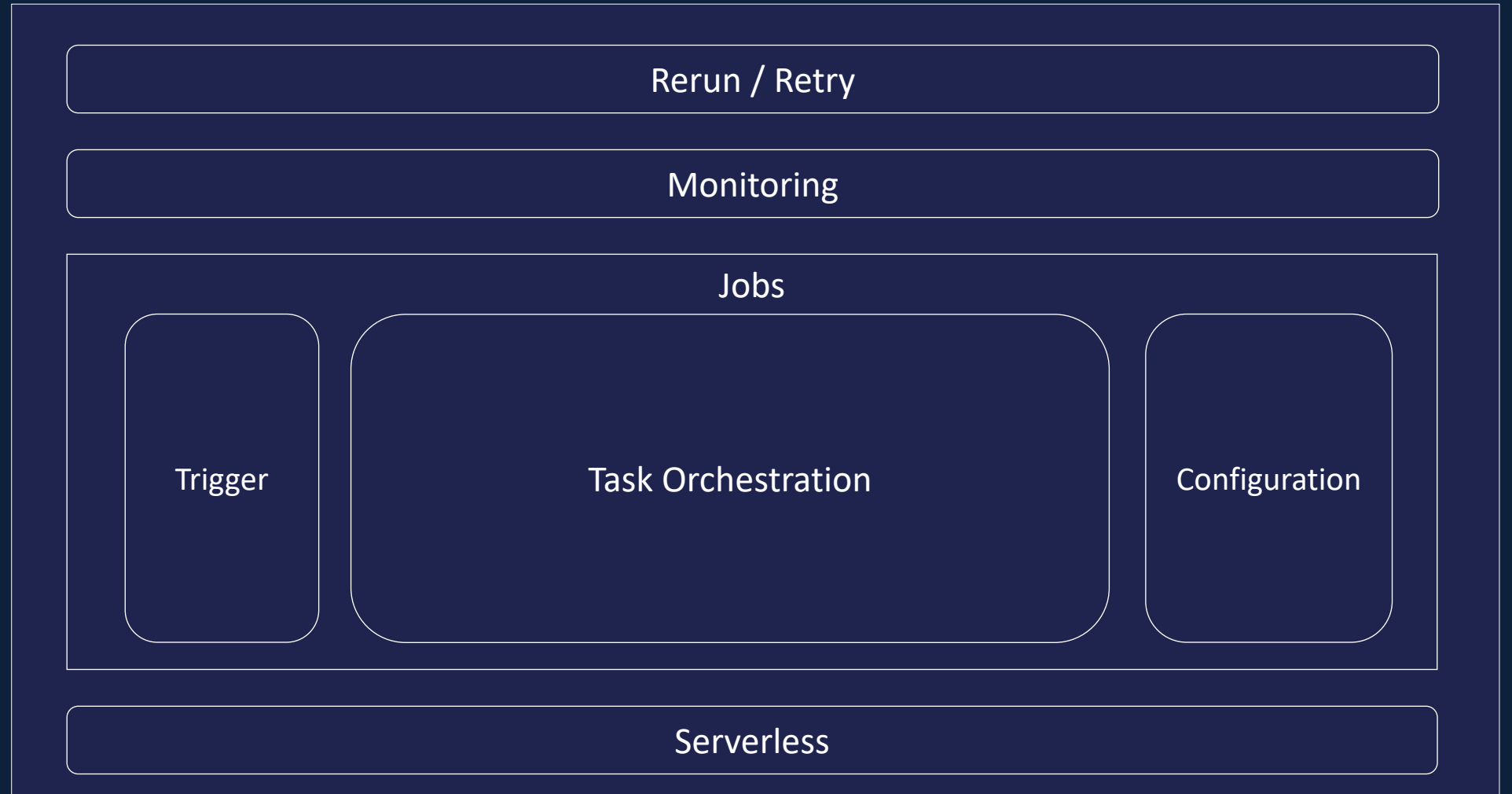
Fully Managed Service



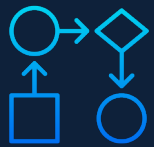
Introduction to Lakeflow Jobs



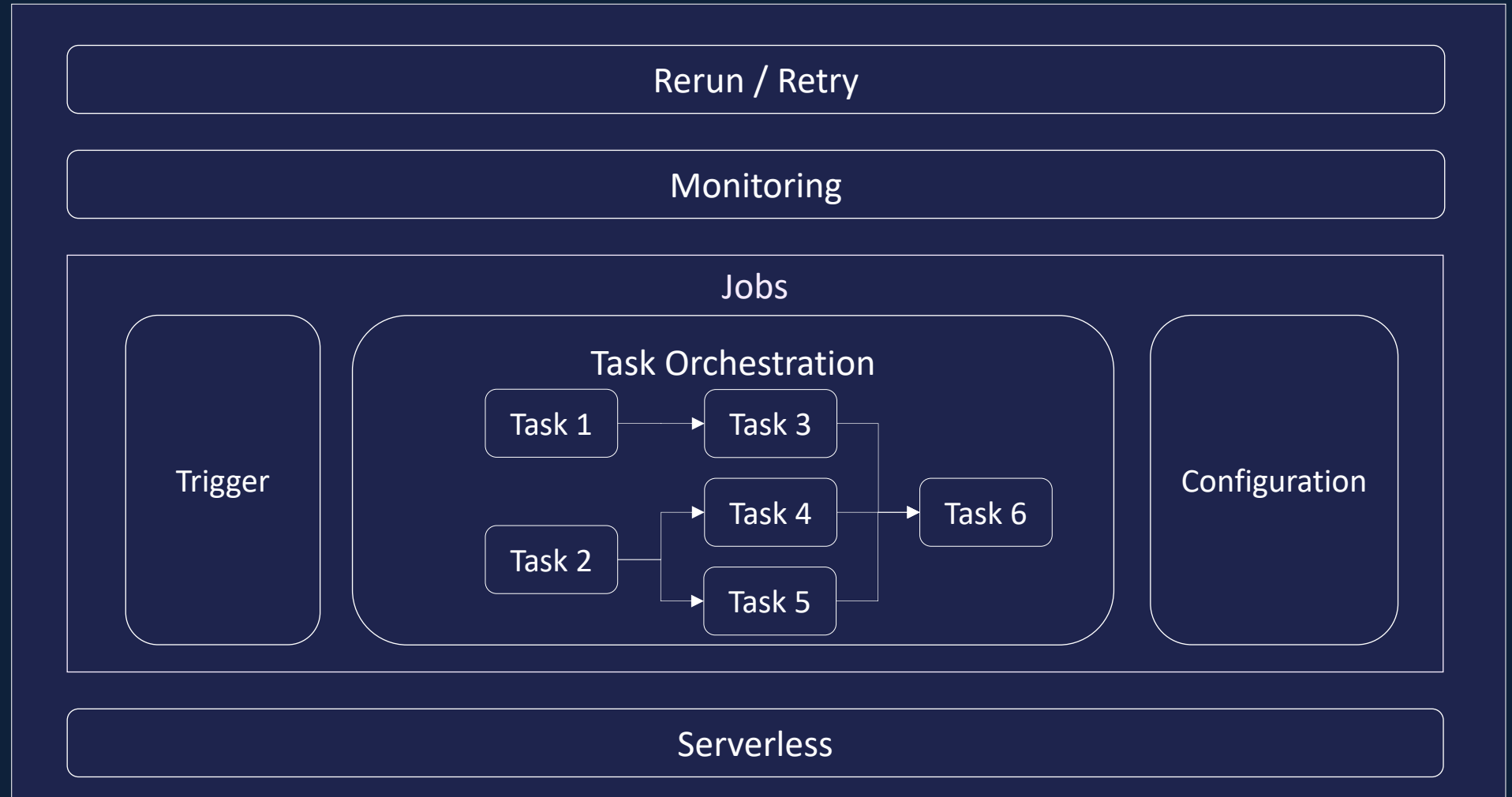
Fully Managed Service



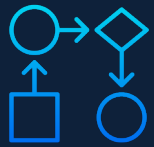
Introduction to Lakeflow Jobs



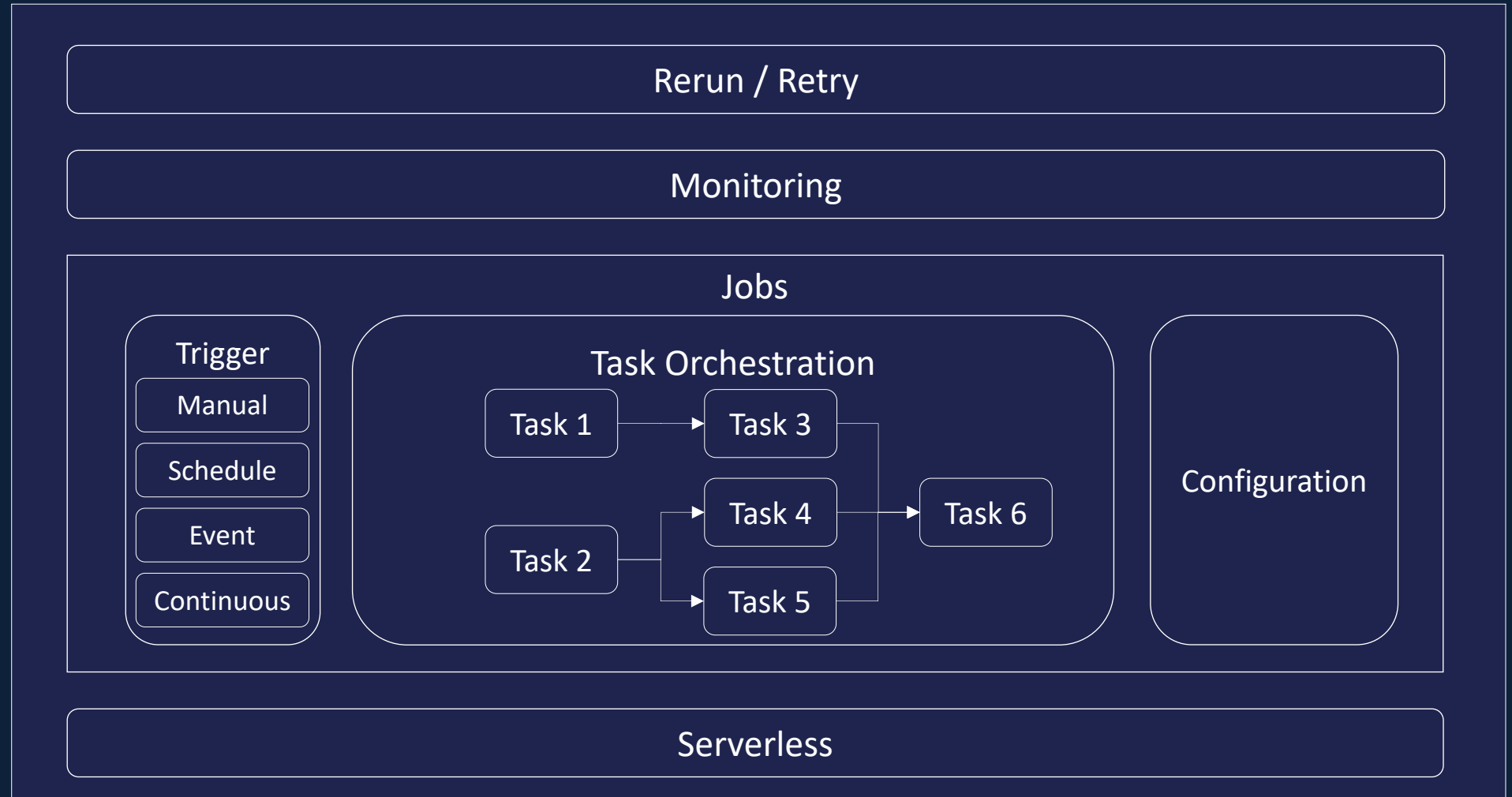
Fully Managed Service



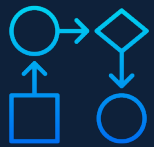
Introduction to Lakeflow Jobs



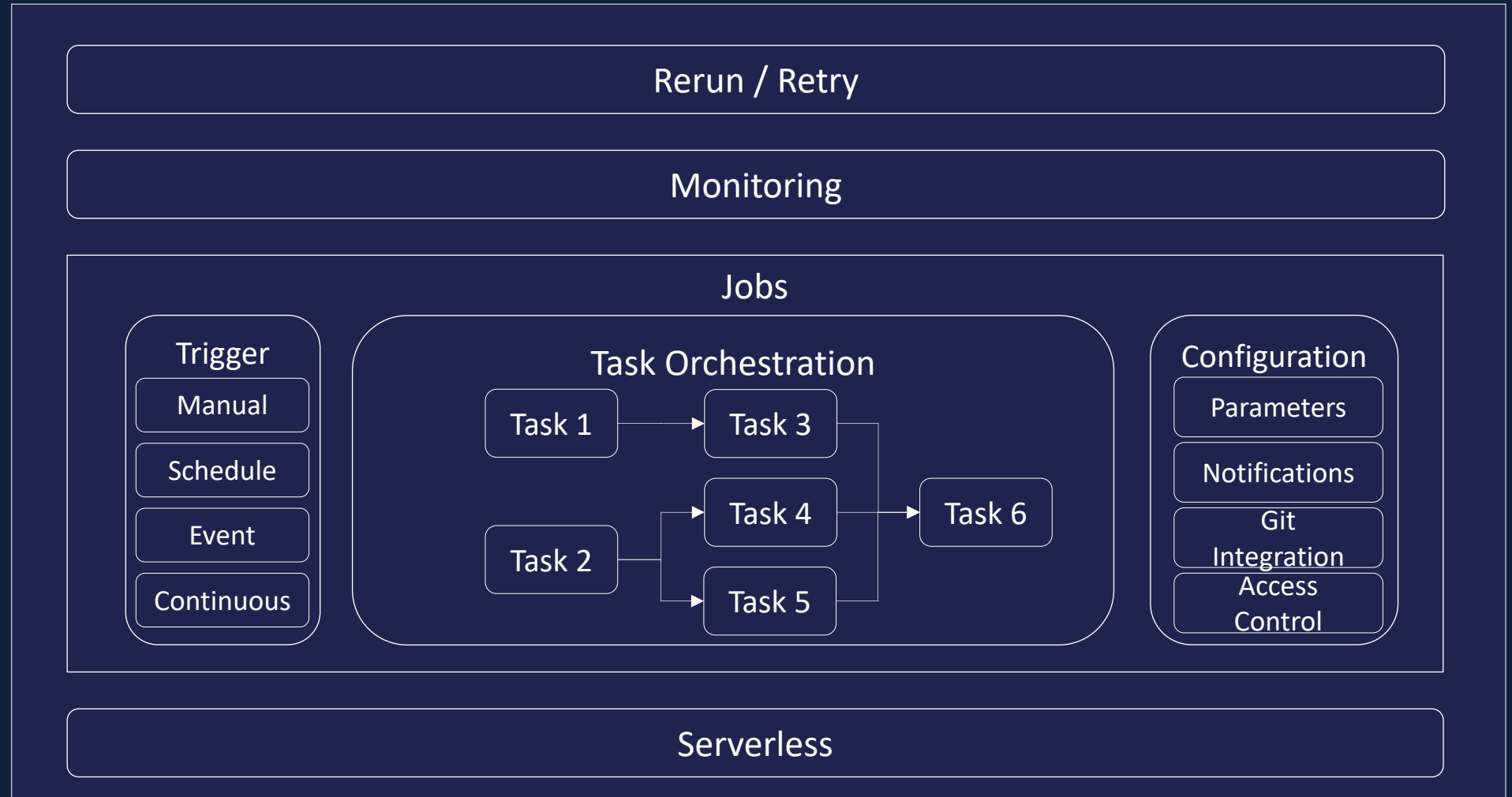
Fully Managed Service



Introduction to Lakeflow Jobs

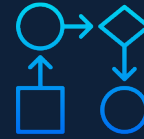


Fully Managed Service



Introduction to Lakeflow Jobs

Lakeflow Jobs



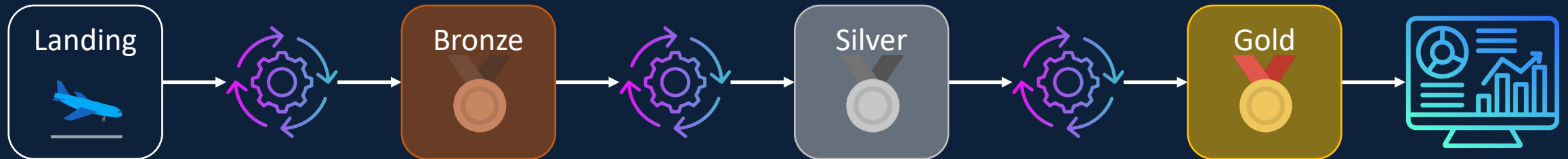
Run Pipelines as coordinated workflow

Schedule execution

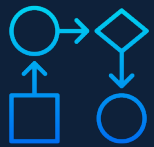
Define dependencies

Handle Failures

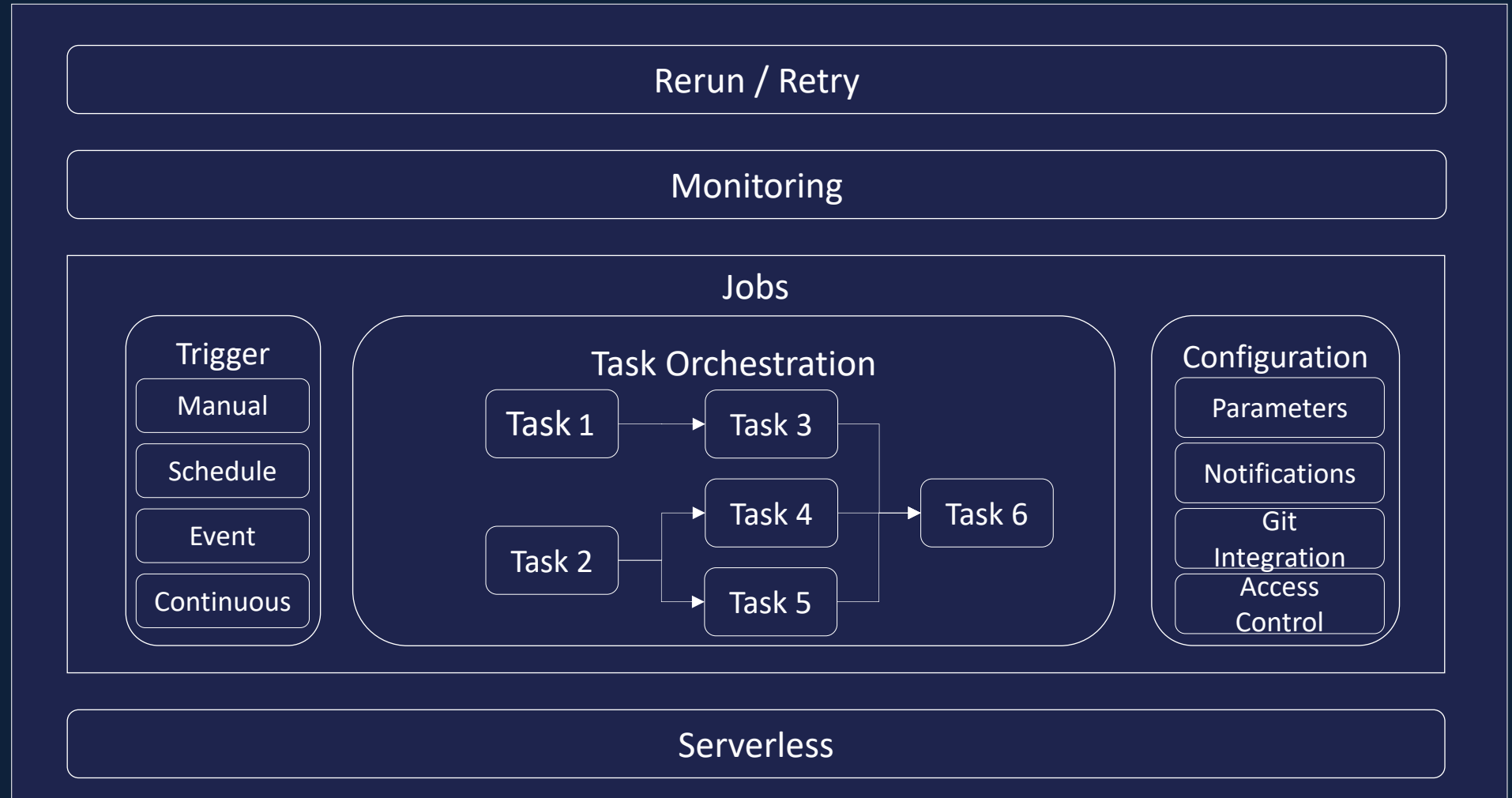
Monitor overall pipeline health



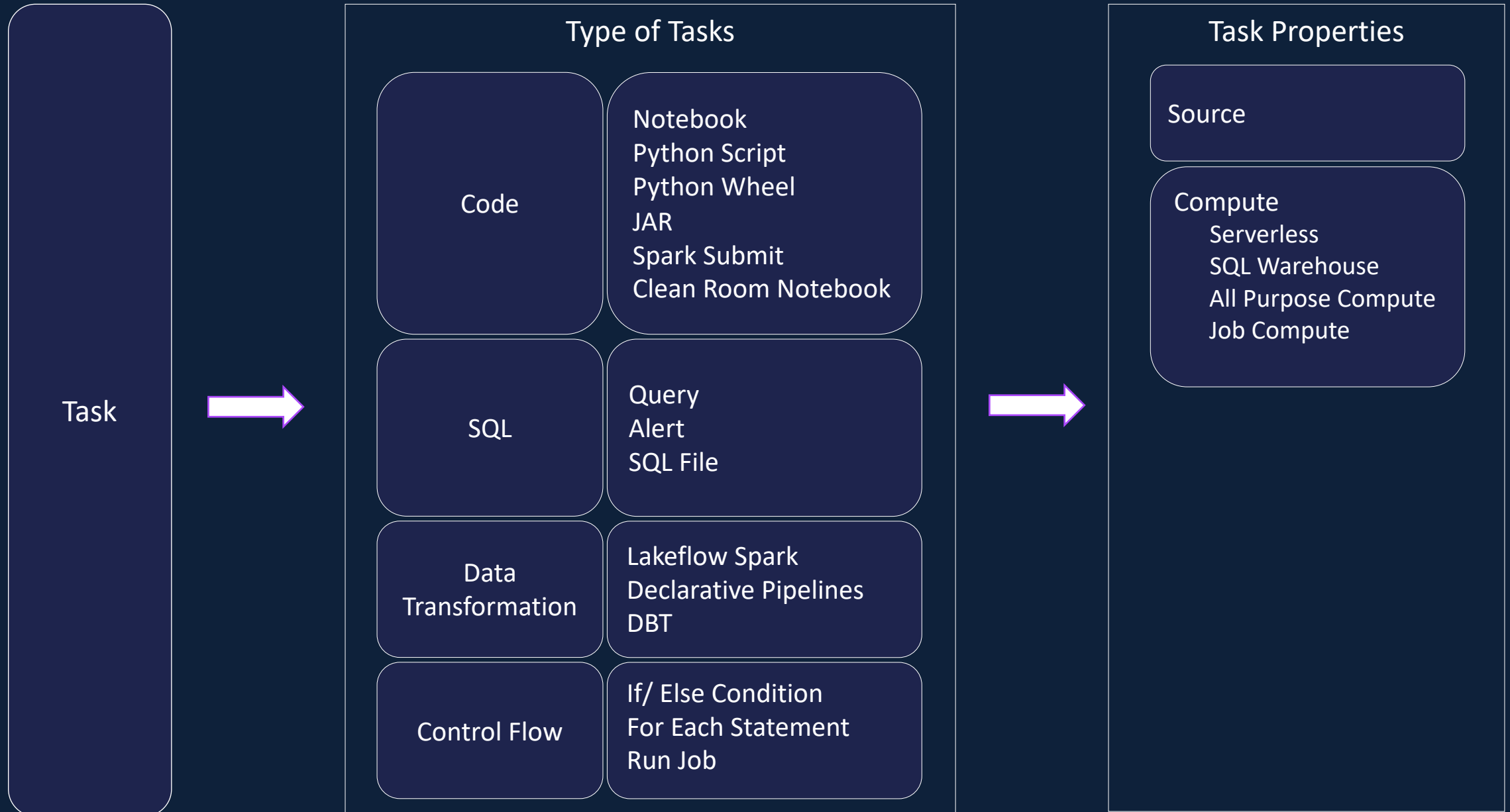
Introduction to Lakeflow Job Tasks



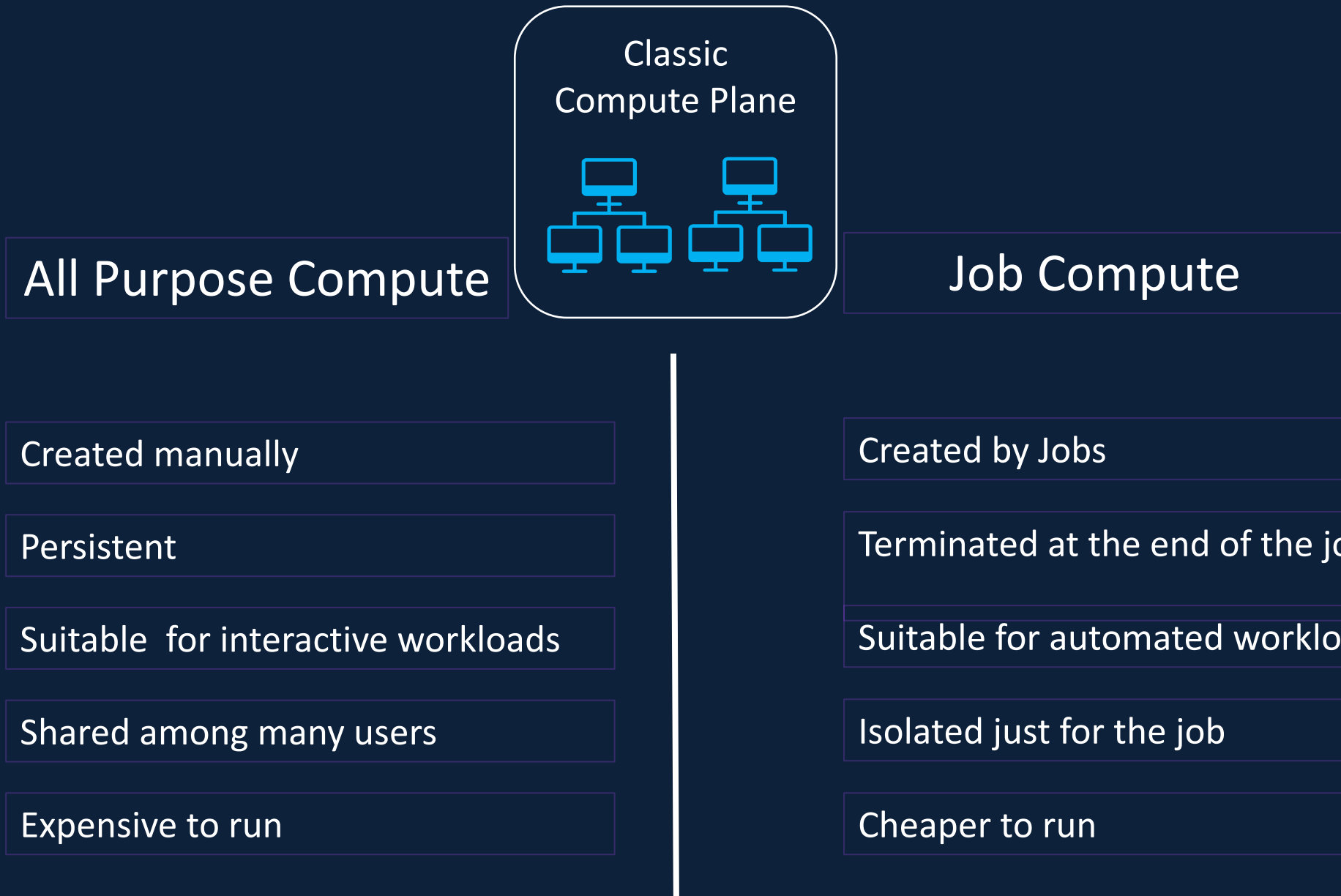
Fully Managed Service



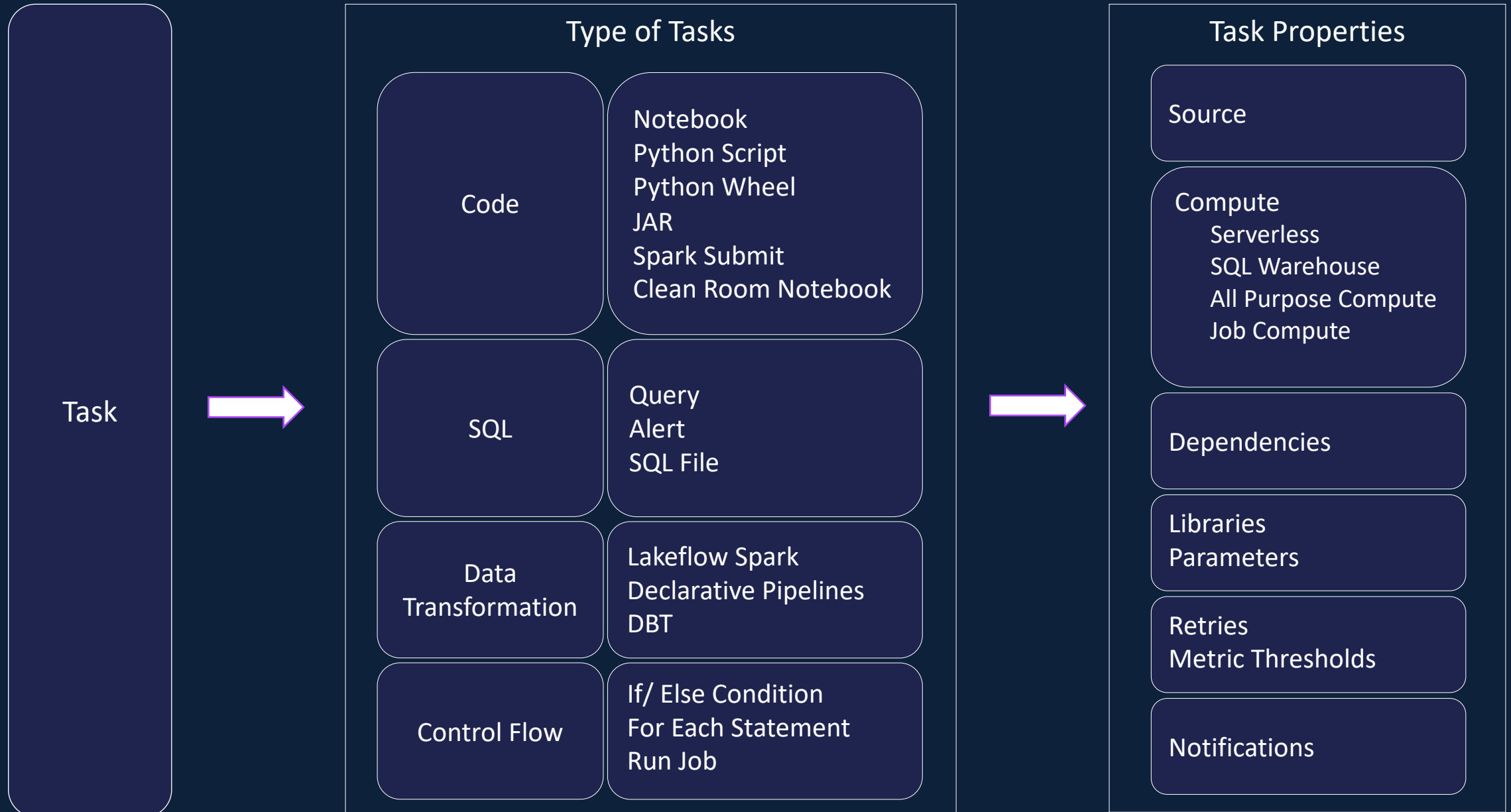
Introduction to Lakeflow Job Tasks



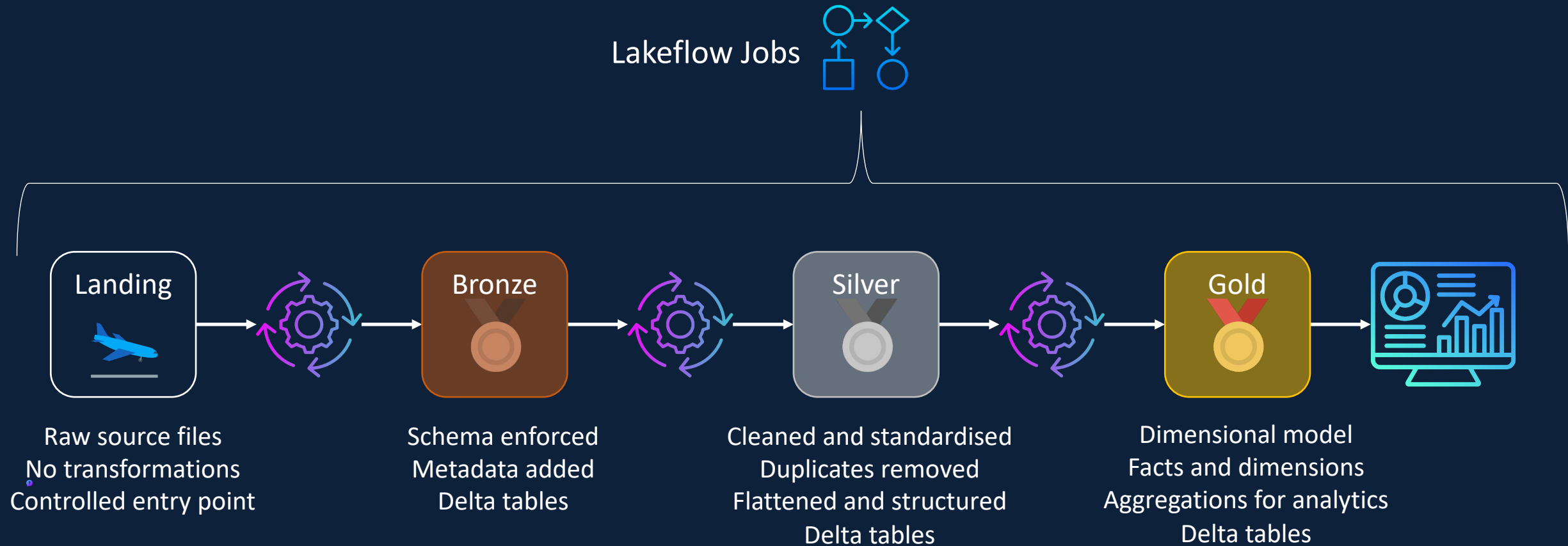
Databricks Classic Compute



Introduction to Lakeflow Job Tasks



Gold Layer – Section Introduction



How do we calculate yearly driver standings?

How do we analyse constructor dominance across multiple seasons?

How can we support efficient analytical queries without repeatedly rebuilding complex joins?

Reporting & Analytical Requirements



Driver standings for each season

Constructor standings for each season

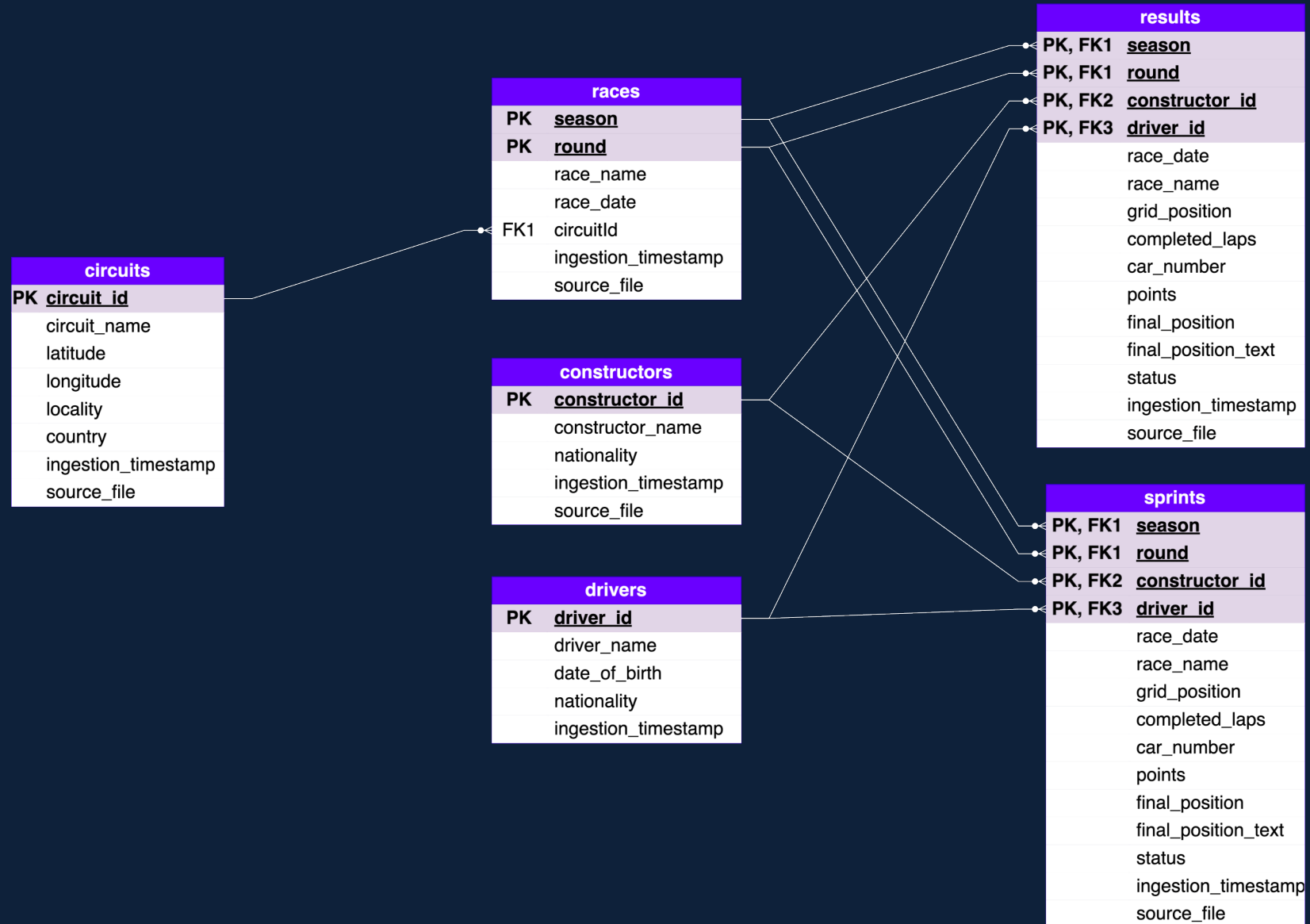
Analyse dominant drivers and teams

Support recent and historical analysis

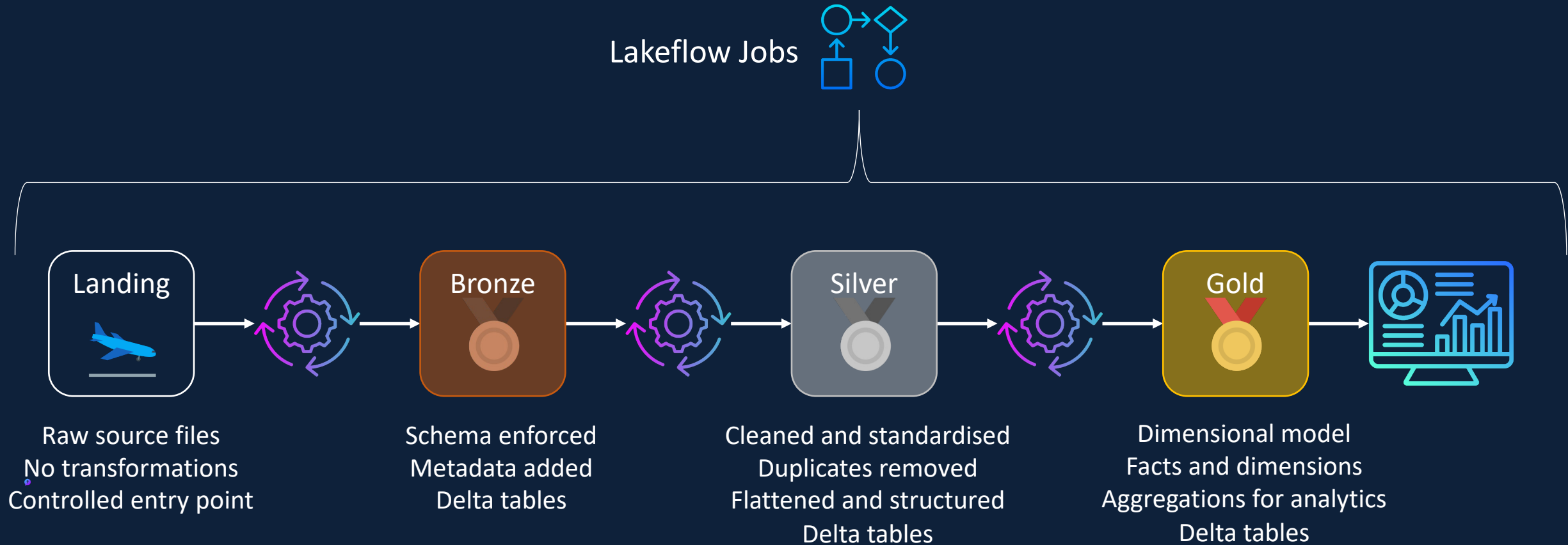
Optimised for reporting and analytical queries

Support for geographical analysis

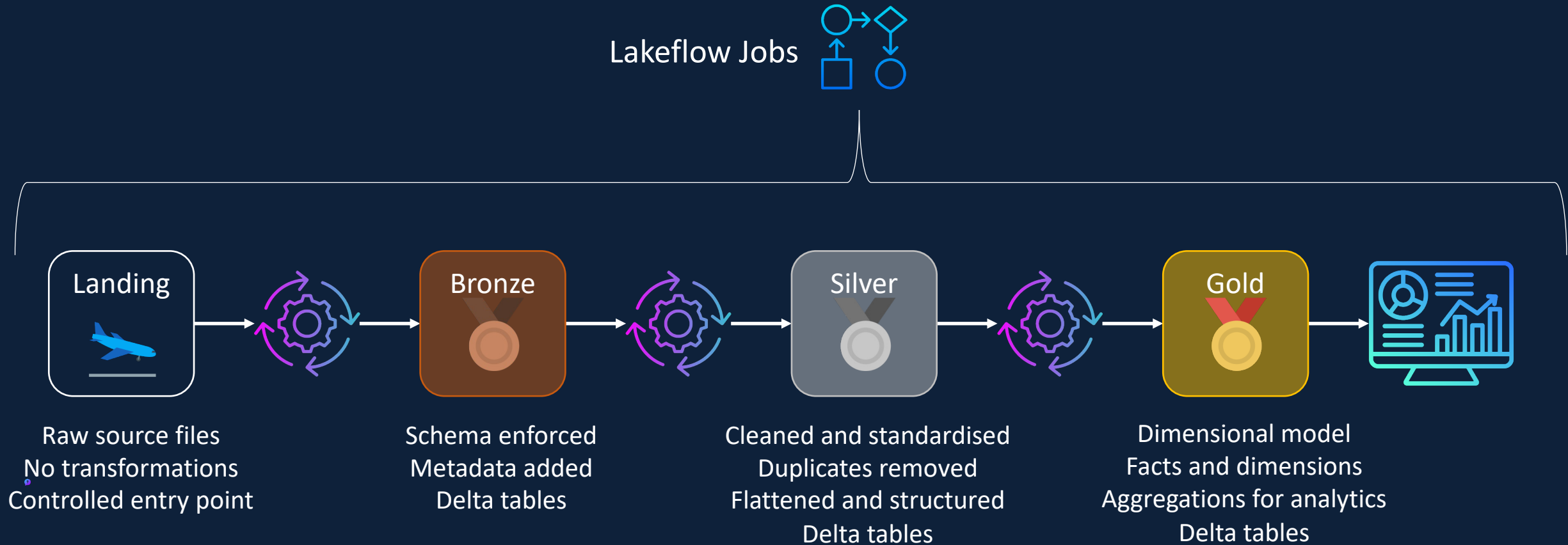
Silver Layer



Gold Layer



Gold Layer – Data Modelling



Gold Layer – Data Modelling

Fact

Measurable Events

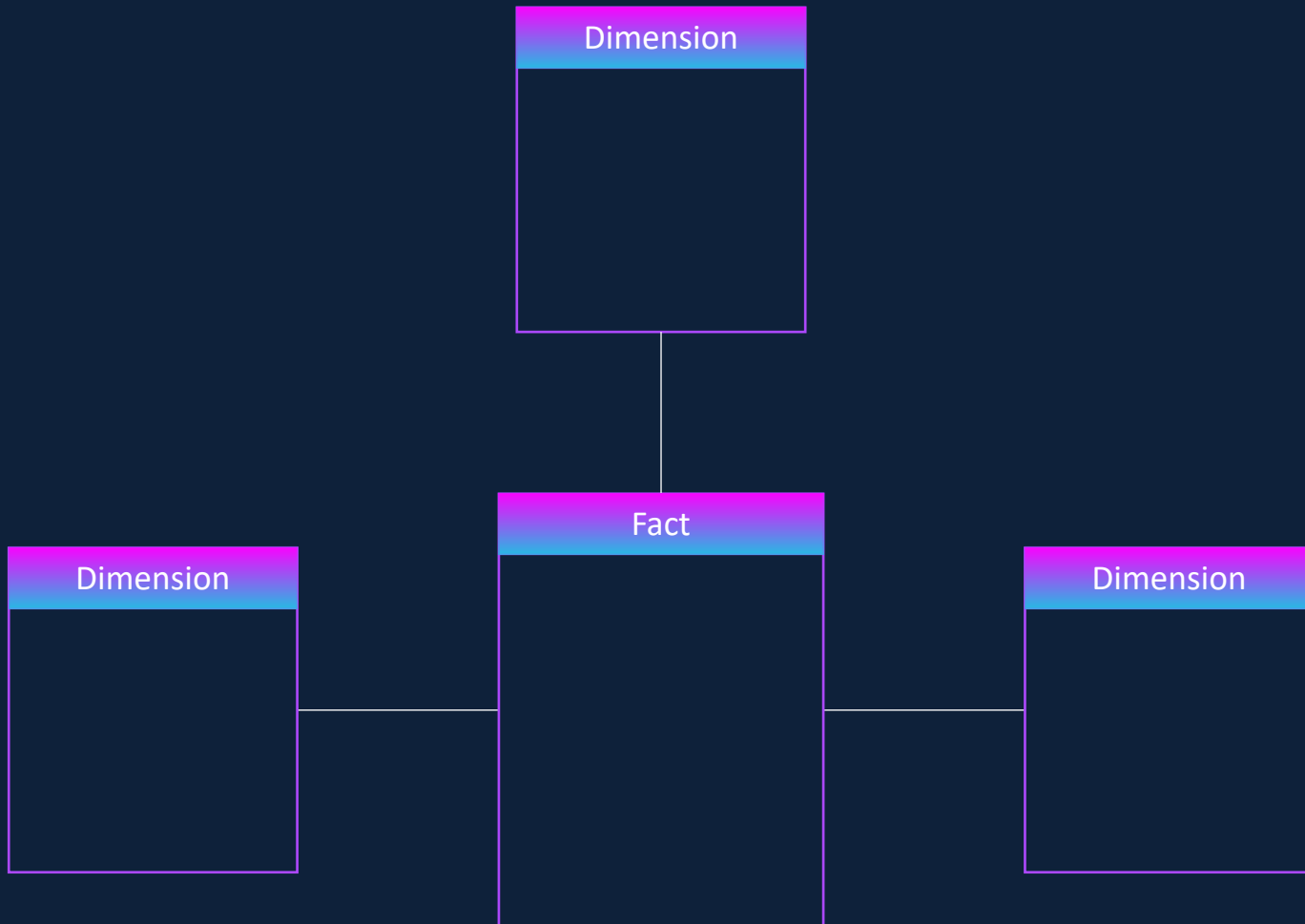
E.g., Race Results , Sprint Results

Dimension

Descriptive context around the Facts

E.g., Drivers, Constructors, Races

Gold Layer – Data Modelling

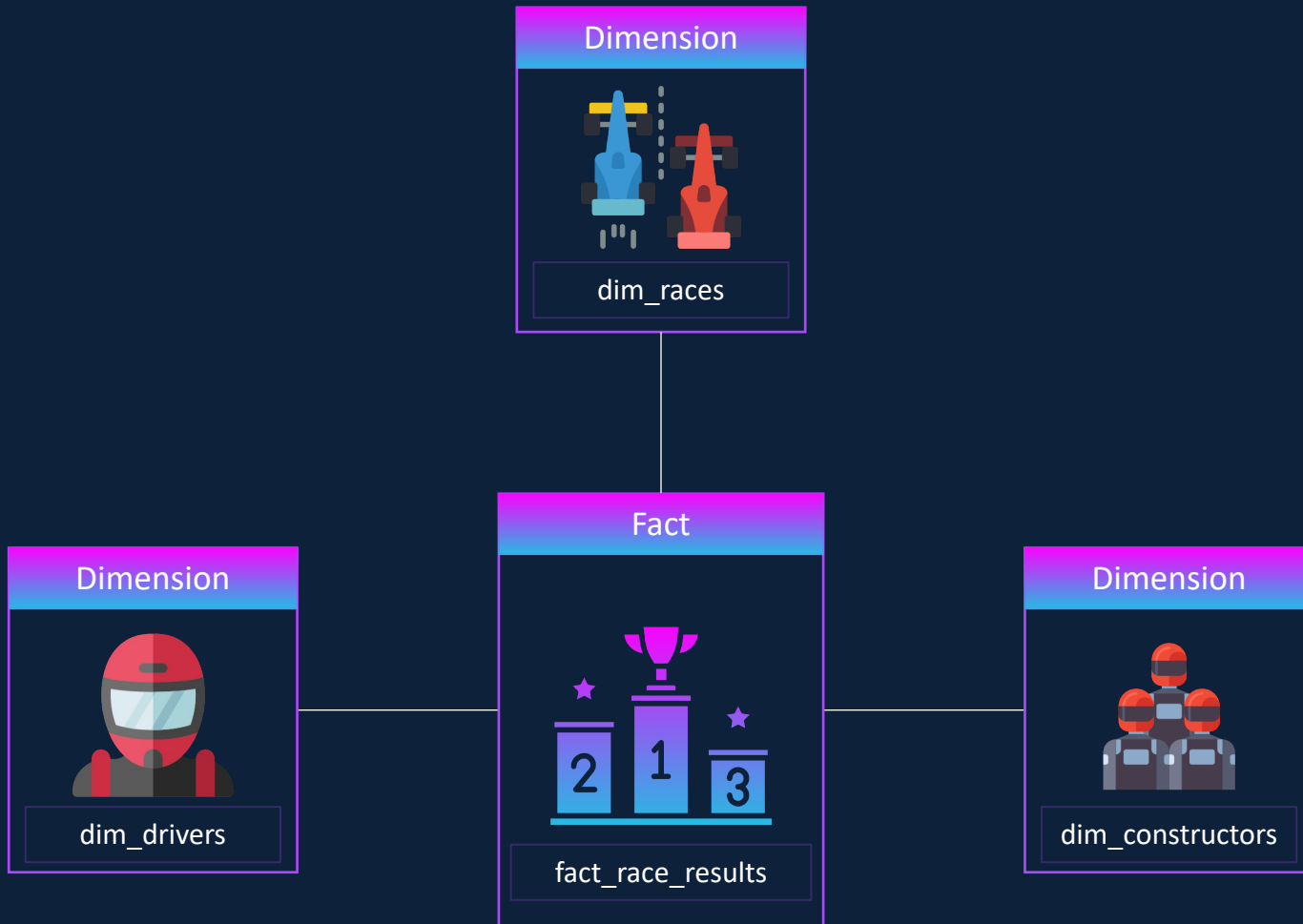


Filter by attributes in dimensions

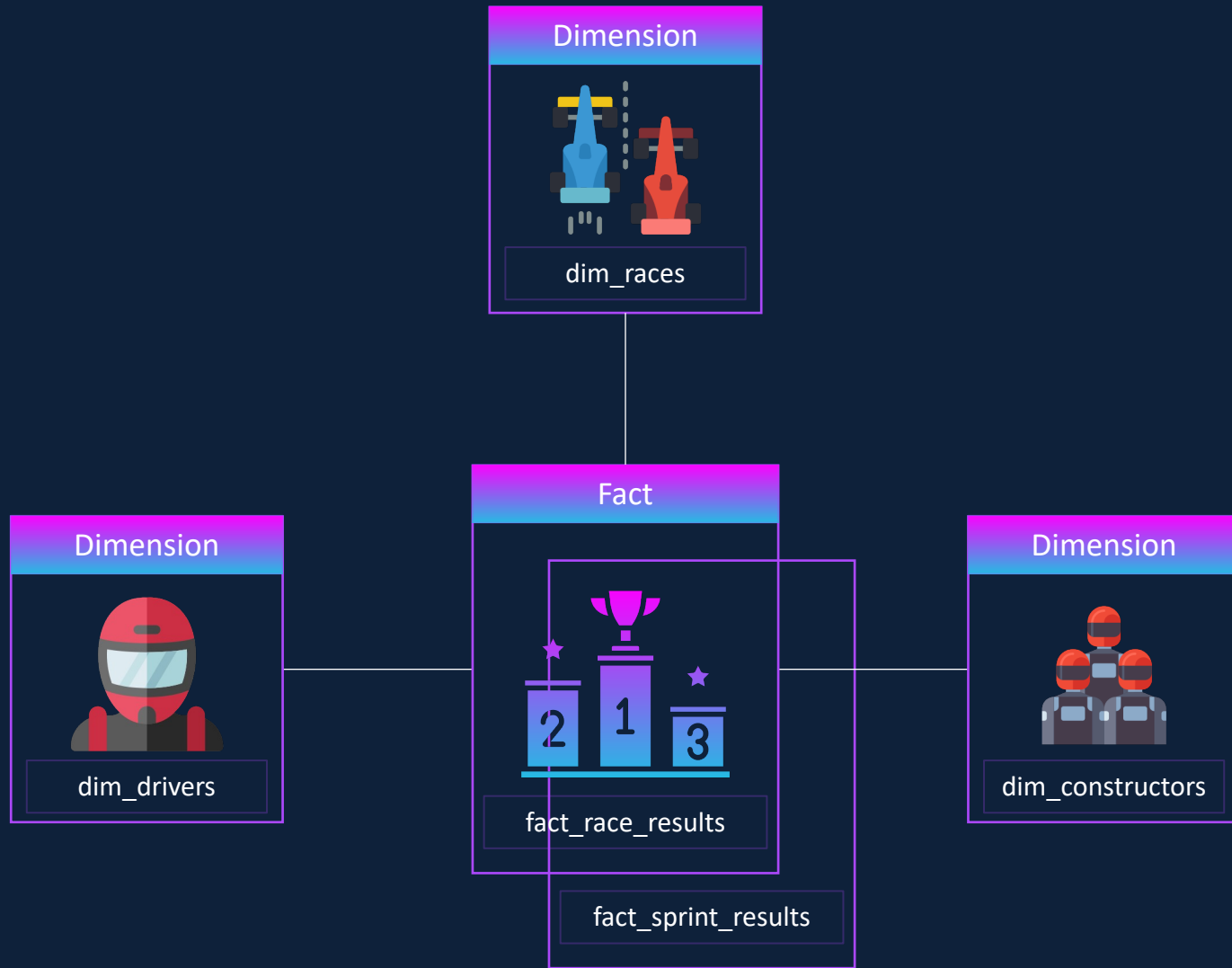
Aggregate measures from facts

Group by descriptive fields

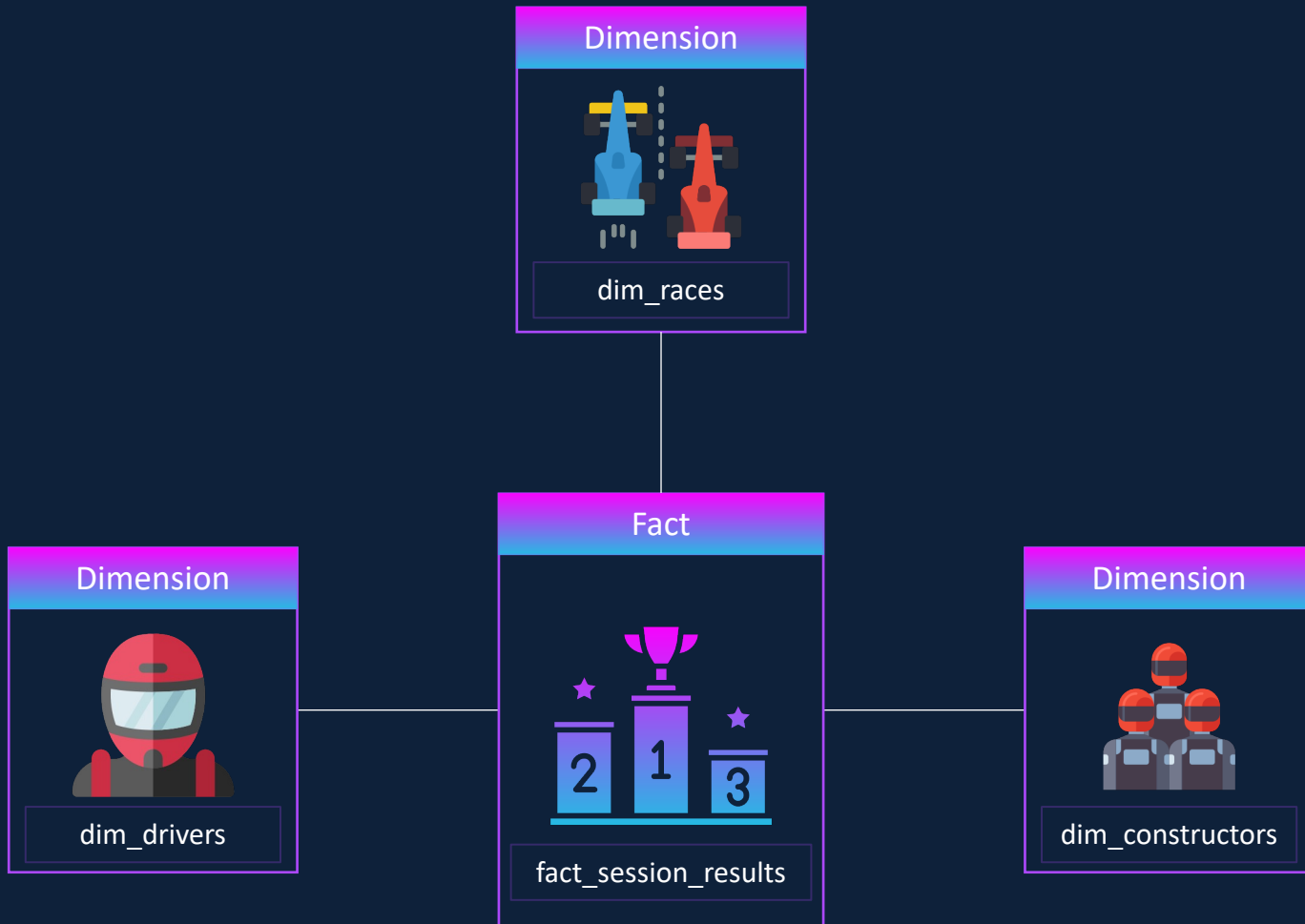
Gold Layer – Data Modelling



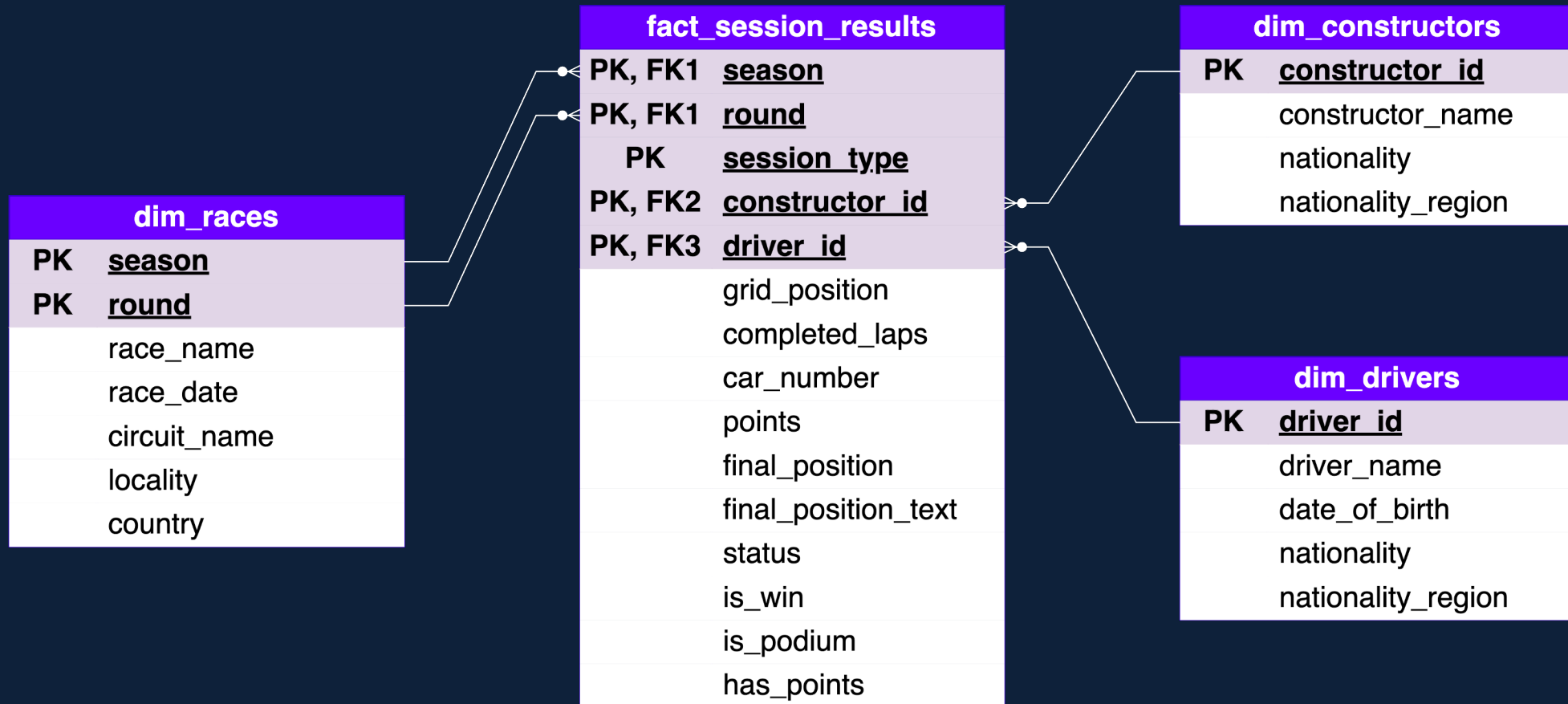
Gold Layer – Data Modelling



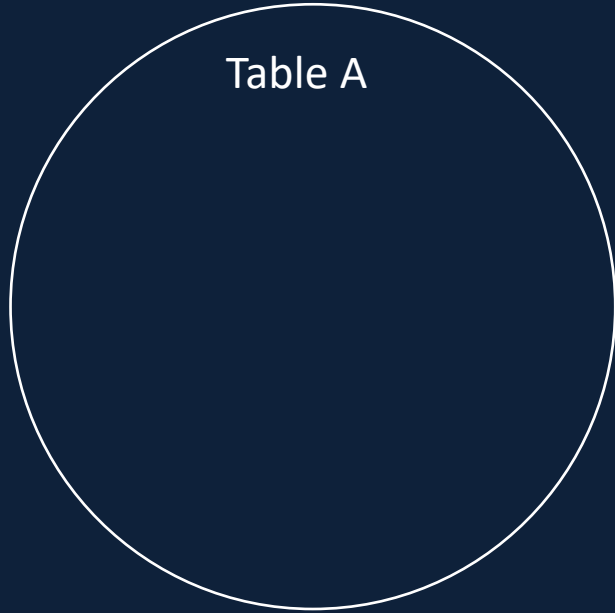
Gold Layer – Data Modelling



Gold Layer – Data Modelling

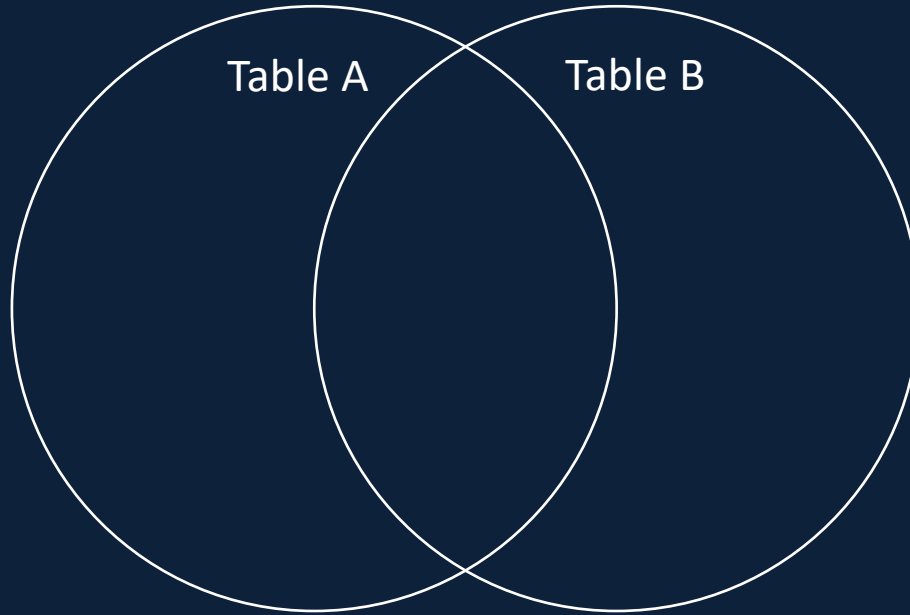


Types of Joins



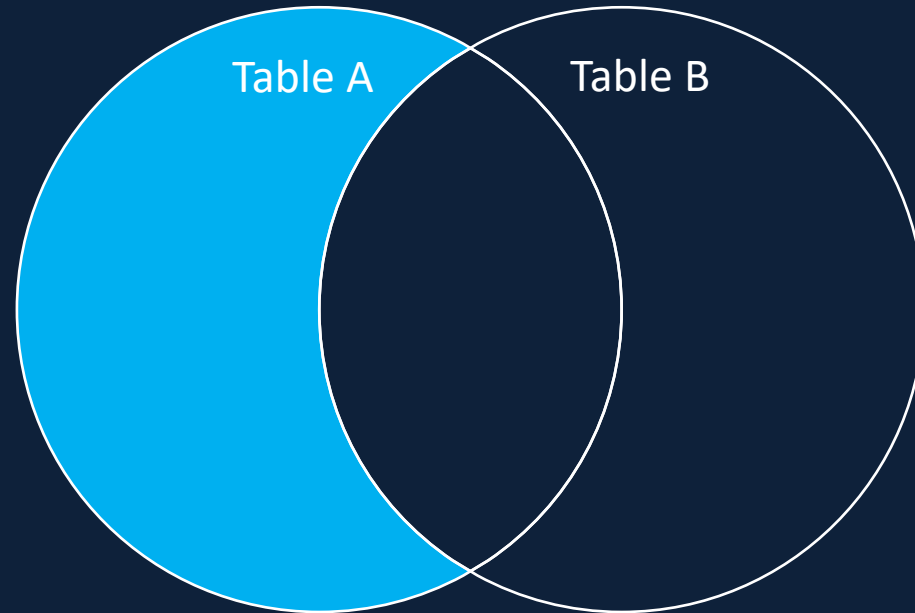
Types of Joins

Join Condition
 $\text{TableA.Id} = \text{TableB.Id}$



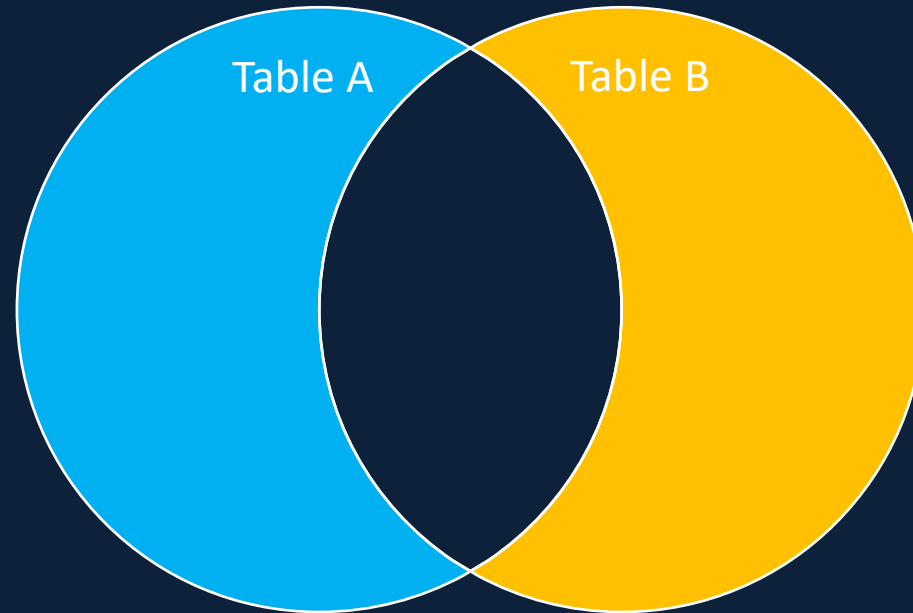
Types of Joins

Join Condition



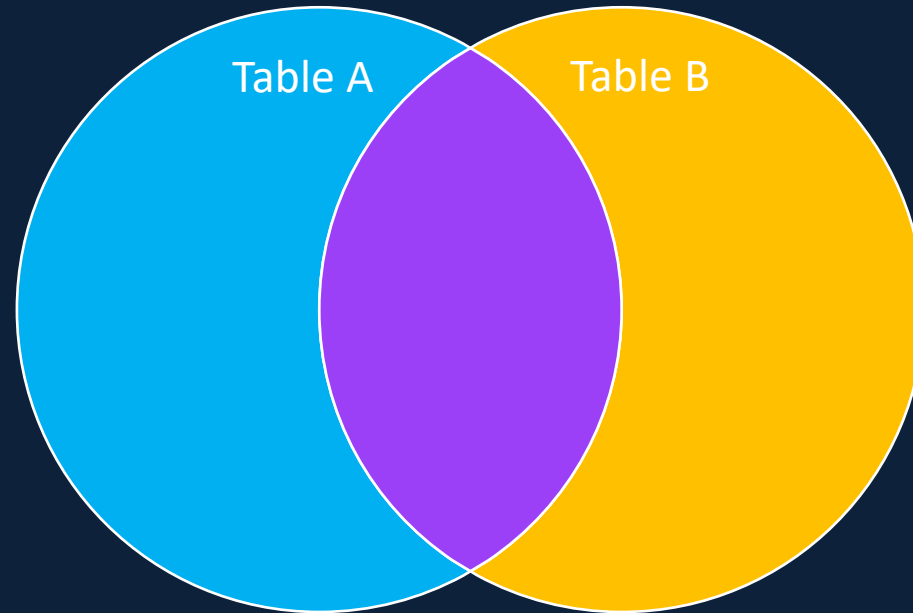
Types of Joins

Join Condition



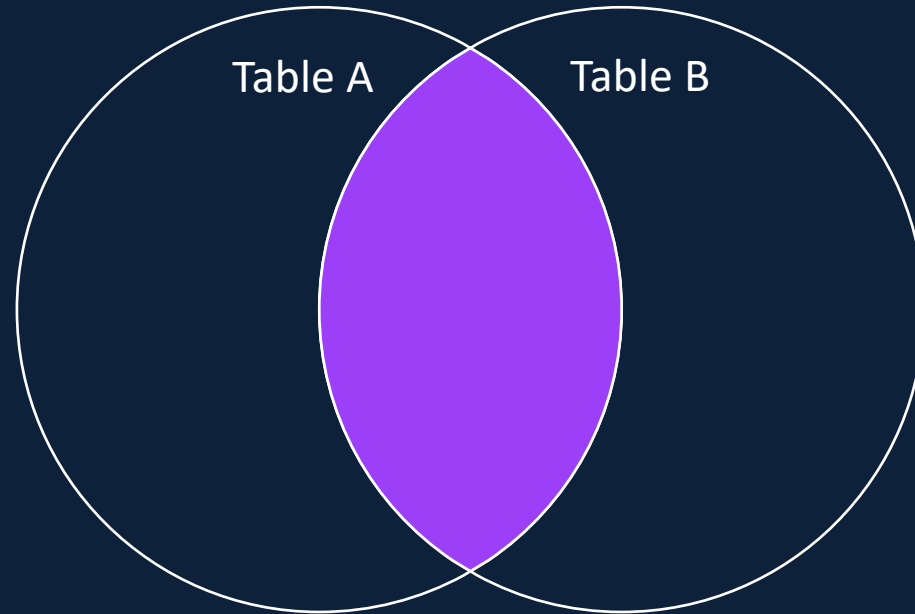
Types of Joins

Join Condition



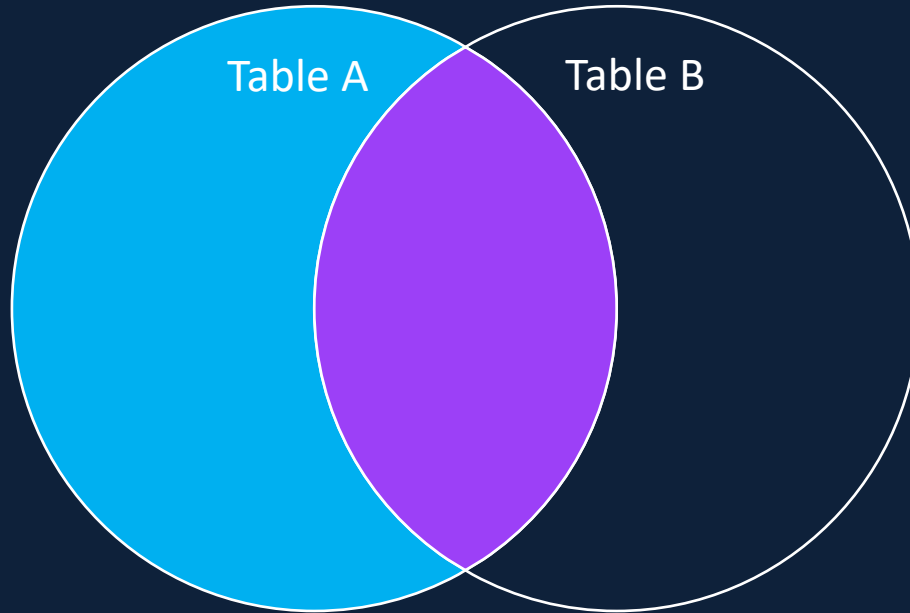
Types of Joins

Join Condition



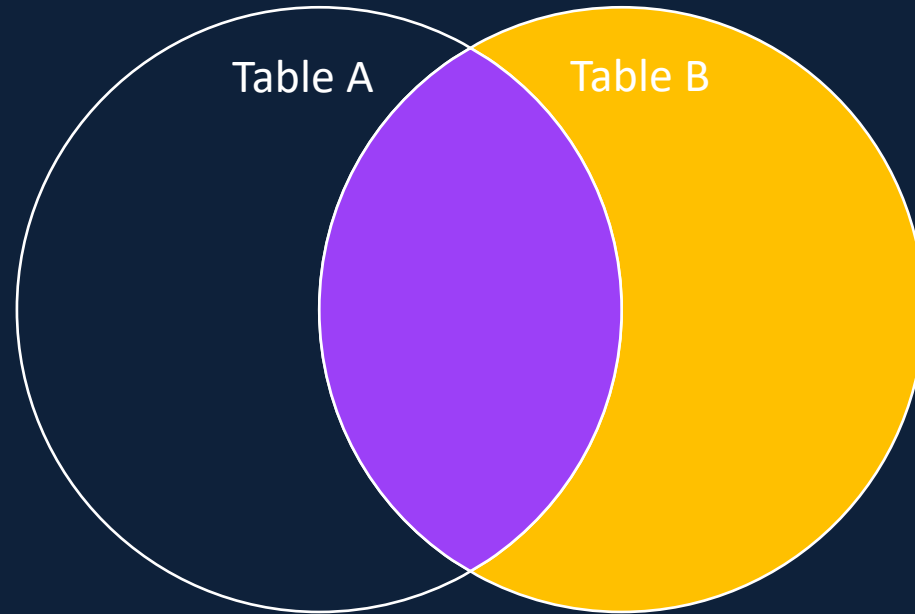
Types of Joins

Join Condition



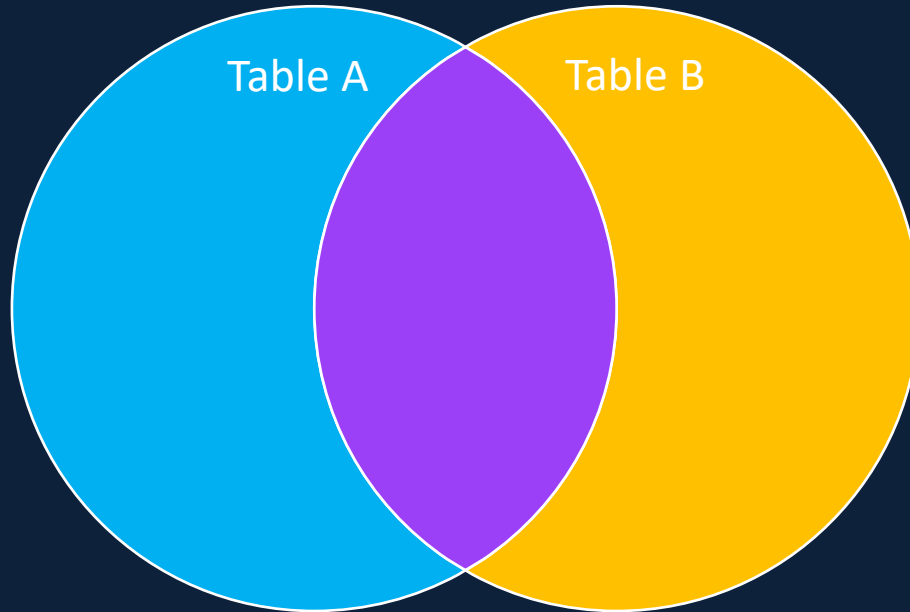
Types of Joins

Join Condition



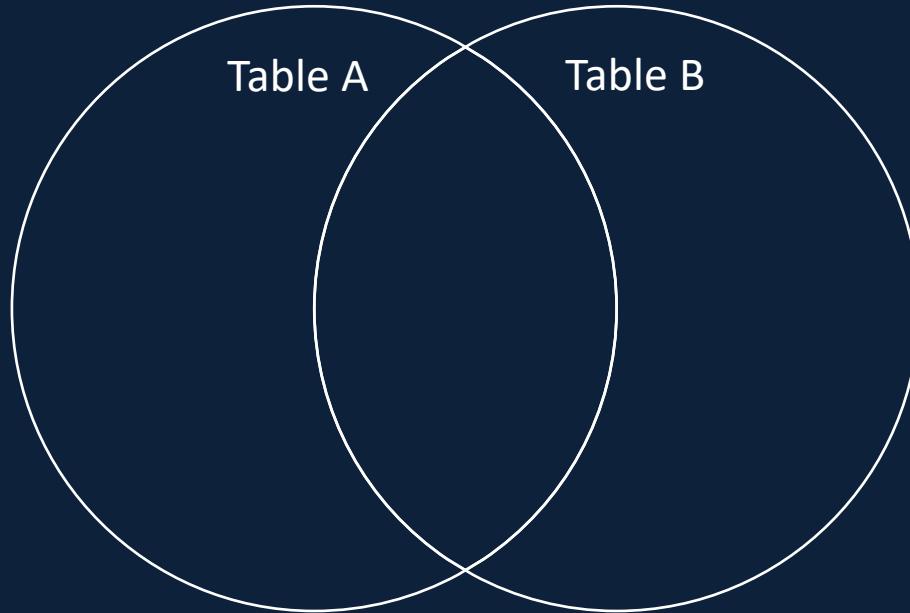
Types of Joins

Join Condition



Types of Joins

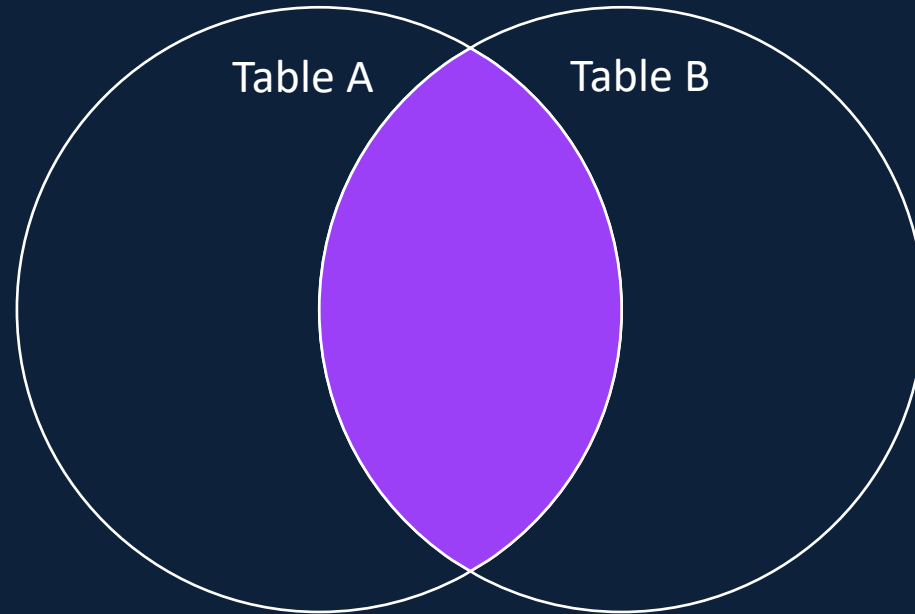
Join Condition



Types of Joins

Inner Join

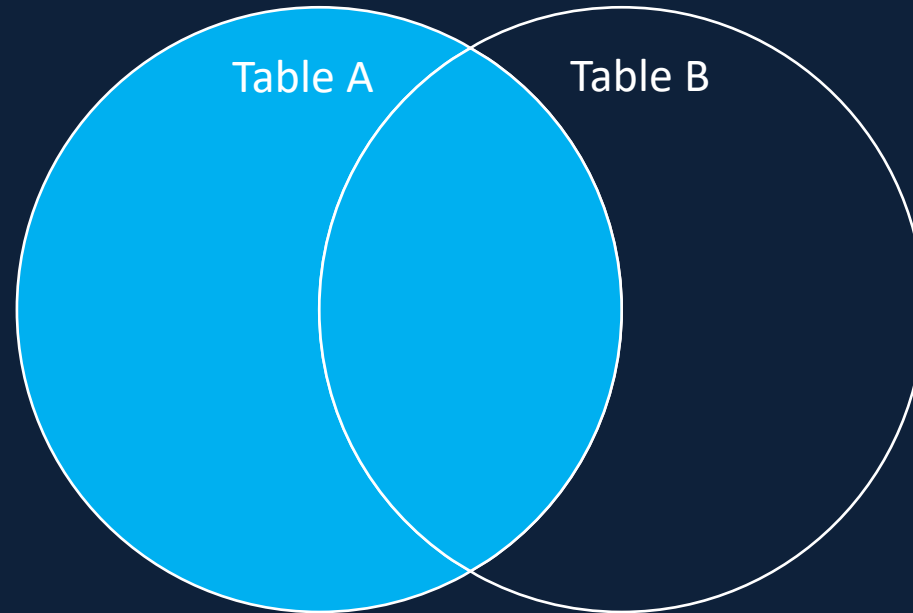
Join Condition



Types of Joins

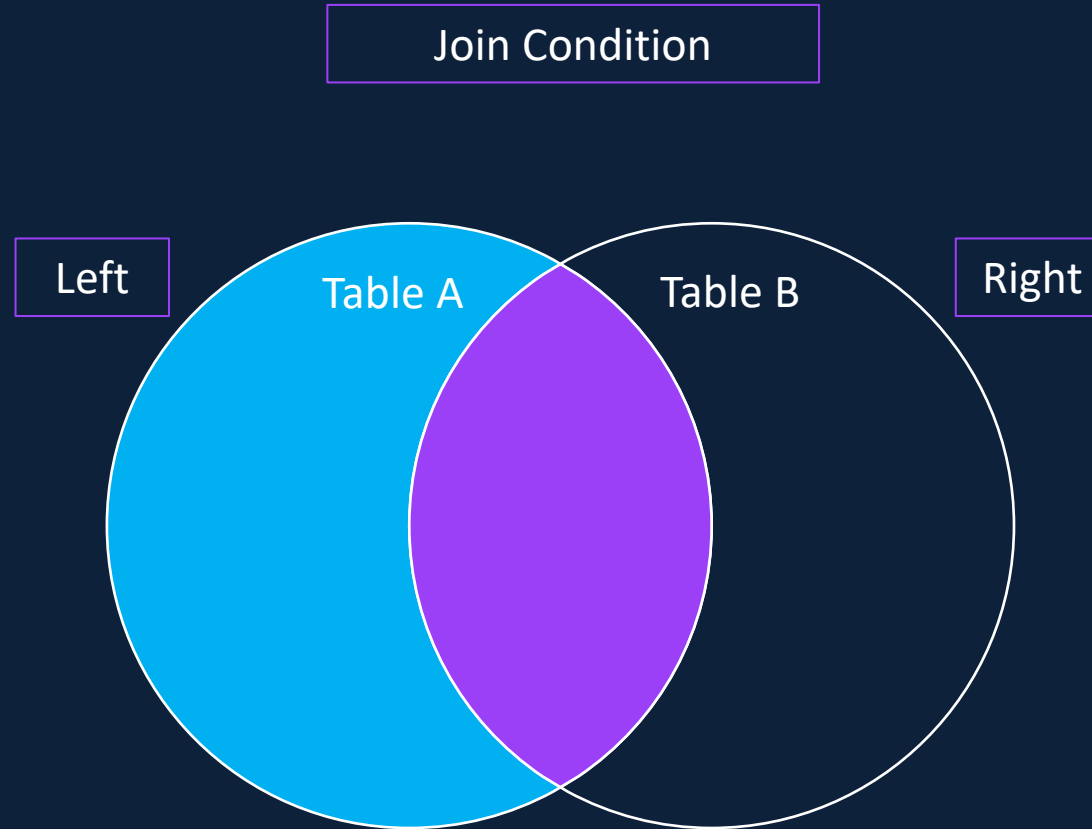
Left Outer Join

Join Condition



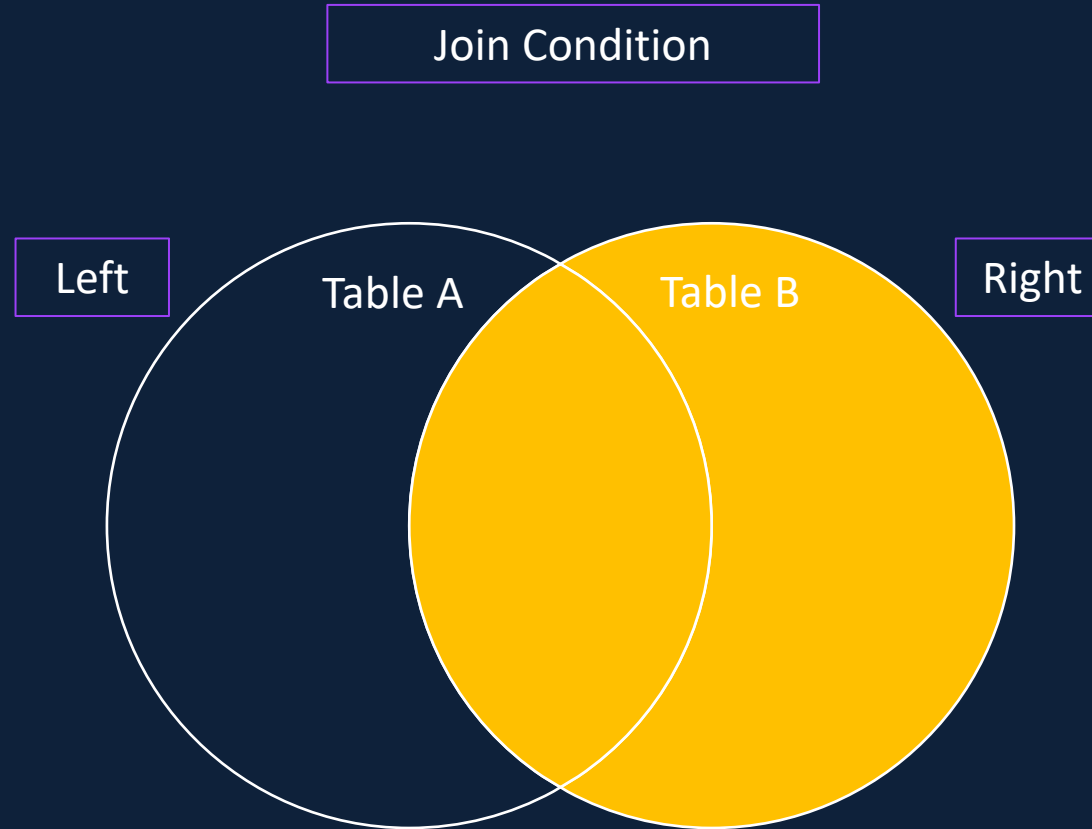
Types of Joins

Left Outer Join



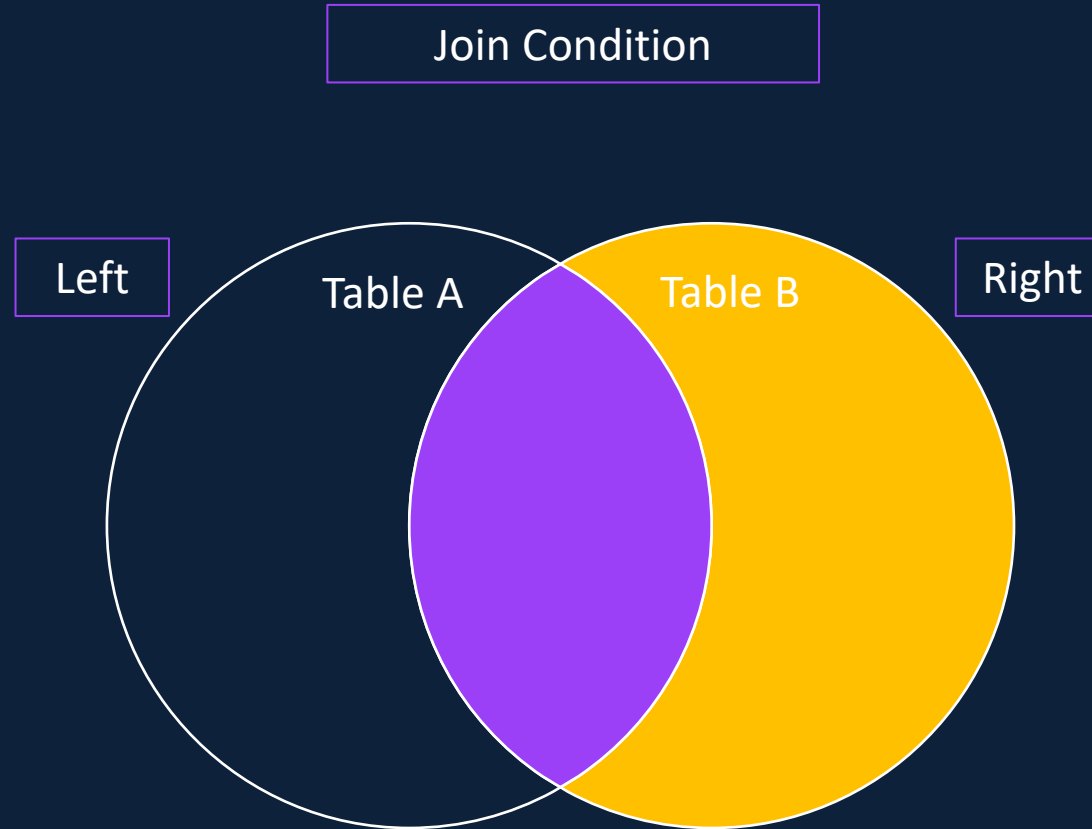
Types of Joins

Right Outer Join



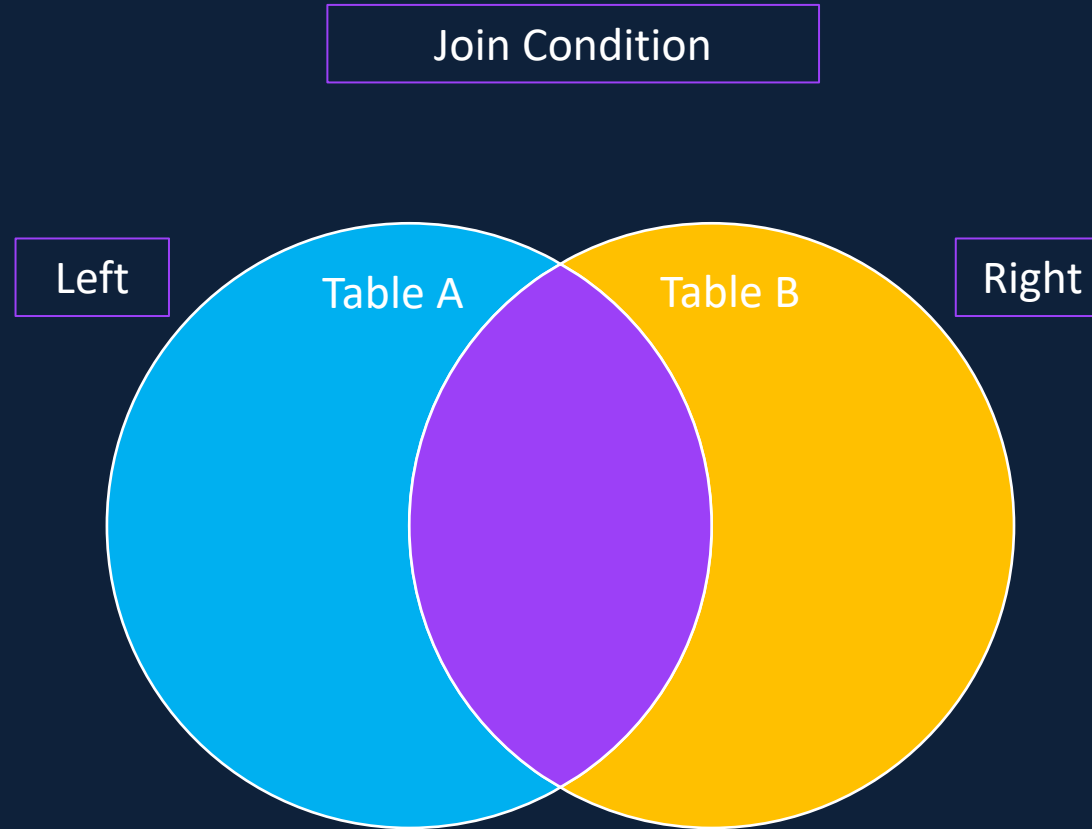
Types of Joins

Right Outer Join

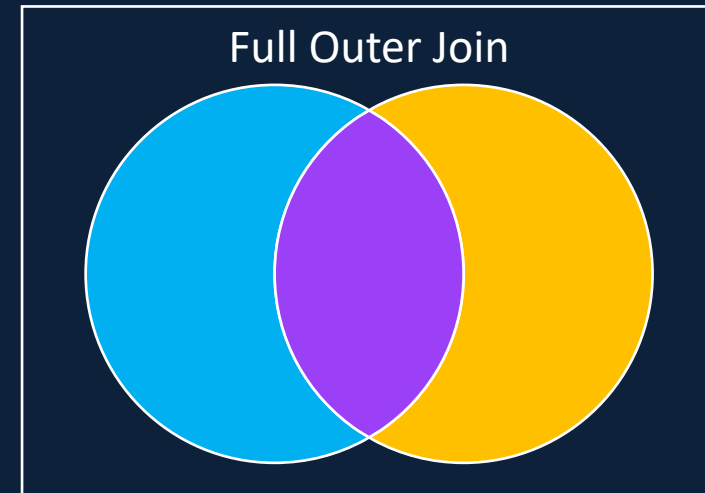
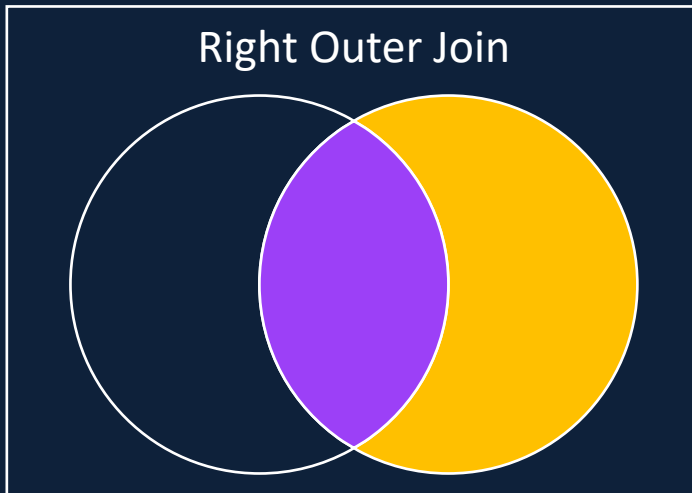
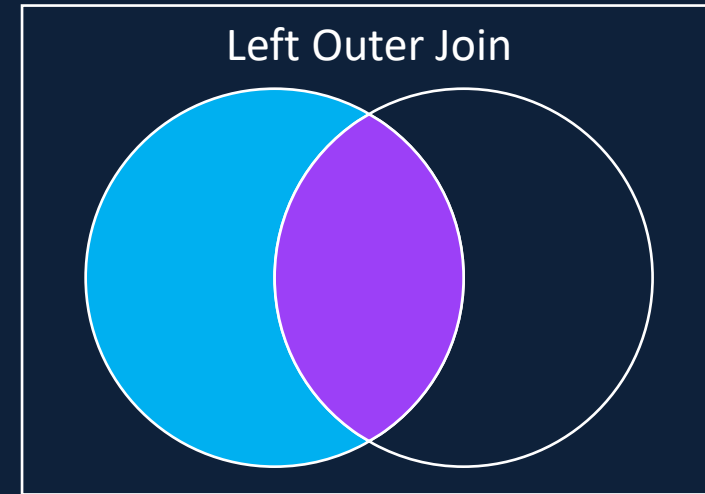
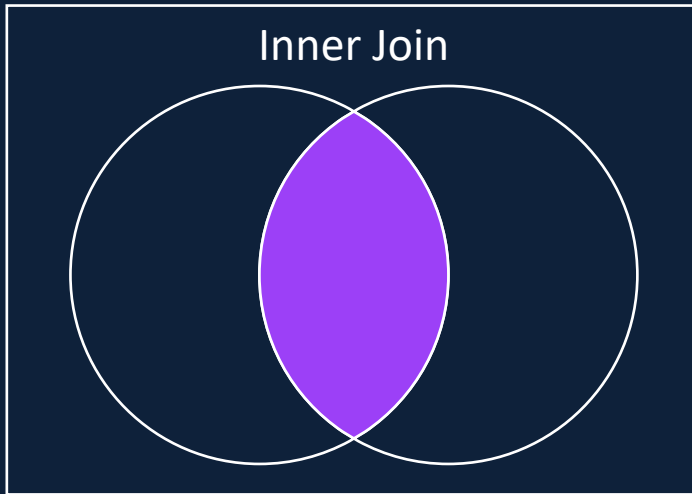


Types of Joins

Full Outer Join

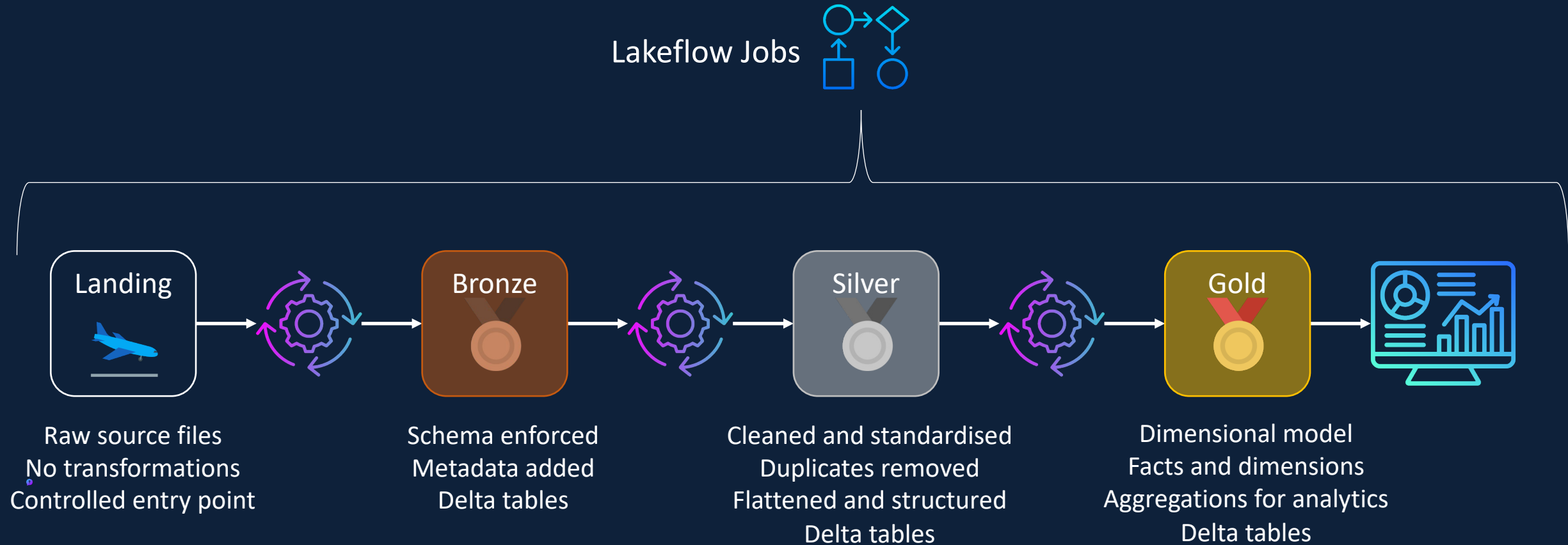


Types of Joins



Reporting & Analytical Requirements

Reporting & Analytical Requirements



Reporting & Analytical Requirements



How did the drivers perform during the championship in each season?

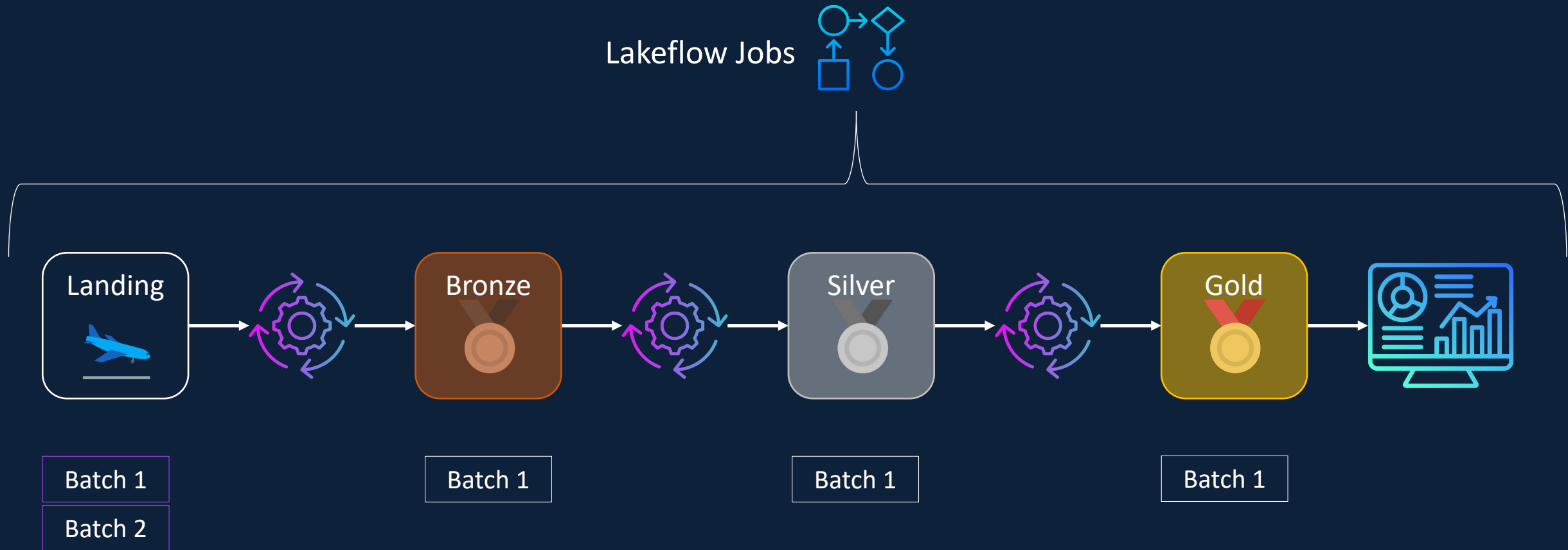
How did the constructors perform over each of the seasons?

Which drivers have been the most dominant in Formula 1 history?

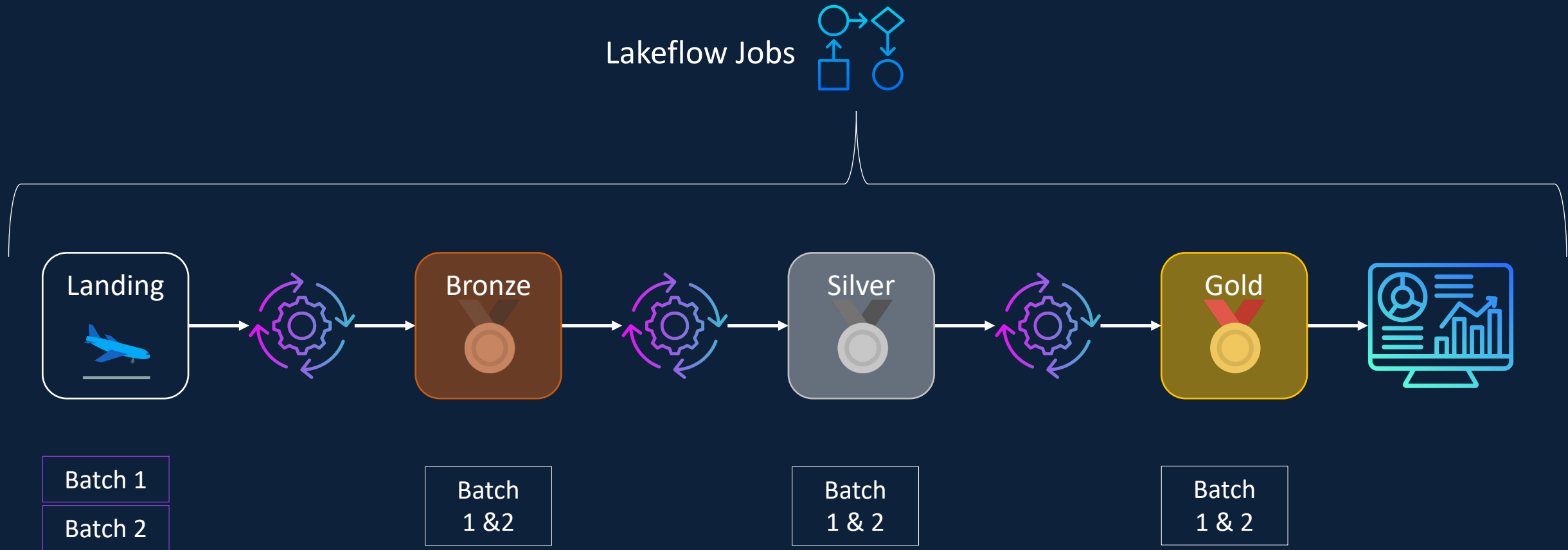
Which constructors have achieved the greatest success over time?

Incremental Data Processing

Full Refresh

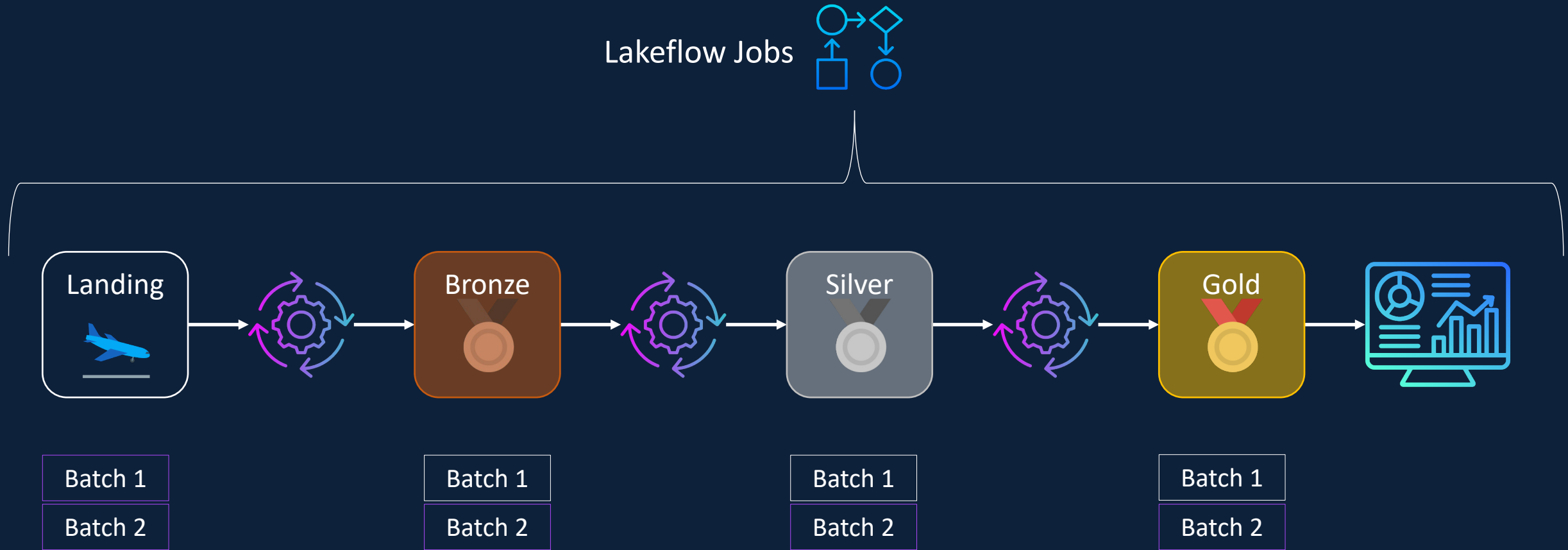


Full Refresh



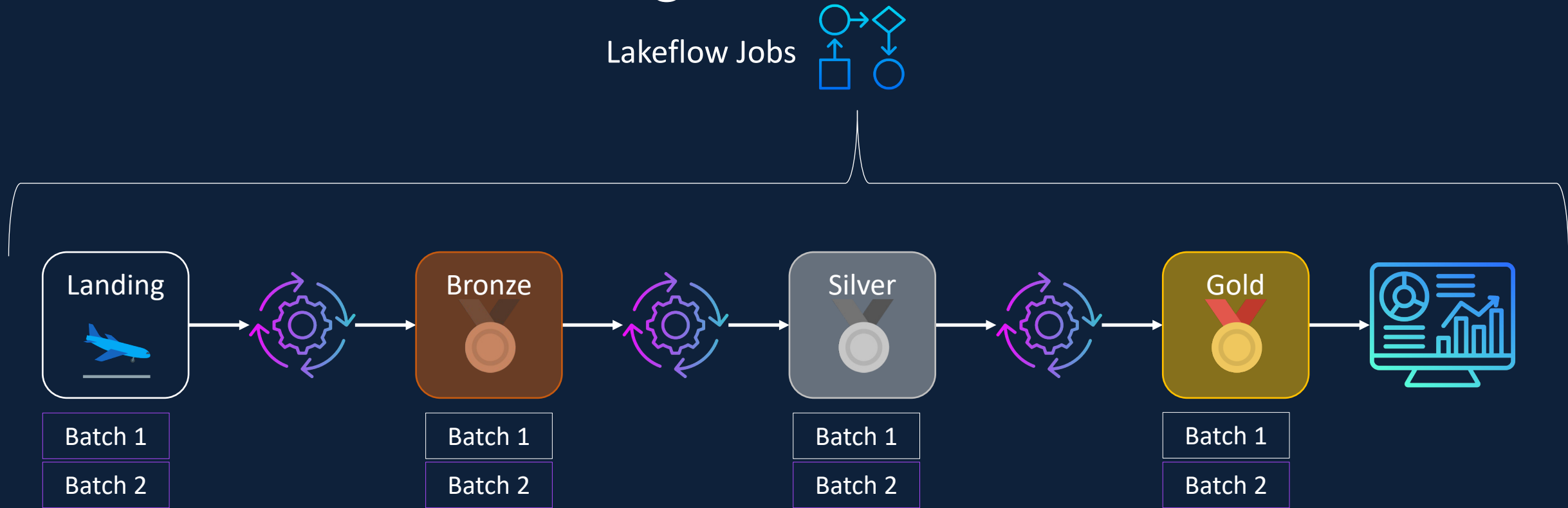
Every Execution Processes All Data

Incremental Processing



Only New Data is Processed

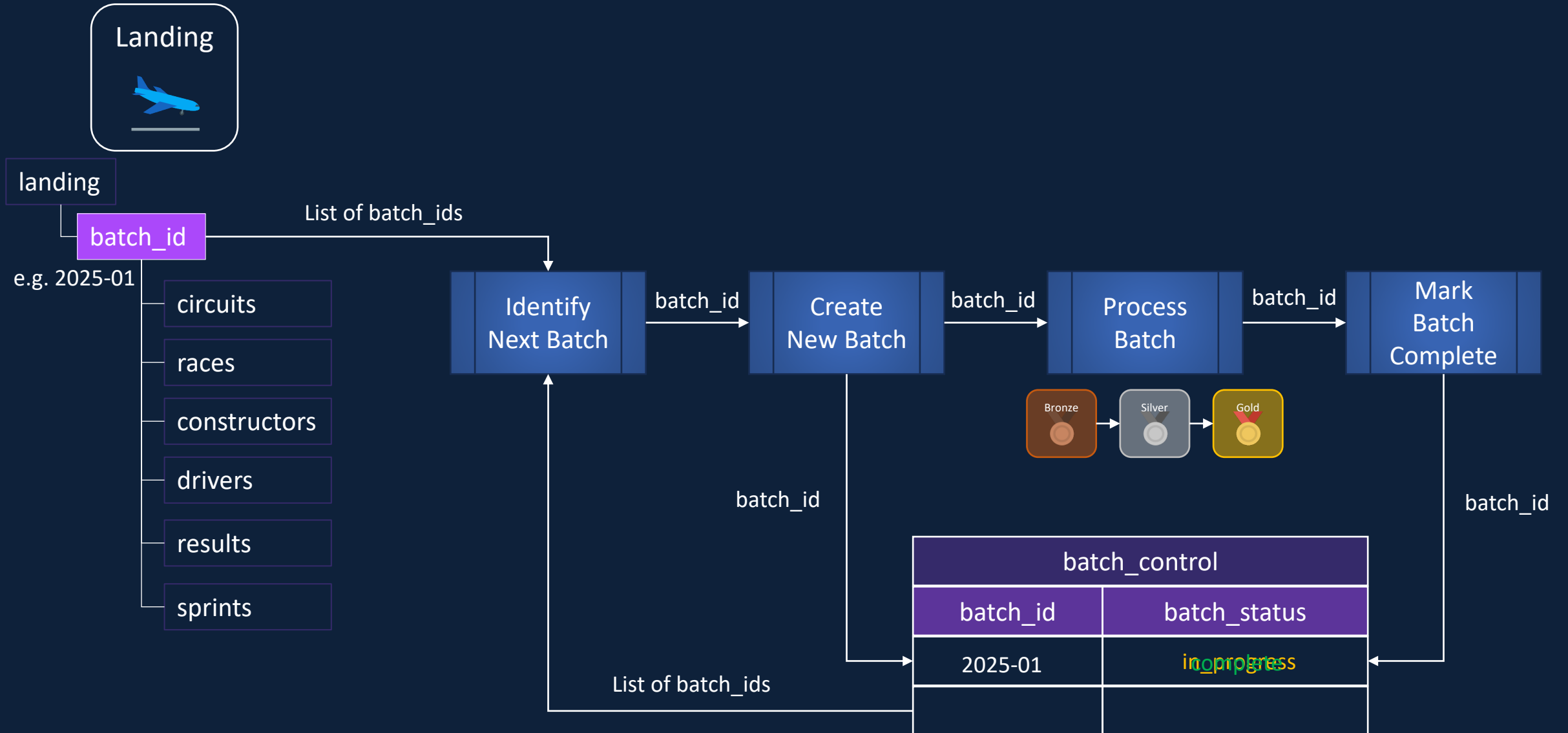
Incremental Processing



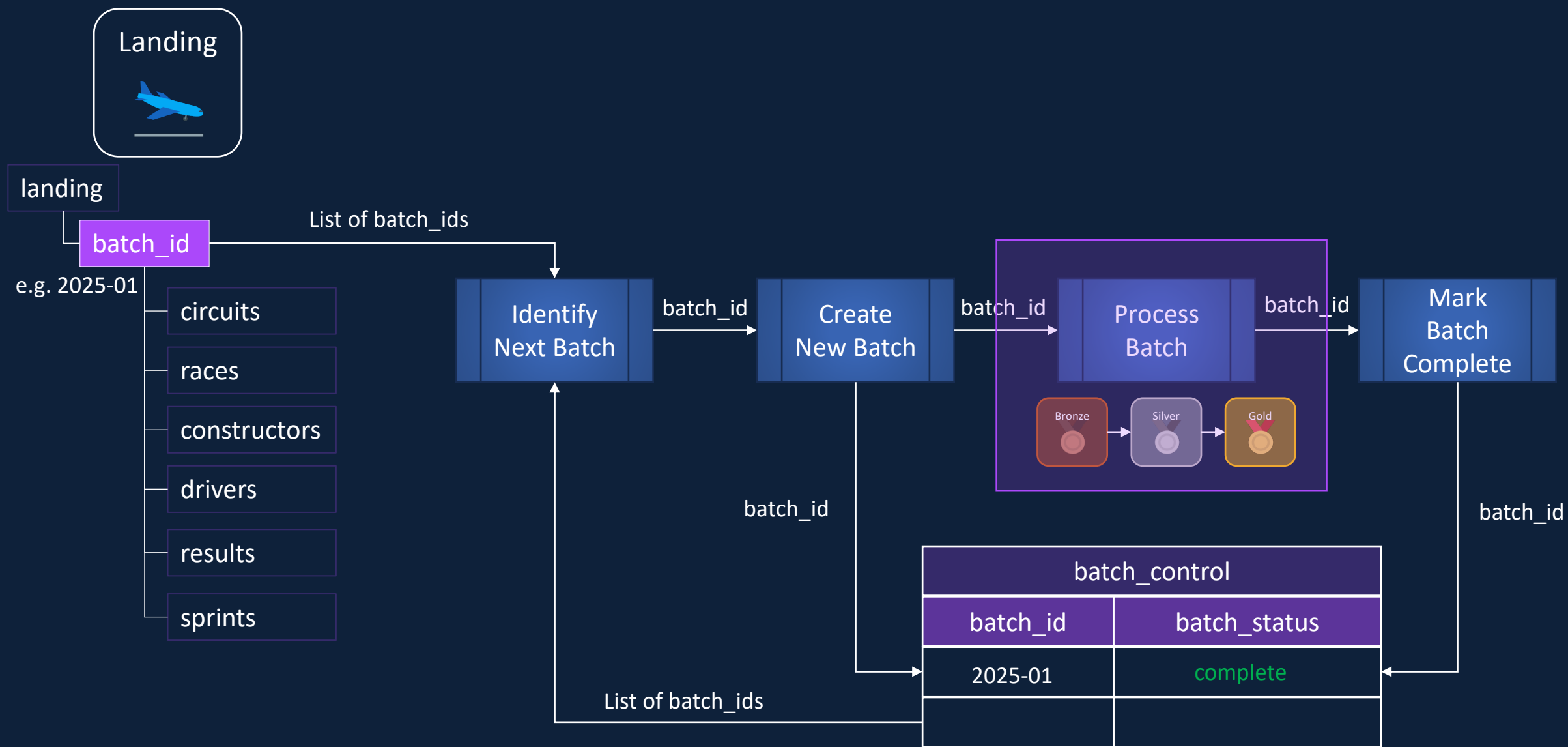
There are multiple ways to build incremental pipelines

We will be using simple batch based approach

Incremental Processing

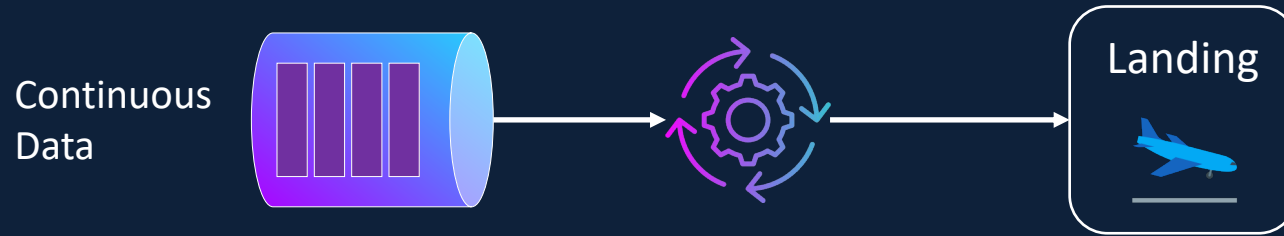


Incremental Data Processing

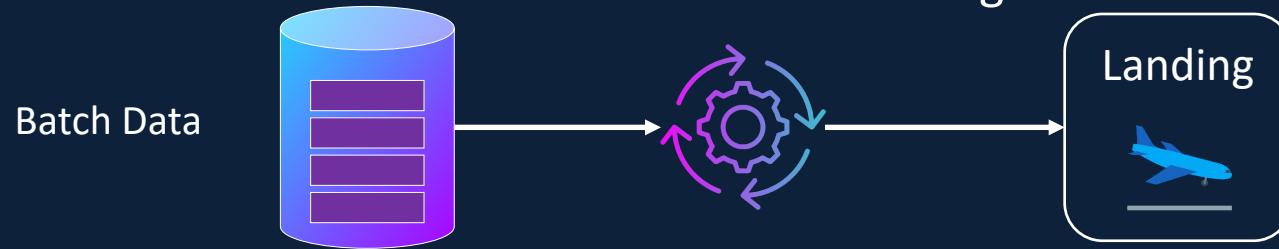


Incremental Data Processing

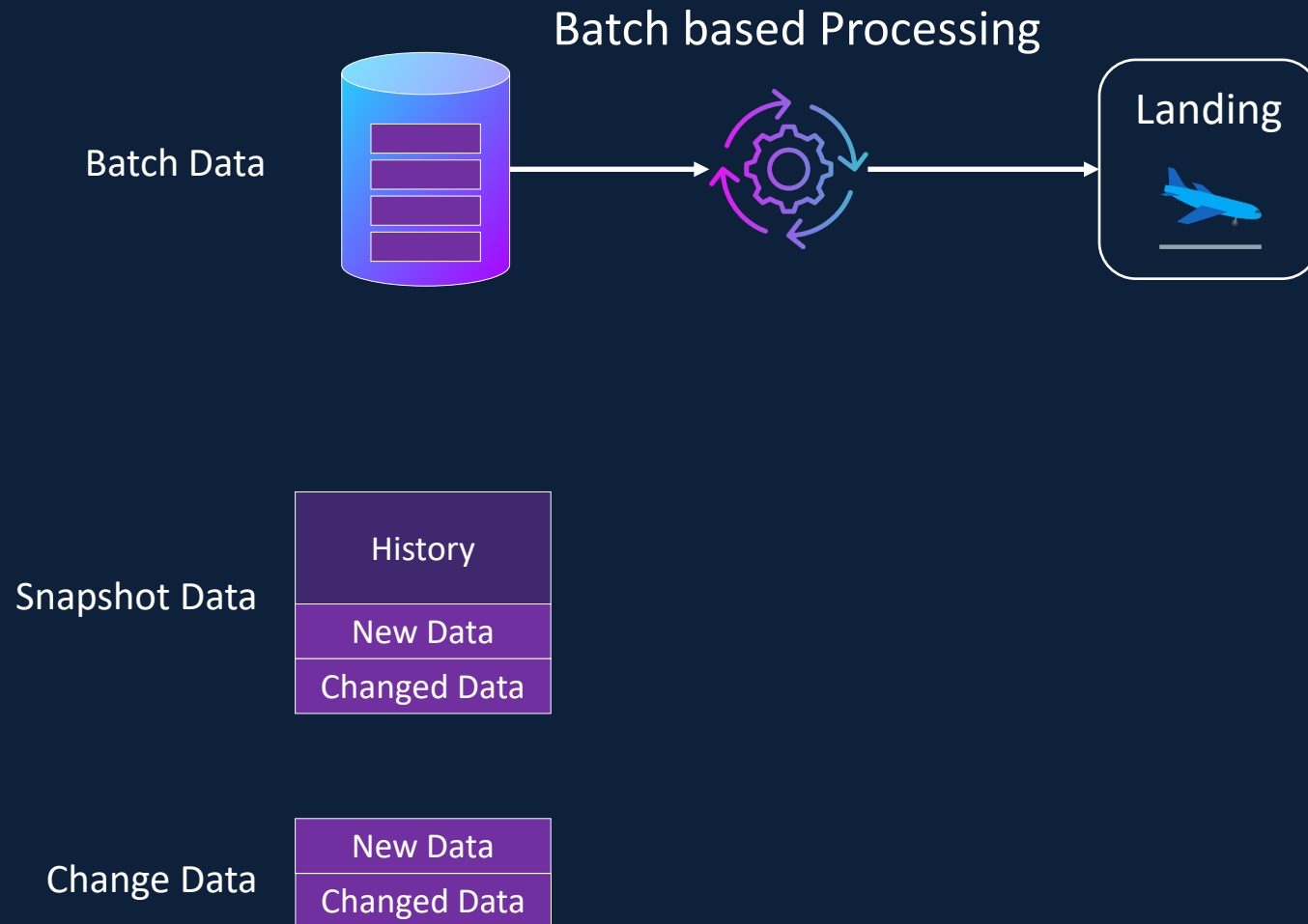
Stream Processing



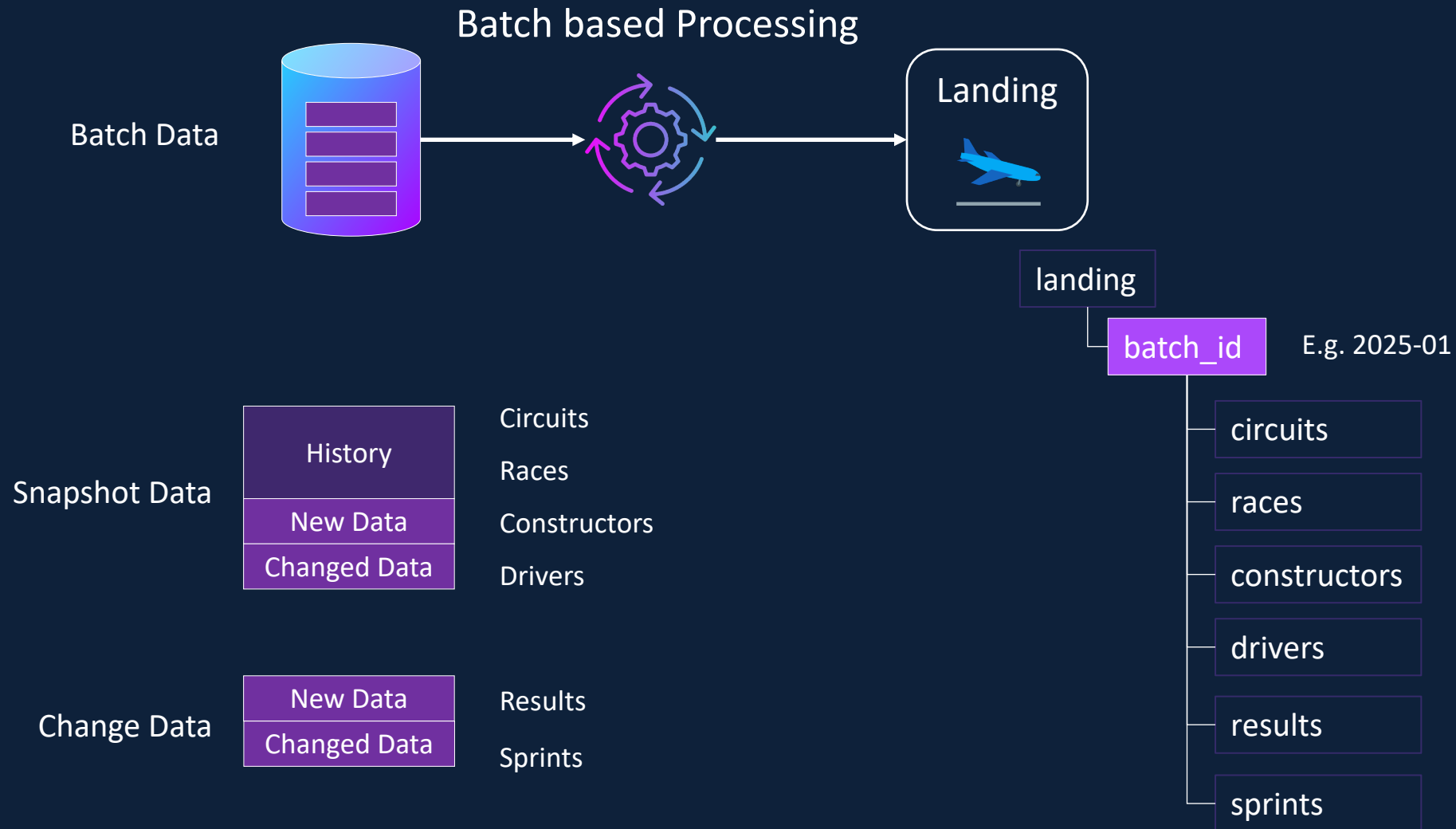
Batch based Processing



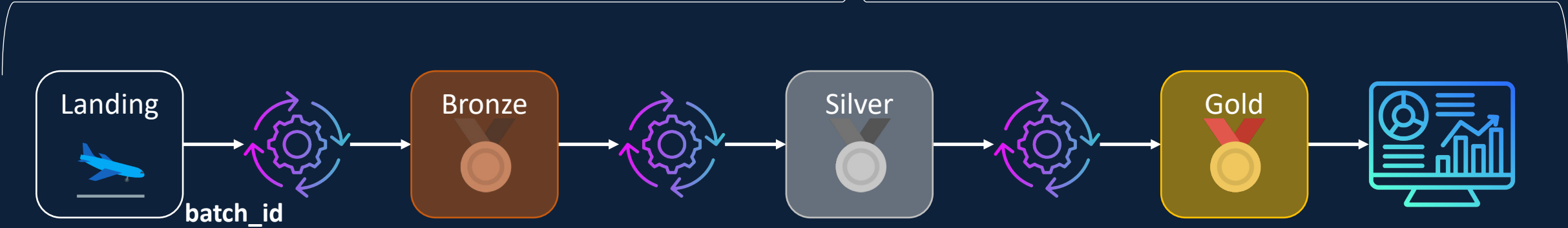
Incremental Data Processing



Incremental Data Processing



Incremental Data Processing



Snapshot Data

History
New Data
Changed Data

- Circuits
- Races
- Constructors
- Drivers

Append

OMerge

OMerge

Change Data

New Data
Changed Data

- Results
- Sprints

Append

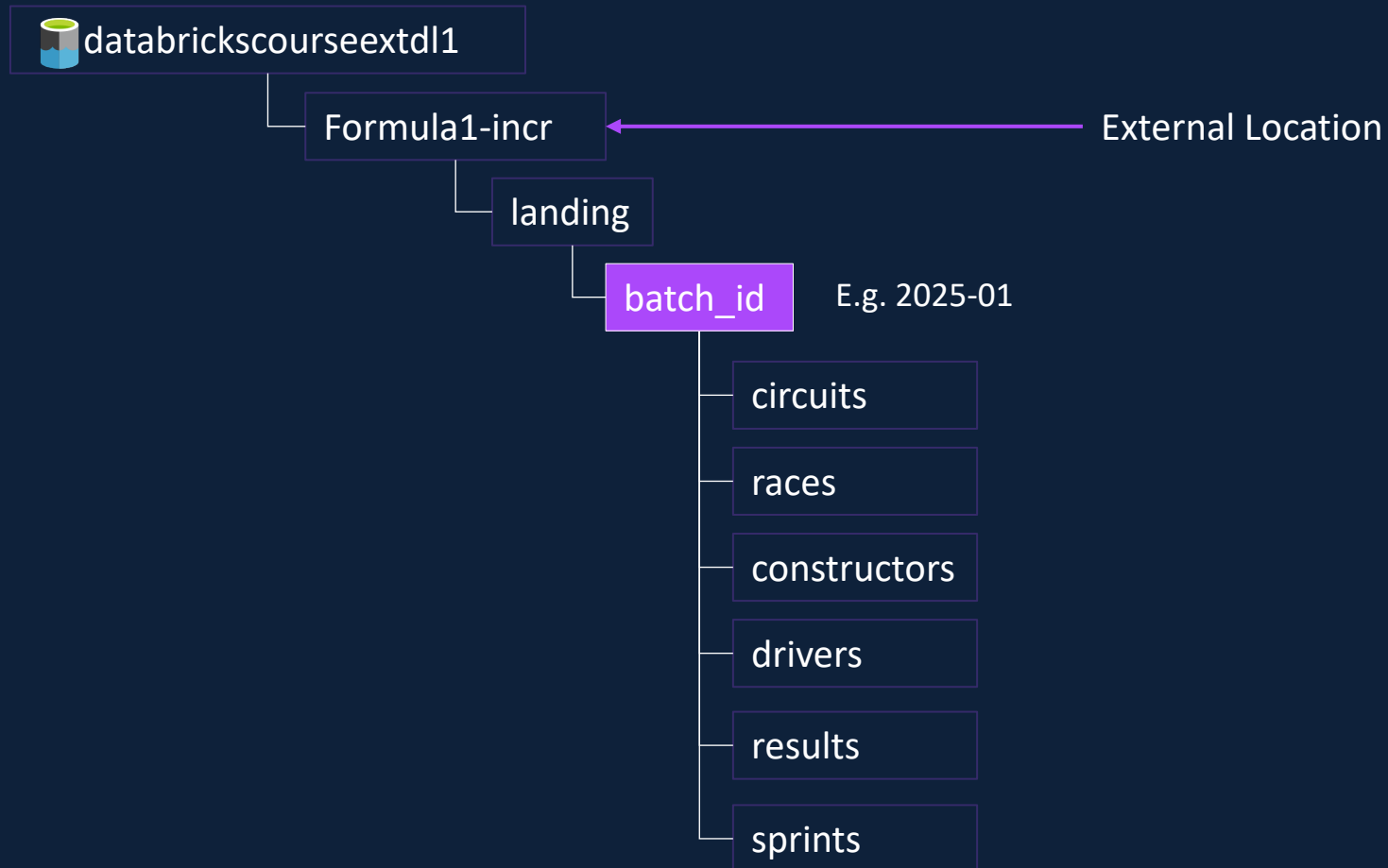
Merge

Merge

Incremental Load – Project Set-up

Incremental Load – Project Set-up

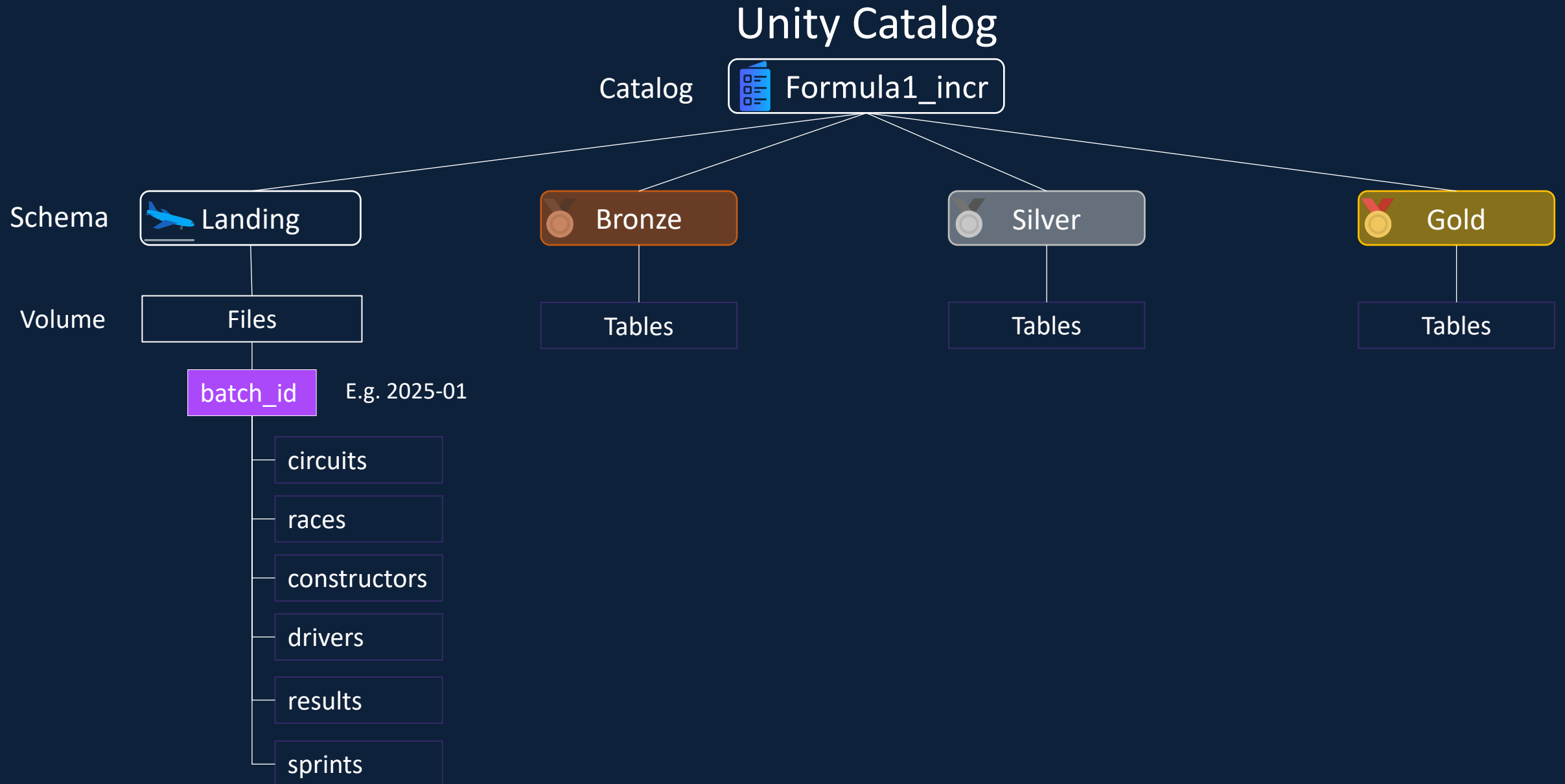
Azure Data Lake Storage Set-Up



Unity Catalog Set-up

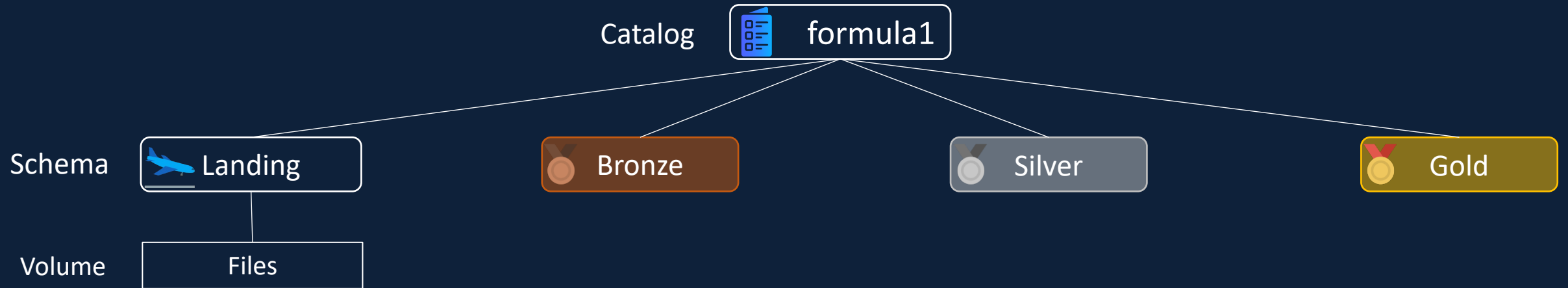
`databricks_course_ext_dl_formula1_incr`

Data Architecture



Data Architecture

Unity Catalog



Congratulations!
&
Thank you





Good Luck!



Ratings & Review



Thank you

Document History

Version	Date	Description
1	April 2026	Initial Version